

MATH3302

Coding and Cryptography

Coding Theory

2026

Contents

Chapter 1. Coding Theory Motivation	3
1.1. Some Examples	3
1.2. Informal Discussion	5
Chapter 2. Background Material	9
2.1. Notation & Review of Arithmetic Modulo 2	9
2.2. Distances and Weights	11
2.3. Floor and Ceiling	12
Chapter 3. Introduction to Coding	15
3.1. Basic Definitions & Setup	15
3.2. Error Detection	17
3.3. Error Correction	21
3.4. The Rate of a Code	24
3.5. Probability	25
3.6. Overall Success (Q_C) and Fail (F_C) Probabilities	27
3.7. Shannon's Theorem	29
Chapter 4. Linear Codes I	33
4.1. Review of Linear Algebra	33
4.2. Basic Properties of Linear Codes	35
4.3. Bases for Codes	36
4.4. Generating Matrices	37
4.5. Systematic and Equivalent Codes	40
4.6. Dual Codes	42
4.7. Parity Check Matrices	47
4.8. Error Detecting using the Parity Check Matrix	48
4.9. Distance of a Linear Code	50
4.10. Cosets	51
4.11. IMLD for Linear Codes	53
4.12. Encoding and Decoding Summary	58
4.13. Reliability of IMLD for Linear Codes	58
4.14. Generalising to larger fields: q -ary codes	60
Chapter 5. Bounds on Codes	63
5.1. The Singleton Bound	63
5.2. The Hamming Bound	64
5.3. The Griesmer Bound	65
5.4. The Gilbert–Varshamov Bound	68
5.5. Perfect Codes	70
5.6. Interlude: Sphere Packing	72
5.7. Constructing new codes from old	74
Chapter 6. Linear Codes II	79
6.1. Reed-Muller Codes	79
6.2. Hamming Codes	81

6.3. The Extended Golay Code	82
6.4. Decoding the Extended Golay Code	85
6.5. The Golay Code	89
Chapter 7. Cyclic Codes	91
7.1. Introduction to burst errors	91
7.2. Words Viewed as Polynomials	93
7.3. Introduction to cyclic codes	94
7.4. Generating matrices for linear cyclic codes	97
7.5. Finding a generator polynomial for a linear cyclic code	100
7.6. Error detection and correction using cyclic codes	102

CHAPTER 1

Coding Theory Motivation

Coding $\neq$ Cryptography

Cryptography = keeping information *secret*

Coding = keeping information *error-free*

The aim of cryptography is to convert (*encrypt*) a message from a readable state into apparent nonsense, in such a way that an authorized party can convert (*decrypt*) the apparent nonsense back into the original message, but an unauthorized party cannot.

In cryptography, an encryption + decryption algorithm form a *cipher* (not a *code*).

We will not be concerned with *secrecy* at all in the coding part of the course. Instead we will be focused on *accuracy*.

Coding theory is concerned with the *reliable and efficient transmission of information through unreliable channels*. Coding theory began with a 1948 paper of Shannon titled *A mathematical theory of communication*.

1.1. Some Examples

1.1.1 Example Hard Drives From www.pcguide.com



It is **normal** for a hard disk to encounter errors during reading. As hard disks are pushed to the limits of technology, with closer sectors, weaker magnetic signals (to prevent interference), and faster spin rates, the chances of an error occurring while reading the disk increase dramatically. It is **not practical to even try to avoid them**.

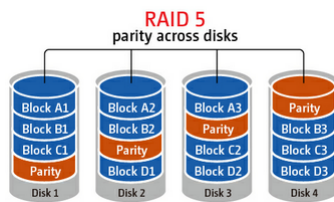
Since errors are unacceptable, hard disk makers use coding theory to *detect* and *correct* hard disk errors. This allows them to make drives that *appear* to the user to be error-free. The more the technology is pushed, the more sophisticated the error correction protocols must be.

When a sector is written to the hard disk, **extra error correcting code bits are generated and stored**. When the sector is read, **these extra bits can be used to indicate if any errors occurred during the read**. Where possible, errors are corrected using the redundant bits before passing the data to the rest of the system. This is done “on the fly” with no user intervention.

The same holds for solid state drives. In critical applications even RAM utilizes error correction.

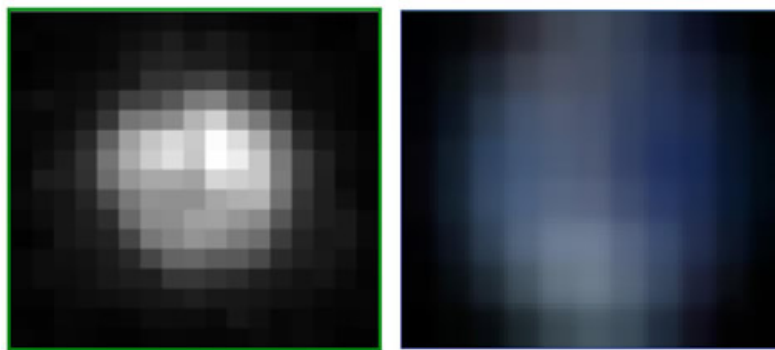
1.1.2 Example raid arrays

To increase reliability of storage, multiple devices can be linked together, using a *Redundant Array of Independent Disks* (raid). In a raid5 configuration, $1/4$ of the storage is set aside for redundancy. At the cost of losing $1/4$ of the storage space, even if one entire hard drive dies, no data¹ is lost. There are other variations, balancing robustness against efficiency.



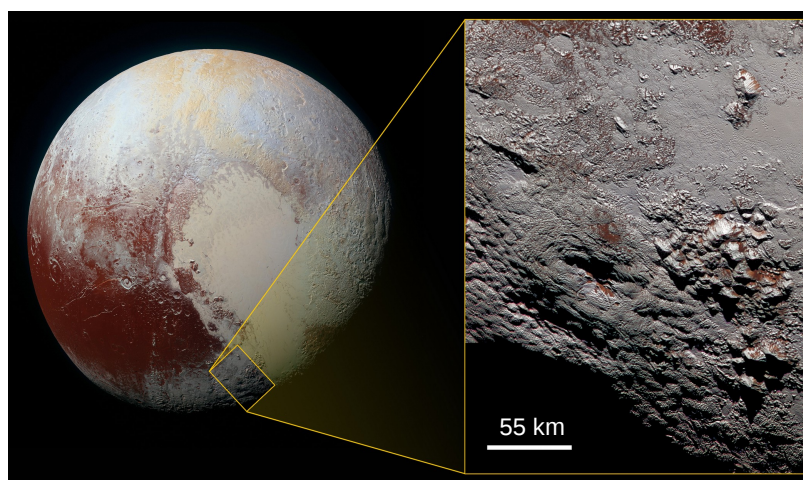
In each position $A, B, C, D, E, \dots$, 3 bits are used for data and the 4th bit, called the *parity check bit*, is the sum of the others (mod 2). If 1 of the 4 bits cannot be read, its value can be deduced from the remaining 3: Given $w = x + y + z$, and 3 of x, y, z, w it is easy to find the value of the remaining bit.

1.1.3 Example The *New Horizons* probe was launched in 2006, and flew by Pluto in July (2015). It transmitted 6.5 Gb of data 5 billion km back to Earth, using a transmitter similar in power to a fridge light. With such a faint signal² in the presence of background noise, many errors occurred in transmission. These were corrected using error correcting codes.



Left: Best pre-2015 picture of Pluto

Right: Earth at the same resolution.



¹This example is slightly different from the setup in most of the course, because if a hard drive cannot be read, we know where the error is. In general we shall not assume the location of error(s) can be detected via external means; detecting the error position(s) will be part of the job of the code

²The transmission took about 18 months, which works out to about 140 bits/second. Bits, not kb, Mb, ...

1.1.4 Example [DisplayPort] From extremetech & AnandTech:

The DisplayPort 1.4 standard was announced on 1/3/16, with support for video up to 7680×4320 resolution (8k) at 60 Hz.



DisplayPort 1.4 uses Display Stream Compression (dsc) to compress the signal. Corruption errors in the encoded data will wreck the decoded output in a way that users will immediately notice. An error in a compressed data chunk will cause decoding to fail or make the decoded result wrong over a large number of pixels, making the error far more noticeable.

For that reason, DisplayPort 1.4 adds error correction for the first time. Although error correction increases bandwidth requirements slightly, the additional, redundant data it carries allows for errors to be corrected, making dsc suitably reliable over DisplayPort connections. This is the key change that allows dsc to be used.

[Note that if data is compressed or encrypted, and then even a single bit error occurs, this may translate to a huge effect once the decompression/ decryption is applied.]

In summary: the primary goals of coding theory are to provide:

- (a) fast encoding of information;
- (b) easy transmission of encoded messages;
- (c) fast decoding of received message;
- (d) detection and correction of errors introduced in the channel;
- (e) maximum transfer of information per unit time.

These goals are not necessarily compatible! Goal (d) is where we will spend most of our discussion.

1.2. Informal Discussion

[This section is deliberately a bit vague. Precise definitions follow later.]

Data can be stored in many different formats. DNA can be viewed as data written with an alphabet with 4 letters C, A, G, T. English is written using a 26-letter alphabet.

However, throughout these notes, we will assume that whatever data we wish to send has been transformed into a binary sequence. Thus we assume our data is a (long) string of 0s and 1s.

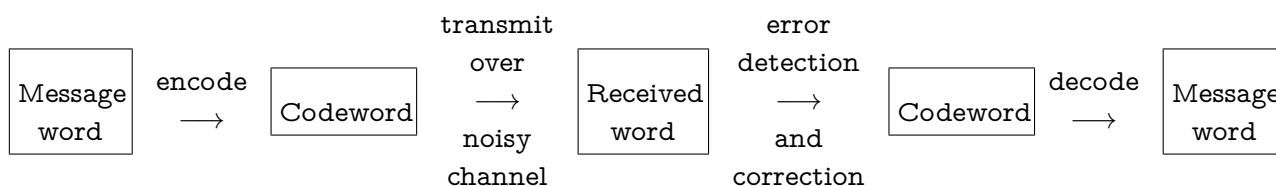
Since we do not know how long a message may be, we chop our long binary sequence into pieces of some fixed length k , and deal with each piece independently. So, we assume all the messages to be sent are binary sequences (=words = vectors) of length k . Such a binary word containing data to be transmitted is called a *message word*.

We wish to send our data across a *channel* : email, telephone line, cable, move from one hard drive to another etc. We assume the channel may be *noisy*. This means that *errors* may occur during transmission. An *error* occurs whenever a transmitted 0 is received as a 1, or a 1 is received as a 0.

To compensate for errors, we *encode* a message word of length k by (somehow) adding extra bits containing error correcting information. The result is called a *codeword* of length $n \geq k$. The set of all codewords is called a *binary block code*.

Further assume:

- ★ Any codeword of 0s and 1s of length n will be received as a word of length n containing 0s and 1s, not necessarily the same as the original codeword. No extra bits appear. No bits disappear. (If a bit cannot be read at all, just fill in that bit randomly; then this bit may well be an error.)
- ★ It is easy to identify the beginning of the first word, so we can identify each received word.



A simple way to detect/correct errors is to transmit the information multiple times.

1.2.1 Example A 3-fold repetition code : Suppose we have four *message words*: 00, 01, 10, 11, and we *encode* them by listing them three times to get the four *codewords* 000000, 010101, 101010, 111111. The *code* is the set C of these four words $C = \{000000, 010101, 101010, 111111\}$. Here $k = 2$ and $n = 6$. Important: no other word except a codeword is ever transmitted.

We transmit these codewords of length 6 over a noisy channel. It is possible that error(s) occur during transmission, altering some 0s to 1s, or vice versa. After transmission, it is possible that we receive a word which is not in the code. In that case, we need to guess which of the codewords was most likely to have been sent.

Suppose the word 011101 is received. Since 011101 does not match any of the codewords and only codewords are ever transmitted, at least one error must have occurred. We have *detected* an error. We compare 011101 to each of the codewords to see which one was most likely to have been sent.

Codeword	0	0	0	0	0	0	# errors
Received	0	1	1	1	0	1	4
Codeword	0	1	0	1	0	1	
Received	0	1	1	1	0	1	1
Codeword	1	0	1	0	1	0	
Received	0	1	1	1	0	1	5
Codeword	1	1	1	1	1	1	
Received	0	1	1	1	0	1	2

Under the assumption that errors occur infrequently, it seems most likely(?) that the word actually sent was 010101. So we *correct* the received word $011101 \mapsto 010101$, corresponding to message word 01. We have *corrected* 1 error.

We cannot rule out the possibility that more errors occurred, and some other codeword was sent. In general, there is no guarantee in coding theory that we correctly recover the message that was

sent. We can only recover the codeword that was most likely sent given the received word. This will require some consideration of probabilities.

There are various points to clarify:

- (a) How much redundant information should be added, and in what form?
- (b) How many errors can we detect?
- (c) If errors do occur, how should we correct them, and recover a codeword?
- (d) How many errors can we correct?

Puzzle Bob is trying to work out Alice's birthday, so he can buy her a surprise present. First he wants to guess the day of the month of Alice's birthday $\in \{1, 2, \dots, 31\}$. He asks her a series of questions with yes/no answers.

- (a) Assuming Alice answers truthfully, how many questions (max) does Bob need? (Easy)
- (b) Suppose that Alice may lie on ≤ 1 of her answers, and tell the truth on the others. Now how many questions does Bob need? (Not so easy)
- (c) What has this to do with coding theory?

CHAPTER 2

Background Material

Before we start coding theory formally, we review some background material.

2.1. Notation & Review of Arithmetic Modulo 2

Our messages will be represented by strings of 0s and 1s.

2.1.1 Definition Let $\mathbb{F}_2 = \{0, 1\}$. We define addition (denoted $+$) and multiplication on $\mathbb{F}_2$ by defining $1 + 1 = 0$. The full tables for addition and multiplication are:

$+$	0	1		$\bullet$	0	1
0	0	1		0	0	0
1	1	0		1	0	1

This is called the set of *integers modulo 2*, or *integers mod 2*, or the *finite field* with 2 elements. (Other notation: $\mathbb{Z}_2$, or $\mathbb{Z}/2\mathbb{Z}$.)

Note: if 0 and 1 represent false and true respectively, multiplication is logical AND, while addition is exclusive or, XOR.

The set

$$\mathbb{F}_2^n = \mathbb{F}_2 \times \cdots \times \mathbb{F}_2 = \left\{ (x_1, x_2, \dots, x_n) \mid x_1, x_2, \dots, x_n \in \mathbb{F}_2 \right\}$$

consists of all n -tuples $(x_1, \dots, x_n)$ with each $x_i \in \mathbb{F}_2$. These are vectors (strings/ words/ sequences), not binary numbers. We write $\mathbf{0}_n$ or just $\mathbf{0}$ for $(0, 0, \dots, 0) \in \mathbb{F}_2^n$. We write $\mathbf{1}_n$ or $\mathbf{1}$ for $(1, 1, \dots, 1) \in \mathbb{F}_2^n$. The individual entries x_i are sometimes called *bits*.

We can add (but not multiply) elements of $\mathbb{F}_2^n$:

$$(x_1, \dots, x_n) + (y_1, \dots, y_n) = (x_1 + y_1, \dots, x_n + y_n).$$

There is no “carry” when we add. We add in each coordinate (bit) independently.

2.1.2 Definition If $u = (u_1, u_2, \dots, u_n)$, and $v = (v_1, v_2, \dots, v_n) \in \mathbb{F}_2^n$ their *dot product* $u \cdot v$ is defined to be $\sum_{i=1}^n u_i v_i \in \mathbb{F}_2$. If $u \cdot v = 0 \in \mathbb{F}_2$ then u and v are said to be *orthogonal* or *perpendicular*. We write $u \perp v$.

(Occasionally it is convenient to view the calculation $\sum_{i=1}^n u_i v_i$ as taking place in $\mathbb{Z}$. Context should make this clear.)

2.1.3 Example $(1, 0, 1) \cdot (1, 1, 1) = 1 + 0 + 1 = 0$ so $(1, 0, 1) \perp (1, 1, 1)$.

We often omit the commas and parentheses:

2.1.4 Example

★ $010 \in \mathbb{F}_2^3$.

$$\star \quad \mathbb{F}_2^2 = \{00, 01, 10, 11\}$$

$$\star \quad \mathbb{F}_2^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}.$$

Note that $\mathbb{F}_2^n$ is a set with 2^n elements.

2.1.5 Definition A finite sequence of 0s and 1s is called a *word*. The *length* of a word is the number of symbols (bits) in the word. Thus $\mathbb{F}_2^n$ is the set of words of length n .

We may define a word w as the *concatentation* of two shorter words u, v .

2.1.6 Definition $w = (u \mid v)$ means: w is the word consisting of all the bits of u followed by the bits of v .

2.1.7 Example If $u = 110$ and $v = 011$ then $(u \mid v) = 110011$.

2.1.8 Definition In $\mathbb{F}_2^n$ we let θ_j denote the word that has a 1 in position j , and 0s elsewhere.

2.1.9 Example In $\mathbb{F}_2^3$ we have words $\theta_1 = 100$, $\theta_2 = 010$, $\theta_3 = 001$.

2.1.10 Definition If $A, B \subseteq \mathbb{F}_2^n$ we write

$$A + B = \{a + b \mid a \in A, b \in B\}$$

Thus $A + B \subseteq \mathbb{F}_2^n$. If B consists of a single element $B = \{b\}$ we write $A + b$ instead of $A + \{b\}$.

We can also form polynomials with $\mathbb{F}_2$ -coefficients. The set of all such polynomials is denoted $\mathbb{F}_2[x]$. We add and multiply such polynomials in the normal way, using the addition and multiplication in $\mathbb{F}_2$ on the coefficients.

2.1.11 Example

$\star$ $x^2, x^2 + 1, x^2 + x, x^2 + x + 1 \in \mathbb{F}_2[x]$ are degree 2 polynomials. In fact, they are all the degree 2 polynomials in $\mathbb{F}_2[x]$.

$$\star \quad (x^2 + x + 1) + (x^2 + 1) = (x^2 + x^2) + x + (1 + 1) = (1 + 1)x^2 + x + (1 + 1) = x.$$

$$\star \quad (x + 1)^2 = (x + 1)(x + 1) = x^2 + x + x + 1 = x^2 + (1 + 1)x + 1 = x^2 + 1.$$

$$\star \quad \text{Check that } (x^3 + x + 1)(x^2 + x + 1) = x^5 + x^4 + 1.$$

Identities valid in $\mathbb{F}_2[x]$ may not hold for polynomials with integer coefficients. Eg $(x + 1)^2 = x^2 + 1$.

In general, if $a \in \mathbb{F}_2$ or $\mathbb{F}_2^n$ or $\mathbb{F}_2[x]$ then¹ $a + a = 0$. So $a = -a$. And if

$$\begin{aligned} c &= a + b, & \text{then adding } a \text{ to each side:} \\ a + c &= a + a + b \\ &= 0 + b = b. \end{aligned}$$

So $c = a + b \implies b = c + a$. Effectively addition is the same as subtraction, modulo 2.

¹Here 0 denotes the element 0, or the zero vector $\mathbf{0}$ or the zero polynomial.

2.2. Distances and Weights

Let $a, b \in \mathbb{F}_2$. Considering the four possibilities $a = 0, b = 0$, $a = 0, b = 1$, $a = 1, b = 0$, $a = 1, b = 1$, we see:

$$(2.2.1) \quad a + b = 0 \iff a = b \quad a + b = 1 \iff a \neq b.$$

2.2.1 Lemma If v, w are words in $\mathbb{F}_2^n$ then $v + w$ is a word containing a 1 in exactly those places in which v and w differ, and a 0 in those places they agree. In particular, $v + w = \mathbf{0}$ iff $v = w$.

Proof Let $v = (v_1, \dots, v_n)$, $w = (w_1, \dots, w_n)$ and $u = v + w = (u_1, \dots, u_n)$. Thus $u_i = v_i + w_i$ for $1 \leq i \leq n$. By Equation 2.2.1, $u_i = 1$ exactly when $v_i \neq w_i$ and $u_i = 0$ exactly when $v_i = w_i$. The last statement follows. $\square$

2.2.2 Definition Let $v \in \mathbb{F}_2^n$ be a word of length n . The *weight* or *Hamming weight* of v , denoted $\|v\|$, is the number of occurrences of the bit 1 in v . Note that $0 \leq \|v\| \leq n$.

2.2.3 Lemma If $v \in \mathbb{F}_2^n$, then

$$(2.2.2) \quad \|v + \mathbf{1}_n\| = n - \|v\|$$

Proof By Lemma 2.2.1 $v + 11 \cdots 1$ contains a 1 exactly where v contains a 0. The number of 0s in v is $n - \|v\|$. $\square$

2.2.4 Example Let $v = 10110001$, $w = 10111010$. Calculate $\|v\|$, $\|w\|$, $\|v + w\|$.

Solution: We just add in each coordinate independently (no “carries”), so $v + w$ is

v	1	0	1	1	0	0	0	1
w	1	0	1	1	1	0	1	0
$v + w$	0	0	0	0	1	0	1	1

So $\|v\| = 4$, $\|w\| = 5$, $\|v + w\| = 3$. The word $v + w$ has 1s exactly where v and w differ, and 0s where they agree. $\circledast$

In the previous example we had $\|v + w\| \neq \|v\| + \|w\|$. In fact:

2.2.5 Lemma

- (a) $\|u + v\| \leq \|u\| + \|v\|$
- (b) $\|u + v\| \equiv \|u\| + \|v\| \pmod{2}$
- (c) If $u \perp v$ then $\|u + v\| \equiv \|u\| + \|v\| \pmod{4}$.

Proof Let $i_1, i_2, \dots, i_A$ be the positions where u and v both have 0s. Let $j_1, \dots, j_B$ be the positions where u has a 0 and v has a 1. Let $k_1, \dots, k_C$ be the positions where u has a 1 and v has a 0. Let $\ell_1, \dots, \ell_D$ be the positions where u and v both have 1s.

Then $u + v$ has 1s in positions $j_1, \dots, j_B$ and $k_1, \dots, k_C$. So $\|u + v\| = B + C$.

But $\|u\| = C + D$ and $\|v\| = B + D$, so $\|u\| + \|v\| = B + C + 2D$. Hence

$$(\clubsuit) \quad \|u\| + \|v\| = \|u + v\| + 2D.$$

(a) and (b) follow from $(\clubsuit)$.

(c) By definition, $u \cdot v$ is a sum of terms that are all 0 except in places where both u and v have 1s. So $u \cdot v$ is the sum of D 1s. Therefore by hypothesis, $D \equiv 0 \pmod{2}$. Let $D = 2E$. Substituting into (♣) gives $\|u\| + \|v\| = \|u + v\| + 4E \equiv \|u + v\| \pmod{4}$. $\square$

2.2.6 Definition The *Hamming distance* $d(u, v)$ between two words $u, v \in \mathbb{F}_2^n$ is the number of places in which their bits differ.

By Lemma 2.2.1

$$(2.2.3) \quad d(u, v) = \|u + v\|.$$

2.2.7 Example Let $x = 111001$ and $y = 001111$. Find $d(x, y)$, $x + y$, and $\|x + y\|$.

Solution: $d(x, y) = 4$,

$$\begin{array}{r|cccccc} x & 1 & 1 & 1 & 0 & 0 & 1 \\ + y & 0 & 0 & 1 & 1 & 1 & 1 \\ \hline = & 1 & 1 & 0 & 1 & 1 & 0 \end{array}$$

$$\|x + y\| = 4 = d(x, y).$$

⊗

The next result says that “Hamming distance” behaves like a distance measuring function, ie satisfies the properties of a *metric*.

2.2.8 Theorem If $x, y, z \in \mathbb{F}_2^n$ then

- (a) $d(x, y) \geq 0$
- (b) $d(x, y) = 0$ iff $x = y$
- (c) $d(x, y) = d(y, x)$ (symmetry)
- (d) $d(x, z) \leq d(x, y) + d(y, z)$ (the triangle inequality).

Proof (a) Obvious.

(b), (c) These follow from Lemma 2.2.1 together with Equation 2.2.3.

(d) Using Lemma 2.2.5 $d(x, z) = \|x + z\| = \|x + y + y + z\| \leq \|x + y\| + \|y + z\| = d(x, y) + d(y, z)$. $\square$

2.3. Floor and Ceiling

We discuss two useful functions we shall use in the rest of the course.

Coding theory is part of discrete mathematics. Sometimes it is useful to view the parameters in codes as continuous variables, so that we can apply techniques from analysis. At the end we convert the final result back to an integer. There are two commonly used ways to convert a real number to an integer:

2.3.1 Definition For $x \in \mathbb{R}$ the *floor* of x , denoted $\lfloor x \rfloor$, is the largest integer m satisfying $m \leq x$. The *ceiling* of x , denoted $\lceil x \rceil$, is the smallest integer m satisfying $m \geq x$.

2.3.2 Example

- ★ $\lceil \pi \rceil = 4, \quad \lfloor \pi \rfloor = 3.$
- ★ $\lceil 3.98 \rceil = 4, \quad \lfloor 3.98 \rfloor = 3.$
- ★ $\lceil 3 \rceil = 3 = \lfloor 3 \rfloor.$

Clearly for any real number x we have

$$\lfloor x \rfloor \leq x \leq \lceil x \rceil$$

For every integer m we have $\lfloor m \rfloor = m = \lceil m \rceil$. To prove results about the floor and ceiling, note that if m is an integer and $m < x < m + 1$ then $\lfloor x \rfloor = m$ and $\lceil x \rceil = m + 1$.

2.3.3 Lemma Suppose N is some integer quantity we want to bound. If we have a bound $N \leq y$ where $y \in \mathbb{R}$, then $N \leq \lfloor y \rfloor$. Similarly $N \geq y \implies N \geq \lceil y \rceil$.

Proof Obvious. □

Slightly less trivially:

2.3.4 Lemma Let a, b be integers. Then $a < \frac{b}{2} \iff a \leq \left\lfloor \frac{b-1}{2} \right\rfloor$.

Proof $\implies$ If $a < \frac{b}{2}$ then $2a < b$ so $2a \leq b-1$ so $a \leq \frac{b-1}{2}$. But a is an integer, so $a \leq \left\lfloor \frac{b-1}{2} \right\rfloor$ (by Lemma 2.3.3).

$\impliedby$ If $a \leq \left\lfloor \frac{b-1}{2} \right\rfloor \leq \frac{b-1}{2}$ then $2a \leq b-1 < b$, so $a < \frac{b}{2}$. □

The floor and ceiling functions are not very easy to work with:

2.3.5 Example If $x = 2.1$ then $\lceil 2x \rceil = \lceil 4.2 \rceil = 5$, but $2\lceil x \rceil = 2\lceil 2.1 \rceil = 6$. So $\lceil 2x \rceil \neq 2\lceil x \rceil$ in general. Similarly, $\lfloor x+y \rfloor \neq \lfloor x \rfloor + \lfloor y \rfloor$, etc.

Despite the previous example, we do have the following results:

2.3.6 Lemma Let n be a positive integer. Then $\left\lceil \frac{n}{2} \right\rceil + \left\lfloor \frac{n}{2} \right\rfloor = n$.

Proof If n is even we can write $n = 2k$ for some integer k . Then the lhs = $\lceil k \rceil + \lfloor k \rfloor = 2k = n$. If n is odd, write $n = 2k + 1$. In this case the lhs = $\left\lceil k + \frac{1}{2} \right\rceil + \left\lfloor k + \frac{1}{2} \right\rfloor = (k+1) + k = 2k + 1 = n$. □

2.3.7 Lemma Let m, n, x be positive integers. Then $\left\lceil \frac{\lfloor x/m \rfloor}{n} \right\rceil = \left\lceil \frac{x}{mn} \right\rceil$.

Proof We need to know the approximate size (to the nearest integer) of x/m and x/mn . Write² $x = Q(mn) + R$ for integers Q, R with $Q \geq 0$ and $1 \leq R \leq mn$. Then we have $Q < Q + R/mn = x/mn$ and $x/mn = Q + R/mn \leq Q + 1$, so $\lceil x/mn \rceil = Q + 1$.

Write $R = qm + r$ for integers q, r with $q \geq 0$ and $1 \leq r \leq m$. Since $qm < qm + r = R \leq mn$ we have $q < n$, so $q + 1 \leq n$. Now, $x = Qmn + (qm + r)$ implies $Qn + q < x/m$, and $x/m = Qn + q + r/m \leq Qn + q + 1$ so $\lceil x/m \rceil = Qn + q + 1$. Thus $Q < \frac{\lfloor x/m \rfloor}{n}$ and $\frac{\lfloor x/m \rfloor}{n} = Q + (q+1)/n \leq Q + 1$, so $\left\lceil \frac{\lfloor x/m \rfloor}{n} \right\rceil = Q + 1$ also. □

2.3.8 Example Let $x = 102, m = 5, n = 6$. To find $\left\lceil \frac{x}{mn} \right\rceil = \left\lceil \frac{102}{30} \right\rceil$, observe that $102 = 3 \cdot 30 + 12$ so $3 \cdot 30 = 90 < 102 < 4 \cdot 30$, so $3 < \frac{102}{30} < 4$, so $\left\lceil \frac{102}{30} \right\rceil = 4$.

Similarly $\left\lceil \frac{\lfloor x/m \rfloor}{n} \right\rceil = \left\lceil \frac{\lfloor 102/5 \rfloor}{6} \right\rceil = \left\lceil \frac{21}{6} \right\rceil = 4$.

²Normally we would require the remainder to satisfy $0 \leq R < mn$, but remainder 0 can be viewed as remainder mn , by adjusting Q by 1.

2.3.9 Lemma If $m \in \mathbb{Z}$ and $\theta \in \mathbb{R}$, then

$$(2.3.1) \quad \lfloor m - \theta \rfloor = m - \lceil \theta \rceil \quad \text{and} \quad \lceil m - \theta \rceil = m - \lfloor \theta \rfloor.$$

Proof Write $\theta = n + \phi$ where $n \in \mathbb{Z}$ and $0 \leq \phi < 1$. If $\phi = 0$, then $\theta \in \mathbb{Z}$ and $m - \theta \in \mathbb{Z}$ so

$$\lfloor m - \theta \rfloor = \lceil m - \theta \rceil = m - \lceil \theta \rceil = m - \lfloor \theta \rfloor.$$

Otherwise, $0 < \phi < 1$ and

$$\lfloor m - \theta \rfloor = \lfloor m - n - \phi \rfloor = m - n - 1 = m - (n + 1) = m - \lceil \theta \rceil$$

and

$$\lceil m - \theta \rceil = \lceil m - n - \phi \rceil = m - n = m - \lfloor \theta \rfloor.$$

□

CHAPTER 3

Introduction to Coding

3.1. Basic Definitions & Setup

We give the definition of a binary block code, which we just call a code.

3.1.1 Definition Let $n \geq k$ be positive integers. A (binary block) code consists of:

- ★ A set $M \subseteq \mathbb{F}_2^k$ of message words together with
- ★ An injective function called the *encoding function* $f: M \rightarrow \mathbb{F}_2^n$.

We write $C = f(M)$, the image of f , and (abusing terminology) call C the *code*. Elements in C are called *codewords*. The *length* of the code is n , and the *message length* or *dimension* of the code is k .

In fact we will always just take $M = \mathbb{F}_2^k$.

A code C is an (n, k) -code if it has codewords of length n and $|C| = 2^k$.

The encoding function viewed as a map $f: M \rightarrow C$ is a bijection, so $|M| = |C|$. The inverse function $f^{-1}: C \rightarrow M$ is the *decoding function*.

In general, if $f: M \rightarrow \mathbb{F}_2^n$ is a code with $f(M) = C$ we shall often refer only to C , and omit explicit mention of f when it can be deduced. Thus we simply say that a code C is a subset of $\mathbb{F}_2^n$. However in the background there is always a function $f: M \rightarrow C$.

3.1.2 Definition Many (but not all) codes are obtained by taking message words of length k and (somehow) adding another $n - k$ check bits to each word, thus giving codewords of length n .

The encoding function in this case is

$$(3.1.1) \quad m = \overbrace{** \dots *}^k \mapsto \overbrace{** \dots *}^k \overbrace{\bullet \bullet \dots \bullet}^{n-k}.$$

Note: not all codes work this way. In general the encoding process $M \rightarrow C$ can be any injective function, so it does not have to keep any of the bits of a message word fixed.

3.1.3 Example What are n, k, f for the 3-fold repetition code C in Example 1.2.1?

Solution: C is a $(6, 2)$ -code. The encoding function $f: \mathbb{F}_2^2 \rightarrow \mathbb{F}_2^6$ is: $f(00) = 000000$, $f(01) = 010101$, $f(10) = 101010$, $f(11) = 111111$. We refer simply to the code $C = f(M) = \{000000, 010101, 101010, 111111\}$.

In this example the first two bits encode the message and the remaining four are check bits. $\otimes$

3.1.4 Definition An m -fold repetition code is formed by taking the 2^k message words of length k and forming the codewords of length km by writing down each word of length k a total of m times. (Compare Example 1.2.1).

3.1.5 Example Let $M = \mathbb{F}_2^3$, and let $f: \mathbb{F}_2^3 \rightarrow \mathbb{F}_2^4$ be defined by

$$f(x_1, x_2, x_3) = (x_1, x_2, x_3, x_1 + x_2 + x_3).$$

This is similar¹ to the raid5 example. The last bit is called a (*parity*) *check bit*.

Explicitly: $f(000) = 0000$, $f(001) = 0011$, $f(010) = 0101$, $f(011) = 0110$, $f(100) = 1001$, $f(101) = 1010$, $f(110) = 1100$, $f(111) = 1111$. So $C = \{0000, 0011, 0101, 0110, 1001, 1010, 1100, 1111\}$.

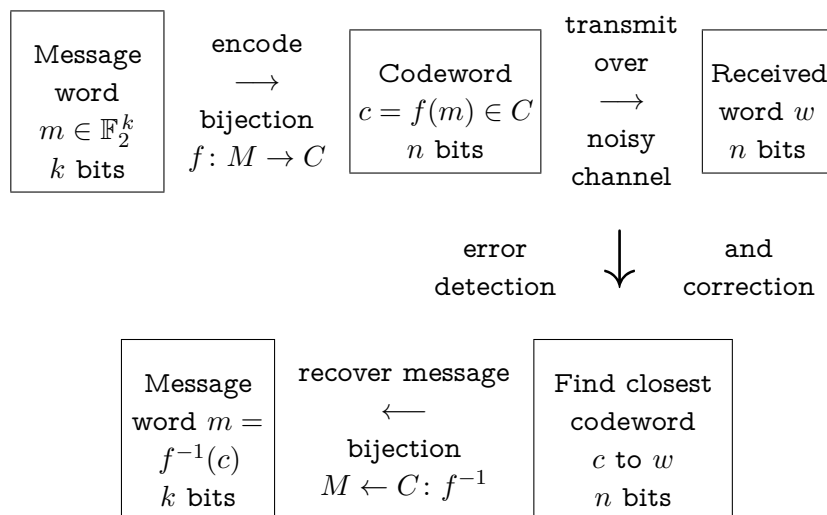
Exercise: Convince yourself that in this code, every word has even weight. If a single bit of a codeword is changed, the result will be an odd weight word, and we will detect an error.

Encoding & Decoding Process

Consider an (n, k) -code, $f: \mathbb{F}_2^k \rightarrow C$.

Encode each message word before transmission. Only codewords will ever be transmitted. Therefore, if any non-codeword is received, error(s) must have occurred.

- ★ Suppose we receive a word w (with n bits).
- ★ If w is a codeword ($w \in C$), assume no errors occurred. Recover the message word $m = f^{-1}(w)$.
- ★ If w is not a codeword ($w \notin C$) one or more errors must have occurred. If there is a unique nearest codeword c to w , assume c was the word transmitted. Recover the message word $m = f^{-1}(c)$.

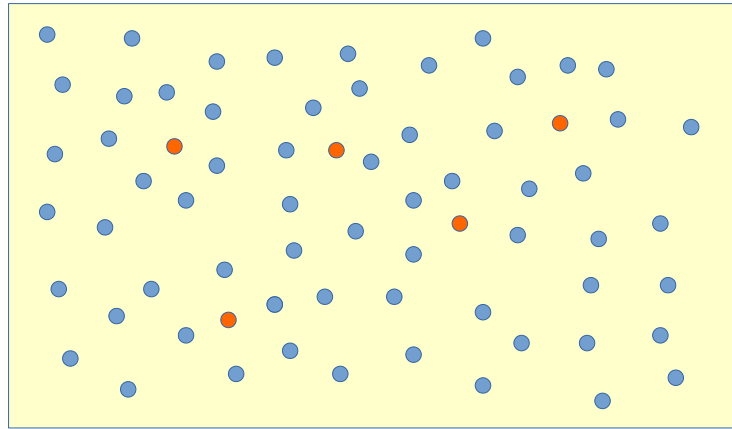


“Nearest codeword” means at smallest Hamming distance. At the last step, if there are two or more equally close codewords, we cannot decode successfully. Formally:

3.1.6 Definition *Incomplete Maximum Likelihood Decoding* or *IMLD*. Let C be the set of codewords, and w the word received. If there is a unique codeword $c \in C$ such that $d(w, c) < d(w, b)$ for any other $b \in C$, $b \neq c$, then we decode w as c . If there are two or more equally close codewords to w , we request retransmission (=decoding fails).

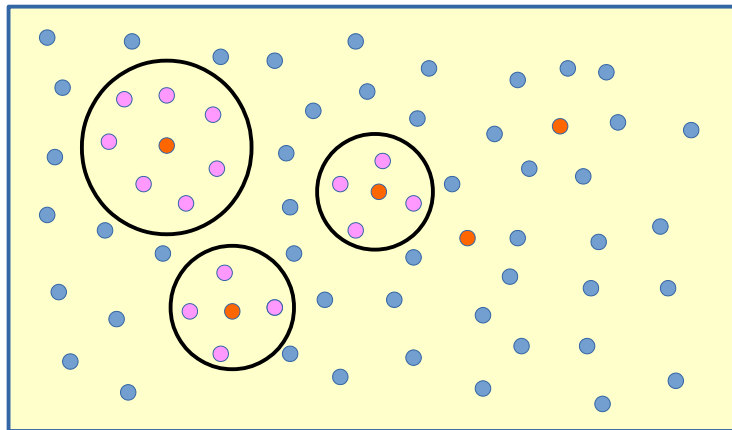
Unless stated otherwise, we will always assume that IMLD is being used.

¹But not quite the same. The way the raid5 example was described, if a bit is corrupted, the hard drive will notify the raid controller, so the position where the error occurs is known. This is probably an over simplification.



Codewords in orange.

Non-codewords in blue.



Codewords in orange. Blue and pink non-codewords. Pink words are decoded to nearest orange word.

3.1.7 Definition The *trivial code* consists of taking the encoding function f to be the identity map.

3.1.8 Example Let $M = \mathbb{F}_2^3$ and take f to be the identity map $\mathbb{F}_2^3 \rightarrow \mathbb{F}_2^3$. We “encode” message words of length 3 by ... doing nothing.

This code is useless for error correction, because no matter what errors occur, we will always receive a codeword (since everything is a codeword). So we cannot detect or correct any errors.

The trivial code example indicates that we should try to choose the encoding function f in such a way that the “distance” between all the codewords is large, so that the codewords are “spread out” from one another.

3.2. Error Detection

Recall that only codewords are ever sent. Suppose a word is received that is not a codeword. Then we have *detected* that at least one error has occurred in transmission.

3.2.1 Definition Suppose that the codeword $c \in \mathbb{F}_2^n$ was transmitted, and the word w was received. The corresponding *error pattern* or *error* is defined to be the word e for which $c + e = w$.

By Lemma 2.2.1 the error pattern e is 1 in exactly those places in which c and w differ. Thus $\|e\|$ is the number of errors that occurred. Note that $w = c + e \iff e = c + w \iff c = w + e$.

3.2.2 Definition A code C detects an error pattern $e \neq 0 \iff c + e$ is not a codeword, for every $c \in C$. If $c + e \in C$ for some $c \in C$ then we say C cannot detect error pattern e (meaning: at least sometimes, cannot detect.)

Let $e \neq 0$ be an error pattern.

- ★ If $c + e \in C$ ever occurs (for any $c \in C$) then C cannot detect e .
- ★ To show that C can detect e , we must show that $c + e \notin C$ for every $c \in C$.

Recall the 3-fold repetition code C , Example 1.2.1. Suppose the message was 10. The corresponding codeword is 101010. Suppose errors occur at positions 2, 4 and 6. That is error pattern 010101. $\therefore c = 101010$ sent + error $e = 010101$ gives $w = 111111$ received. But 111111 is also a codeword. If we received this word we would assume (incorrectly) that no errors had occurred. So this particular code C cannot (always) detect the error pattern 010101.

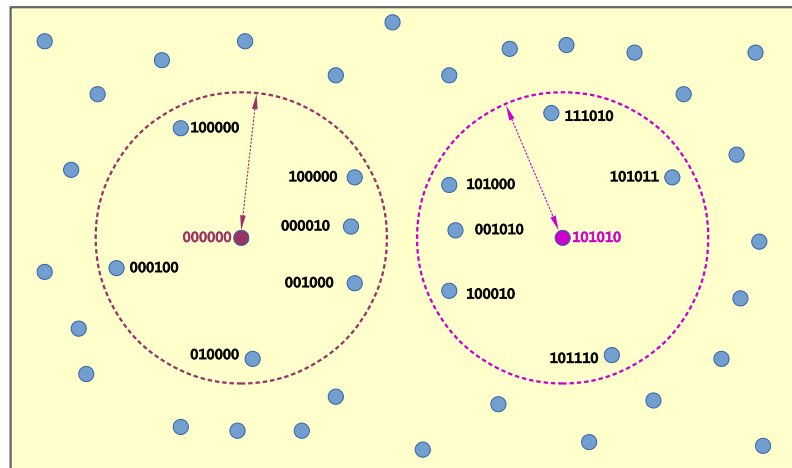
3.2.3 Example Again let C be the 3-fold repetition code, Example 1.2.1. Suppose the message was 10, and suppose exactly 1 error occurred during transmission. Find all the possible received words.

Solution: The codeword is 101010. If one error occurs, the error pattern will have weight 1. There are 6 words of weight 1: 000001, 000010, 000100, 001000, 010000 and 100000.

Error pattern e	$c + e$	$w = c + e$	$w \in C$?
000001	101010 + 000001 =	101011	☒
000010	101010 + 000010 =	101000	☒
000100	101010 + 000100 =	101110	☒
001000	101010 + 001000 =	100010	☒
010000	101010 + 010000 =	111010	☒
100000	101010 + 100000 =	001010	☒

The 6 possible received words lie in a “sphere” or “ball” of radius 1, centred on 101010. Since none of these 6 words is a codeword, if we receive any of them we will detect that an error has occurred. ⊗

In the previous example we can perform a similar calculation for each of the 4 codewords. If the codeword is 000000 and 1 error occurs the received word will be $000000 + 000001 = 000001$ or $000000 + 000010 = 000010$ etc. The 6 words at distance 1 from 000000 are 000001, 000010, 000100, 001000, 010000 and 100000. None of these are codewords. Similarly the spheres of radius 1 centred on the other two codewords do not contain any other codeword. So all error patterns of weight 1 will be detected successfully, in this code.



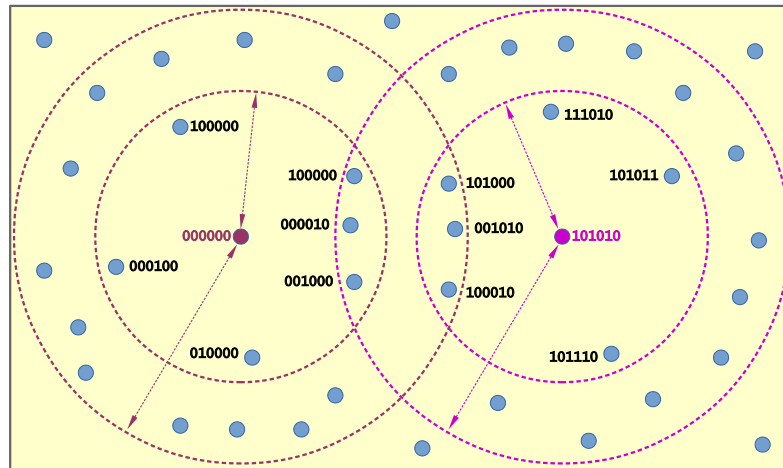
Spheres of radius 1 centred on 000000 and 101010.

3.2.4 Example How many error patterns are there of weight 2 in $\mathbb{F}_2^6$? How many error patterns of weight k in $\mathbb{F}_2^n$?

Solution: We have to choose 2 positions out of 6 to put the 1s, so $\binom{6}{2} = 15$.

In general there are $\binom{n}{k}$ error patterns of length n and weight k . ⊗

Still with C the 3-fold repetition code, we can draw the spheres of radius 2 in C centred on each codeword. The spheres centred on 000000 and 101010 look like this:



Spheres of radius 1 and 2 centred on 000000 and 101010.

One checks (by tedious calculation) that in the 3-fold repetition code C , the spheres of radius 2 centred on each of the 4 codewords of C contain no other codeword: Add each of the weight 2 error patterns to each of the 4 codewords and check the result is never a codeword. So C detects all error patterns of weight 2. (We shall soon develop a much faster method of showing this.)

However, a sphere of radius 3 centred on a codeword does contain another codeword. This means if the error pattern has weight 3 then we may no longer be able to detect it. We saw this above with $e = 010101$.

Some error patterns of weight 3 are detectable. Eg $000000 + 000111 = 000111$, $010101 + 000111 = 010010$, $101010 + 000111 = 101101$, $111111 + 000111 = 111000$, and none of the rhs are codewords. So 000111 is detectable.

In summary, the 3-fold repetition code C will successfully detect 1 or 2 errors, but if 3 or more errors occur the code will (sometimes) “not notice”.

3.2.5 Definition A code C is said to be t -error detecting if it detects all non-zero error patterns of weight $\leq t$.

So we can summarize all the calculations with the 3-fold repetition code C above by saying: C is 2-error detecting, but C is not 3-error detecting.

In general, given a code C , the largest value of t for which C is t -error detecting will depend on how far apart the codewords of C are.

3.2.6 Definition The *distance* (or *minimum distance*) δ of a code is the smallest distance between any pair of distinct codewords:

$$\delta = \min_{b \neq c} d(b, c) = \min_{b \neq c} \|b + c\| \quad \forall b \neq c \in C.$$

An (n, k) -code with minimum distance δ will be described as an (n, k, δ) -code.

The worst case scenario for error detection is when c is a codeword, and $e \neq 0$ is an error pattern such that $c + e = c'$ is another codeword. Then error pattern e will not be detected by the code. We hope that this does not happen unless many errors occur. That is, unless e has high weight. But $\|e\| = \|c + c'\| = d(c, c') \geq \delta$. So the weight of the [lowest weight undetectable error pattern] is $\geq \delta$.

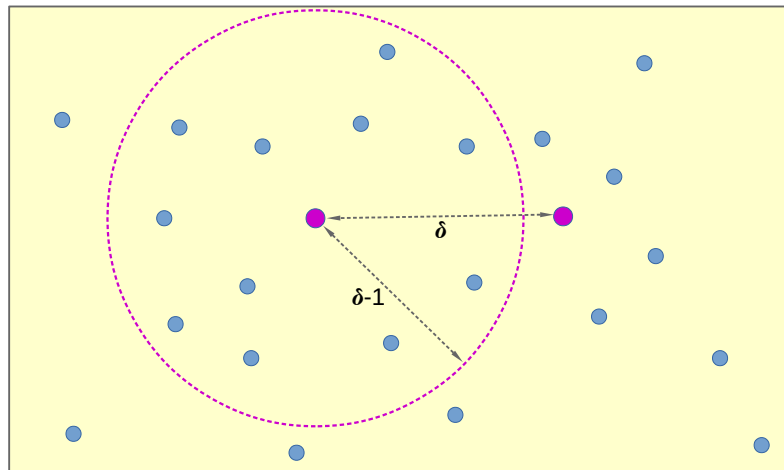
This gives our first theorem.

3.2.7 Theorem Let C be a code with minimum distance δ . Then C will detect all non-zero error patterns of weight $< \delta$. There is at least one error pattern of weight δ that C will not detect. That is, C is $(\delta - 1)$ -error detecting, but not δ -error detecting.

Proof Let e be a non-zero error pattern with $\|e\| < \delta$, and let $c \in C$. Then $d(c, c + e) = \|c + (c + e)\| = \|e\| < \delta$. Since C has distance δ and $c + e$ is closer to c than this, $c + e \notin C$. Therefore C detects e .

On the other hand, from the definition of δ , there exist codewords $b, c \in C$ with $d(b, c) = \delta$. Consider the error pattern $e = b + c$. It has weight $\|e\| = \|b + c\| = d(b, c) = \delta$. And $e = b + c \implies b + e = c \in C$ so C does not detect this error pattern e of weight δ (if b is sent and error pattern e occurs, c will be received and no error detected). $\square$

Note that a code C with minimum distance δ may possibly detect *some* error patterns of weight δ or more, but it does not detect *all* error patterns of weight δ .



All errors of weight $< \delta$ can be detected. Not all errors of weight $\geq \delta$ can be detected.

Recalling Definition 2.1.10, let

$$C + C = \{c_1 + c_2 \mid c_1, c_2 \in C\}.$$

Along the way, we showed the following:

3.2.8 Lemma A code C cannot detect an error pattern $e \neq 0$ iff $e = c_1 + c_2$ for some distinct $c_1, c_2 \in C$:

The set of undetectable non-zero error patterns is $(C + C) \setminus \{0\}$.

Proof Suppose that $e = c_1 + c_2$. Then $c_2 = c_1 + e$, so if the error e occurred while transmitting c_1 , we would conclude that c_2 was sent without error. Thus e is not detected.

Conversely, suppose that error $e \neq 0$ occurs while transmitting c_1 , and this error is not detected. Then some codeword c_2 must have been received. So $c_2 = e + c_1$, and so $e = c_1 + c_2$. $\square$

3.2.9 Example Let $M = \mathbb{F}_2^2$ and let C_2 be the 2-fold repetition code. Show that C_2 can detect some but not all error patterns of weight 2.

Solution:

$$C_2 = \{0000, 0101, 1010, 1111\}.$$

The encoding function is $00 \mapsto 0000$, $01 \mapsto 0101$ etc. The minimum distance is $\delta = 2$, so C_2 is a $(4, 2, 2)$ -code. By Theorem 3.2.7 C is 1-error detecting, but not 2-error detecting. Thus, there exists an error pattern of weight 2 that C_2 cannot detect.

Let $e = 0011$. Then $\|e\| = 2$ and $0000 + e = 0011 \notin C_2$, $0101 + e = 0110 \notin C_2$, $1010 + e = 1001 \notin C_2$, $1111 + e = 1100 \notin C_2$. In every case C_2 detects that errors have occurred, so C_2 can detect the error pattern $e = 0011$.

Thus, the code C_2 detects some errors of weight two, but not all of them. $\otimes$

3.2.10 Example Recall Example 3.1.5: $C = \{0000, 0011, 0101, 0110, 1001, 1010, 1100, 1111\}$. Here $\delta = 2$ (every word has even weight), and C is a $(4, 3, 2)$ -code. So C can detect 1 error. (Namely, if an odd weight word is received, an error must have occurred.)

Finding the precise set of all errors that can be detected by a code is computationally infeasible for reasonably large codes. In practice, when channel reliability is high, we can assume that only a small number of errors will occur in any codeword, and concentrate on the largest value of the parameter t for which C is t -error detecting.

3.3. Error Correction

Suppose we detect that errors have occurred. We would like to be able to correct them.

Suppose c was sent, error e occurred, so we received $w = c + e$. In the decoding process we find the closest codeword c^* to w , and assume that c^* was actually sent. This succeeds provided $c^* = c$. That is, provided c is the closest codeword to $w = c + e$. This breaks down once so many errors occur that $c + e$ is equally close, or strictly closer, to some other codeword c' than to c .

3.3.1 Definition A code C corrects an error pattern $e \neq 0$ if, for all $c \in C$, $c + e$ is strictly closer to c than to any other word $c' \in C$:

$$\forall c, c' \in C \quad c' \neq c \implies d(c + e, c) < d(c + e, c').$$

3.3.2 Definition A code C is said to be t -error correcting if it corrects all non-zero error patterns of weight $\leq t$.

Let $e \neq 0$ be an error pattern.

- ★ If for any $c, c' \in C$ with $c \neq c'$ we have $d(c + e, c) \geq d(c + e, c')$ then C cannot correct e .
- ★ To show that C can correct e , we must show that $d(c + e, c) < d(c + e, c')$ for every $c \neq c' \in C$.

Thus C is t -error correcting if it is possible to take each codeword in C , draw a sphere of radius t centred on it, and have no two spheres intersect. Then any received word which falls within the sphere of a codeword will be corrected unambiguously to the codeword on which the sphere is centred.

We restate Definition 3.3.1 as a sort of analogue of Lemma 3.2.8.

3.3.3 Lemma Let C be a code, $c \in C$ and $e \neq \mathbf{0}$ an error pattern. If c is transmitted and error pattern e occurs, the code C will successfully correct the received word back to c iff $\|c + c' + e\| > \|e\|$ for all $c' \in C \setminus \{c\}$.

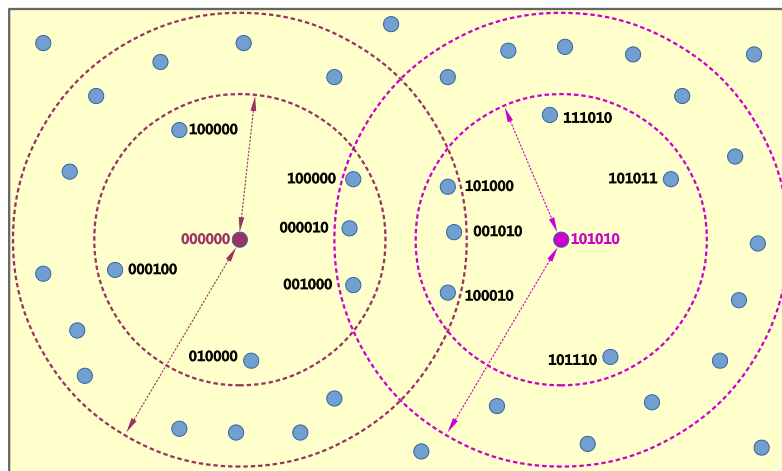
Proof By Definition 3.3.1, pattern e will be dealt with successfully iff the unique closest codeword to $c + e$ is still c , iff

$$c' \neq c \implies d(c + e, c') > d(c + e, c).$$

That is, $\|c + e + c'\| > \|c + c + e\| = \|e\|$. □

Consider the 3-fold repetition code C with $k = 2$ (Example 1.2.1). This is a $(6, 2, 3)$ code.

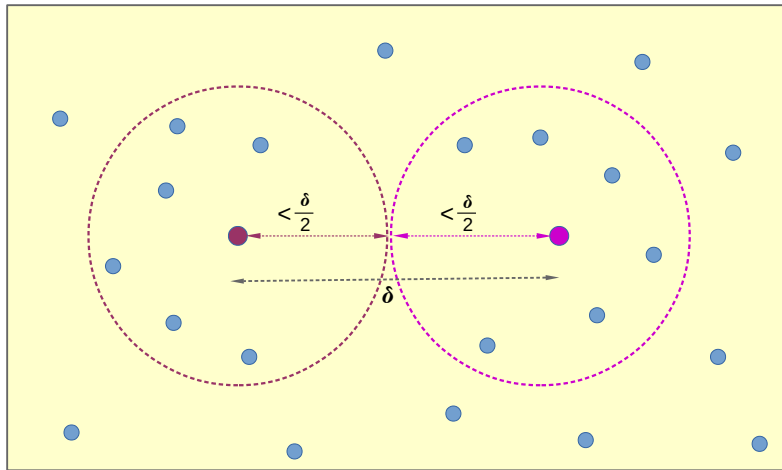
Suppose $c = 101010$ was sent, but error pattern $e = 001010$ occurred, so $\|e\| = 2$. Then $w = c + e = 101010 + 001010 = 100000$. The closest codeword to w is $c' = 000000$. So we would (incorrectly) assume that c' was sent, and “correct” w to c' . The code C fails to correct the error pattern e . So many errors have occurred, that the codeword transmitted (c) has been pushed closer to another codeword (c').



Spheres of radius 1 and 2 centred on 000000 and $c = 101010$.

$$w = c + e = 101010 + 001010 = 100000$$

e cannot be corrected



If the nearest codeword to c_1 is distance δ away, error e can be corrected provided $\|e\| < \frac{\delta}{2}$.

The next result is fundamental. Roughly: we can detect twice as many errors as we can correct.

3.3.4 Theorem Let C be a code with minimum distance δ . Then C can correct all non-zero error patterns with weight $< \delta/2$. This is best possible: there is at least one error pattern of weight $\lceil \delta/2 \rceil$ that C cannot correct.

Proof Let e be any non-zero error pattern with $\|e\| < \delta/2$. Suppose c is transmitted and $w = c + e$ is received. Let $c' \neq c$ be any other codeword. If $d(c', w) \leq d(c, w)$, then using the triangle inequality we have $0 < d(c', c) \leq d(c', w) + d(w, c) \leq 2d(c, w) = 2\|c + w\| = 2\|c + c + e\| = 2\|e\| < \delta$, contradicting the definition of δ . Thus $d(c', w) > d(c, w)$, and so c is the unique closest codeword to w . So the error pattern e is corrected successfully.

The remaining statement is fairly obvious, but a little annoying to write down carefully.

Let c and c' be two codewords at exact distance δ and let $e = c + c'$, so $\|e\| = \delta$. We shall construct a word w with error pattern e_1 of weight $\lceil \frac{1}{2}\delta \rceil$ that cannot be corrected.

Divide e into two parts of weights as close to $\frac{1}{2}\delta$ as possible:

$$\begin{array}{rcl}
 e & = & \overbrace{101010010010110 \dots\dots 01}^{\text{Contains the first } \lceil \frac{1}{2}\delta \rceil \text{ of the 1s in } e} \mid \overbrace{10100100110 \dots\dots\dots 010}^{\text{Contains the last } \lfloor \frac{1}{2}\delta \rfloor \text{ of the 1s in } e} \\
 e_1 & = & 101010010010110 \dots\dots 01 \mid 000000000 \dots\dots\dots 000 \\
 e_2 & = & 0000000000000 \dots\dots 00 \mid 10100100110 \dots\dots\dots 010
 \end{array}$$

$$\text{where } e = e_1 + e_2 \quad \|e_1\| = \lceil \frac{1}{2}\delta \rceil \quad \|e_2\| = \lfloor \frac{1}{2}\delta \rfloor.$$

That is, e_2 replaces (the first $\lceil \frac{1}{2}\delta \rceil$ of the 1s in e) by 0s, and e_1 replaces (the last $\lfloor \frac{1}{2}\delta \rfloor$ of the 1s) by 0s.

$$\text{By Lemma 2.3.6} \quad \lceil \frac{1}{2}\delta \rceil + \lfloor \frac{1}{2}\delta \rfloor = \delta = \|e\|.$$

Let $w = c + e_1$. Then $w + c' = c + e_1 + c' = c + e_1 + (e + c) = c + e_1 + (e_1 + e_2) + c = e_2$. Thus $d(w, c) = \|w + c\| = \|e_1\| = \lceil \frac{1}{2}\delta \rceil$, but $d(w, c') = \|w + c'\| = \|e_2\| = \lfloor \frac{1}{2}\delta \rfloor$. So $d(w, c') \leq d(w, c)$ and thus error pattern e_1 cannot be corrected. $\square$

Recalling Lemma 2.3.4 we see:

3.3.5 Corollary Let C be a code with minimum distance δ . Then C is $\left\lfloor \frac{\delta-1}{2} \right\rfloor$ -error correcting. $\square$

So: if c is transmitted and $\leq \left\lfloor \frac{\delta-1}{2} \right\rfloor$ errors occur, the unique closest word to $c + e$ is still c . A code of distance δ may correct some error patterns of weight $> \left\lfloor \frac{\delta-1}{2} \right\rfloor$. However, it will not correct all such error patterns.

3.3.6 Example Let $C = \{000, 111\}$ (three fold repetition code with $k = 1$). Thus C has distance $\delta = 3$ and hence can detect all error patterns of weight $\leq \delta - 1 = 2$, and correct all error patterns of weight ≤ 1 .

3.4. The Rate of a Code

By adding extra bits of redundancy we can improve error detection. But the more check bits are added, the less efficient the code is.

Consider the following $(n, 3)$ -codes:

- (a) The trivial code: $C_1: \mathbb{F}_2^3 \rightarrow \mathbb{F}_2^3$
- (b) The parity check (Example 3.2.10) code: $C_2: \mathbb{F}_2^3 \rightarrow \mathbb{F}_2^4$
- (c) The 2-fold repetition code (similar to Example 3.2.9): $C_3: \mathbb{F}_2^3 \rightarrow \mathbb{F}_2^6$

Then C_1 is 0-error detecting ($\delta = 1$). We can see that C_2 and C_3 are both 1-error detecting ($\delta = 2$ for both). But C_2 is more efficient than C_3 , since we added only 1 additional bit, rather than 3.

3.4.1 Definition If C is an (n, k) code, its rate is $r = k/n$.

Note that $0 \leq r \leq 1$ (and in practice $0 < r < 1$).

In the case where the set M of message words may not be all of $\mathbb{F}_2^k$ a more general definition of rate is

$$\frac{1}{n} \log_2 |C|.$$

3.4.2 Example The rates of C_1 , C_2 and C_3 above are 1, $\frac{3}{4}$ and $\frac{1}{2}$ respectively.

A code with $\delta = 3$ is quite impressive when $n = 7$, but not so impressive if n is large. Consider the *relative distance* $\partial = \delta/n$. This approximately represents the fraction of bits that may contain errors and still be detected, while $\partial/2$ is approximately the fraction of bits that can be corrected.

Because of Theorems 3.2.7 and 3.3.4, a main goal of coding theory is to construct codes with large minimum distance. For small values of n and k the maximal possible value of δ for which an (n, k, δ) code exists is known, but for general n and k this is an open problem. We will consider bounds on δ in terms of n and k later. Ideally we would like both r and ∂ close to 1, but we shall see that this is impossible.

In practice, one must balance the decrease in rate against the increase in accuracy obtained by using a code. Factors that influence the choice of code for a particular application include the reliability of the channel, the possibility of re-transmitting data if required and the sensitivity of the data to uncorrected errors.

Before considering these issues, we should justify the fact that IMLD is the schema that gives the highest probability of success.

3.5. Probability

We consider the probability that a code does detect/correct errors, assuming an idealised mathematical model for the distribution of errors.

Recall: a codeword is transmitted by sending its bits, one at a time and in order, across a *binary channel*.

For most of the course, we shall assume that errors are distributed uniformly, independently, with a fixed probability. (We shall consider other situations at the end of the course.) In detail:

Let $0 \leq p \leq 1$. We assume our binary channel has the following properties:

- ★ Errors in distinct positions are independent.
- ★ If we send a 0 through our binary channel, we receive a 0 with probability p . If we send 1, we receive 1 also with probability p .

A channel with these properties is called a *binary symmetric channel* or *BSC* of *reliability* p .

So reliability $p = 0.9$ means a 10% chance that any given bit is corrupted. Note:

- ★ If the channel has $p \leq 0.5$, it can be converted to a channel with $0.5 \leq p \leq 1$, by adding 1 to each received word. So we may assume $p \geq 0.5$.
- ★ If $p = 0.5$ the channel is perfectly random. Every possible received word of length n is equally probable, no matter what message was sent. There is no correlation between what is sent and what is received. This renders error detecting and correcting impossible to achieve. We assume this is not the case, so $p > 0.5$.
- ★ If $p = 1$ there are never errors and there is no need for a code, so assume $p < 1$.
- ★ Thus from now on: assume we have a BSC with reliability p with $0.5 < p < 1$.

Let us examine how probable various error occurrence scenarios are. Let c be the codeword of length n transmitted, let w be the word received and assume communication is over a BSC with reliability p as described above.

3.5.1 Definition Let $\mathbb{P}_p(c \rightarrow w)$ be the probability that if c is transmitted then w is received.

If c is sent then w is received iff error pattern $e = c + w$ occurs. But (under our assumptions) the probability of a given error pattern occurring is independent of the word c sent. For error pattern e to occur, we need an error to occur in the $\|e\|$ bits where e has a 1, which happens with probability $1 - p$ for each such bit, and we need no error to occur in the $n - \|e\|$ bits where e has a 0, which happens with probability p for each such bit. So

$$(3.5.1) \quad \mathbb{P}_p(c \rightarrow w) = p^{n-\|e\|}(1-p)^{\|e\|} \quad \text{where } e = c + w.$$

Letting $d = \|e\|$ be the number of errors that occurred, we can rewrite (3.5.1) as:

$$(3.5.2) \quad \mathbb{P}_p(c \rightarrow w) = p^{n-d}(1-p)^d.$$

3.5.2 Example Consider a BSC with reliability $p = 1 - 10^{-8} = 0.99999999$. Suppose that 10^7 bits per second are transmitted over the channel, and our message length is $k = 11$.

What is the expected frequency of undetected errors if we use the trivial code $C \subseteq \mathbb{F}_2^{11}$ consisting of all 2^{11} words of length 11 ?

Solution: The code C cannot detect any errors ($\delta = 1$). The probability that no error occurs in a word is p^{11} . So one or more errors occur with probability $1 - p^{11} = 1 - (1 - 10^{-8})^{11} = \pi$, say. Recall the binomial approximation

$$(1 + x)^n \approx 1 + nx, \quad \text{for small } |x|$$

So $\pi = \left[1 - (1 - 10^{-8})^{11}\right] \approx 1 - (1 - 11/10^8) = 11/10^8$. Thus the expected number of words per second with errors is (probability π of error) $\times$ (# words per second) $\approx \frac{11}{10^8} \cdot \frac{10^7}{11} = 0.1$ per second, or approximately 1 undetected error every 10 seconds. $\circledast$

Compare this to what happens if we add a parity check bit.

3.5.3 Example Let D be the code obtained from the code C in Example 3.5.2 by adding a parity-check bit to each codeword of C , in such a way that the number of 1s in each codeword in D is even (compare Example 3.1.5).

What is the expected frequency of undetected errors?

Solution: The purpose of making every word have even weight is that it ensures that in D we have $\delta = 2$, so D will detect a single error in a transmitted word. There are $\binom{12}{0}$ error patterns of weight 0 and $\binom{12}{1}$ of weight 1. Thus the probability π of an incorrect word being received and² not detected is

$$\pi \leq 1 - \mathbb{P}(0 \text{ errors}) - \mathbb{P}(1 \text{ error}) = 1 - \binom{12}{0}p^{12}(1-p)^0 - \binom{12}{1}p^{11}(1-p)^1 \approx 6.6 \times 10^{-15}.$$

The words in D have length 12. So the expected number of words per second with errors is (probability π of error) $\times$ (# words per second) $\lesssim (6.6 \times 10^{-15}) \cdot \frac{10^7}{12} = 5.5 \times 10^{-9}$ per second. This is approximately 1 word every 6 years, instead of every 10 seconds. $\circledast$

By adding only a small amount of extra information (1/12 or about 8% of bits are redundant) we drastically reduce the number of incorrect words that slip through without detection. The price we pay is that information is transmitted at rate $r = \frac{11}{12}$ of the speed that it would be without any coding. (To incorporate error correction into the system, we would need to increase either the redundancy or the block length of the code.)

We are using IMLD in which to decode a received word, we find the codeword which differs from it in the smallest number of places possible. (Least Hamming distance). This makes intuitive sense, but we should justify that this is valid. We prove that this method gives the highest probability of recovering the sent word.

3.5.4 Theorem Suppose communication is via a BSC with reliability p with $0.5 < p < 1$. Let c_1 and c_2 be codewords of length n , and let w be any word of length n . Then

$$\mathbb{P}_p(c_1 \rightarrow w) \geq \mathbb{P}_p(c_2 \rightarrow w) \iff d(c_1, w) \leq d(c_2, w).$$

In other words, the closer a codeword is to w , the more likely it is that it was the word sent.

²Note that some error patterns of weight ≥ 2 may possibly be detected, which we have not allowed for, so the expression is an upper bound for π .

Proof First note that $0.5 < p < 1 \implies 0 < 1 - p < p$. Hence $\frac{p}{1-p} > 1$. Let $d_1 = d(w, c_1)$, $d_2 = d(w, c_2)$. Then:

$$\begin{aligned}
 \mathbb{P}_p(c_1 \rightarrow w) &\geq \mathbb{P}_p(c_2 \rightarrow w) \\
 \iff p^{n-d_1}(1-p)^{d_1} &\geq p^{n-d_2}(1-p)^{d_2} && \text{by Equation (3.5.2)} \\
 \iff \left(\frac{1-p}{p}\right)^{d_1} &\geq \left(\frac{1-p}{p}\right)^{d_2} \\
 \iff \left(\frac{1-p}{p}\right)^{d_1-d_2} &\geq 1 \\
 \iff \left(\frac{p}{1-p}\right)^{d_2-d_1} &\geq 1 \\
 \iff d_2 &\geq d_1 && \text{because } \frac{p}{1-p} > 1. \quad \square
 \end{aligned}$$

Recall that $\mathbb{P}_p(c \rightarrow w)$ is the probability that the word w is received if the codeword c is transmitted over a BSC with reliability p . In practice we know the received word w , but we do not know the codeword transmitted, c . However, we do know all of the codewords, so we can calculate $\{\mathbb{P}_p(b \rightarrow w) \mid b \in C\}$. If this collection has a unique maximum value, then we should choose the corresponding unique b as the most likely transmitted word. By Theorem 3.5.4, this is the unique codeword that differs from w in the fewest positions (if there is such a unique word). This justifies IMLD.

Since $d(c_1, w) = \|c_1 + w\|$, we can restate Theorem 3.5.4 as

$$\mathbb{P}_p(c_1 \rightarrow w) \geq \mathbb{P}_p(c_2 \rightarrow w) \iff \|c_1 + w\| \leq \|c_2 + w\|.$$

The most likely codeword sent is the one with the error pattern of smallest weight.

IMLD involves calculating the distance from every received word to every codeword. This is generally not practical. Much of the course will be devoted to finding codes with efficient decoding algorithms.

3.6. Overall Success (Q_C) and Fail (F_C) Probabilities

Above we considered the probability of various error patterns occurring. Perhaps a more practical question is: Given a code, what is the probability that “it works”?

Let C be a code transmitted over a BSC. Let $c_0 \in C$.

- ★ Let $Q_C(c_0)$ denote the probability that if c_0 was sent, the received word will be successfully corrected back to c_0 .
- ★ Let Q_C be the average³ of all the $Q_C(c_0)$ as c_0 varies across C . This is a measure of the overall success rate of C .

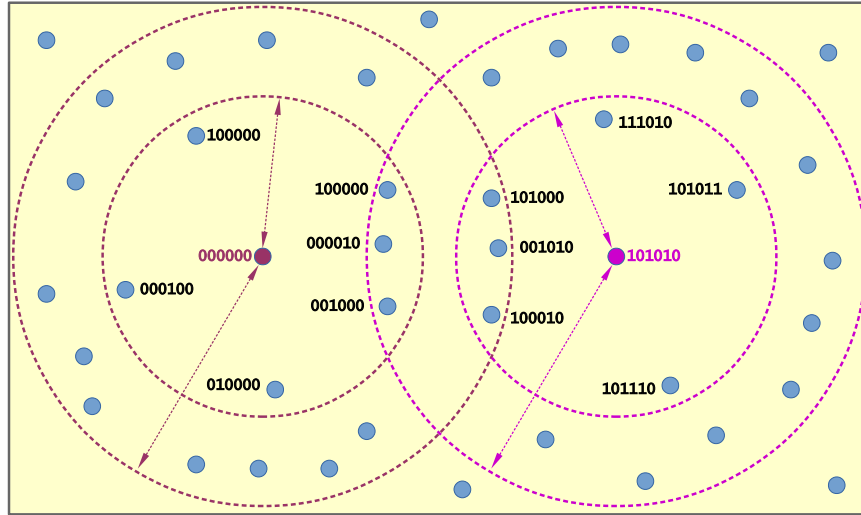
Both $Q_C(c_0)$ and Q_C will be functions of p .

Consider the 3-fold repetition $(6, 2, 3)$ -code C . Fix a single codeword c_0 , say $c_0 = 000000$. If c_0 is sent, what is the probability $Q_C(000000)$ that the received word w is successfully corrected back to c_0 ?

³Of course, not all message words may be equally likely, so this may not be ideal. However if the message to be sent is first encrypted or compressed then the message words may be approximately equally likely.

We need to take the sum over all error patterns e that can be corrected starting from c_0 , weighted by their probability of occurrence. Since $\delta = 3$, C is 1-error correcting, so all error patterns of weight ≤ 1 are dealt with successfully. Some higher weight error patterns may also be dealt with. Which ones?

There are 15 error patterns of weight 2. All are closer to $c_0 = 000000$ than they are to 111111, but 3 of them are closer to 101010 than 000000, and 3 are closer to 010101 than 000000:



This leaves 9 patterns of weight 2 that are corrected, namely $e = 000011, 000110, 001001, 001100, 010010, 011000, 100001, 100100, 110000$.

No error pattern e with $\|e\| \geq 3$ can be dealt with successfully. To see this, let $c' = 111111 = \mathbf{1}$. Then

$$d(c_0 + e, c') = \|\mathbf{1} + e\| \stackrel{\text{Lemma 2.2.3}}{=} 6 - \|e\| \leq 3 \leq \|e\| = d(c_0, e).$$

So $c' = 111111$ is closer (or equally close) to the received word than is c_0 .

(These arguments were a bit ad hoc, but can be made systematic via Lemma 3.3.3.)

The probability that $\left[\text{if } c_0 = 000000 \text{ is sent then } c_0 \text{ is recovered} \right]$ is given by summing over all the ways this can happen: $e = \mathbf{0}$, or e equal to any of the 6 patterns with $\|e\| = 1$, or e equal to any of the 9 patterns with $\|e\| = 2$ that are successfully corrected. Thus

$$Q_C(c_0) = p^6 + 6p^5(1-p) + 9p^4(1-p)^2 = p^4(2p-3)^2.$$

To measure the overall reliability of the code, we should calculate $Q_C(c)$ for each of the (in this case, 4) codewords in C , and take the average:

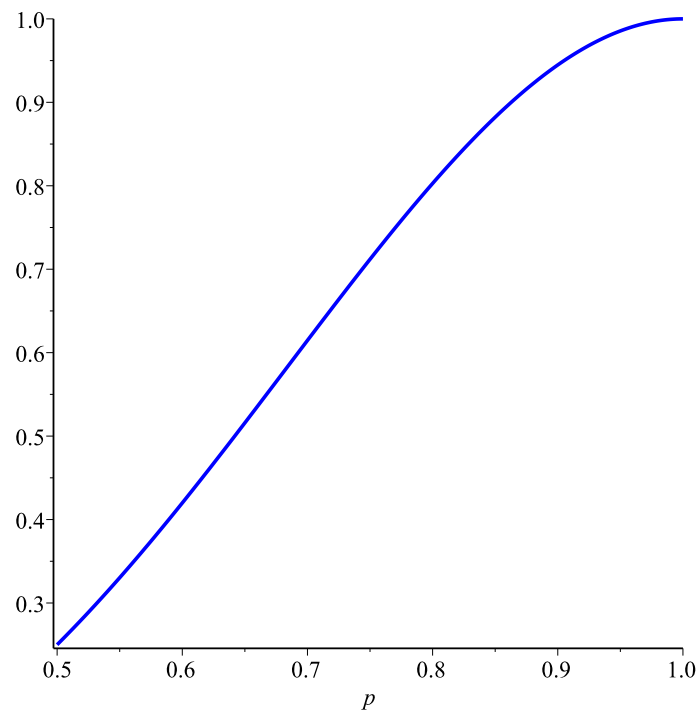
$$Q_C = \frac{1}{|C|} \sum_{c \in C} Q_C(c).$$

Then Q_C is the average probability that the code “works”: that a sent word is correctly recovered.

For this particular code C it turns out (exercise!) that $Q_C(c)$ is the same for each of the 4 codewords. So this code has

$$Q_C = p^4(2p-3)^2.$$

A plot of $Q_C = p^4(2p-3)^2$ as a function of $p > 0.5$:



If $p = 0.9$ then $Q_C \simeq 0.945$. If $p = 0.97$ then $Q_C \simeq 0.995$. If $p = 0.5$ (which is prohibited) then $Q_C = 0.25$, because no matter which word is received we are effectively guessing which word was sent, which will only work $1/4$ of the time.

The overall *failure probability* of the code is

$$F_C = 1 - Q_C.$$

This is the average probability that the code fails to correctly recover a transmitted word. We would like F_C to be as close to 0 as possible.

3.7. Shannon's Theorem

It is intuitively clear that there are limits on what codes can achieve. If there is a 40% chance of error on each bit, we probably would not expect to produce a code that adds 1 bit of redundancy every 1000 bits (rate 0.999), and still expect to correct essentially all errors that occur. Whereas, if there is only a 1% chance of error, then a well designed code with rate say 0.8 (?) should be able to correct almost all errors.

It is a profound theorem of Shannon that this intuition is correct, can be made precise, and there is even an exact description of the best possible rate, called the *channel capacity*.

Suppose we have a BSC with reliability $p > 0.5$. We would like to be able to produce codes that can be used to transmit information across the channel such that the failure probability F_C can be made arbitrarily small. More precisely, we would like a family of codes $C_1, C_2, \dots$ of blocklengths $n_1 < n_2 < \dots$ such that the failure probabilities $F_{C_1}, F_{C_2}, \dots \rightarrow 0$ as $n \rightarrow \infty$. This allows us to pick a suitable code to achieve whatever accuracy is required.

Of course, this is possible by taking n huge compared to k , so there is a gigantic amount of redundancy and extremely low rate $r \rightarrow 0$ as $n \rightarrow \infty \dots$. This is not very interesting.

Instead, we would like to know that we can achieve such a family of codes with r staying fixed as $n \rightarrow \infty$. So that, with only a constant percentage overhead, we can achieve failure probability arbitrarily close to 0. It is not obvious that this is possible.

Shannon's Theorem says that it is possible, provided r is not required to be too large. We say that the *channel capacity* R of the BSC is the largest rate for which it is possible. More precisely, the *channel capacity* R of the BSC is the supremum of all rates r for which there exists a family of codes $C_1, C_2, \dots$ all of rate $\geq r$, of strictly increasing blocklengths, such that $F_{C_n} \rightarrow 0$ as $n \rightarrow \infty$.

Roughly speaking, we can transmit information at a rate slightly less than R across our BSC, with essentially no errors (we can arrange that the probability F_C of failure is $< \epsilon$). But with rate slightly above R we cannot achieve this goal, no matter how large we make n . So R is the intrinsic limit of what is possible to achieve with this channel.

3.7.1 Theorem [Shannon, 1948] The channel capacity of a BSC with reliability p exists, and is given by

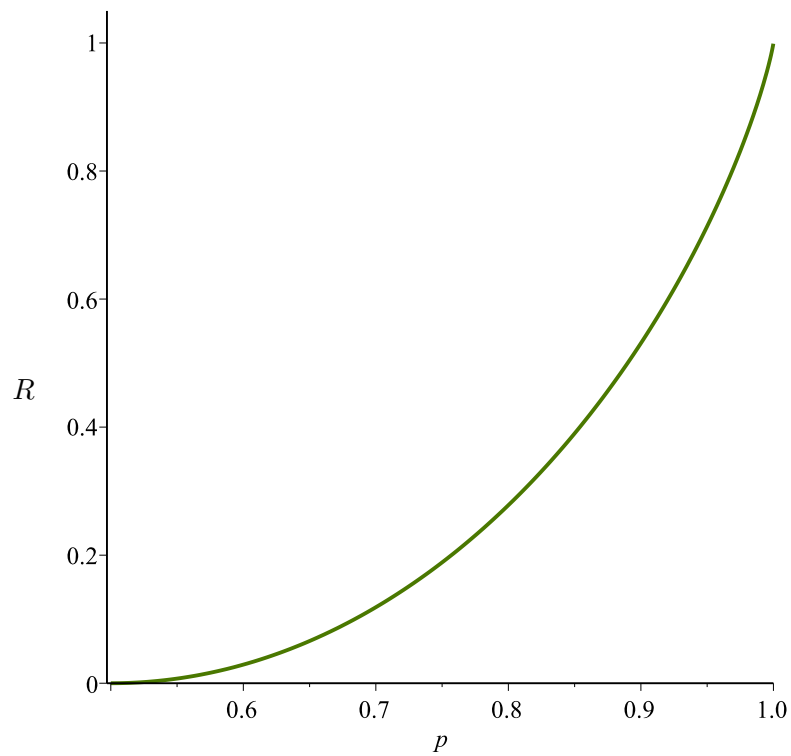
$$R = 1 + p \log_2(p) + (1 - p) \log_2(1 - p).$$

Shannon's theorem shows that there exist codes with good error correcting properties, limited only by the reliability of the channel, and describes the best we could possibly achieve. However it does not give a usable construction of such codes.

These Shannon optimal codes are difficult to generate, and generally do not have good algorithms for encoding and decoding. In particular, for decoding we may not be able to do better than calculate the distance between a received vector and all possible codewords. If the code is large (k is large) this may be too slow.

In practice it is very often preferable to impose some extra structure on the code, which will allow easy (fast) encoding and decoding, even if this means the error correcting ability is not absolutely at the Shannon limit. The gains in efficiency generally outweigh the losses in rate/error-correction. We discuss how to do this in the next chapter.

Below is a plot of the channel capacity R of a BSC with reliability p , as given by Shannon's Theorem.



Reliability p	0.50	0.66	0.75	0.80	0.85	0.90	0.95	0.98	0.99	0.999
Capacity R	0	0.075	0.189	0.278	0.390	0.531	0.714	0.859	0.919	0.989

CHAPTER 4

Linear Codes I

The essential goal of coding theory is to find codes which transmit information at reasonable rates, yet also detect and correct most transmission errors. In this section we discuss a broad class of codes which provide these features, based heavily on linear algebra.

In this case, encoding will be matrix multiplication (which can be implemented efficiently in hardware, and so is extremely fast). For certain classes of codes, decoding can be achieved via one matrix multiplication and one vector addition.

Recall that a code is an injective function $f: \mathbb{F}_2^k \rightarrow \mathbb{F}_2^n$. Let C be the image of f . How should we choose f ? What properties do we desire of C ?

If we translate all the words in C by a fixed word, then the distance between codewords, and hence the characteristics of C do not change. Thus wlog we may assume $\mathbf{0} \in C$. Now recall Lemma 3.2.8, which says $(C + C) \setminus \{0\}$ is the set of non-zero error patterns that C cannot detect. We would like to minimize this set.

Since $\mathbf{0} \in C$, $\mathbf{0} + c = c \in C + C$ for every $c \in C$, so $C + C \supseteq C$. So the best (smallest) we could hope for is $C + C = C$, which says C should be *closed under addition*. This means that C will be a *vector space* over $\mathbb{F}_2$, and $f: \mathbb{F}_2^k \rightarrow \mathbb{F}_2^n$ should be an injective map between vector spaces. The most natural choice for f would be a *linear map*. Then f will be described by a matrix of full rank.

4.1. Review of Linear Algebra

We can do linear algebra over $\mathbb{F}_2$ in the same way as over $\mathbb{R}$ or $\mathbb{C}$. All of the same methods and results still apply: row reduction, determinants etc.

Recall that a non-empty subset $U \subseteq \mathbb{F}_2^n$ is a $\mathbb{F}_2$ -vector space iff U is closed under addition and scalar multiplication. Since the only scalars are 0, 1 $\in \mathbb{F}_2$, and $0 \cdot v = \mathbf{0}$, U is a vector space iff $\left[\mathbf{0} \in U \text{ and } U \text{ is closed under addition} \right]$. If $U \subseteq \mathbb{F}_2^n$ is non-empty and closed under addition, then U must contain the zero vector (since if $u \in U$, then $u + u = \mathbf{0} \in U$), so will be an $\mathbb{F}_2$ -vector space.

4.1.1 Definition A code C is a *linear code* if the sum of any two codewords is also a codeword. That is if $v, w \in C$, then $v + w \in C$. We shall assume $C \neq \{\mathbf{0}\}$, ie that $k > 0$.

Thus a linear code is exactly a vector subspace of $\mathbb{F}_2^n$. Almost every code we consider for the remainder of this course will be linear.

4.1.2 Example Show that $C_1 = \{0000, 0101, 1010, 1111\}$ is linear, but that $C_2 = \{0000, 1010, 0011, 1111\}$ is not linear.

Solution: In C_2 , $1010 + 0011 = 1001 \notin C_2$. C_1 : exercise. ⊗

A *linear combination* of vectors $v_1, \dots, v_n$ is an expression $\sum \alpha_i v_i$, where the α_i are scalars. Over $\mathbb{F}_2$ the only scalars are 0 and 1, so a linear combination of vectors is just a sum of some of them: v_4 , $v_1 + v_3 + v_5$, $v_1 + \dots + v_n$, etc.

A set of vectors $\{v_1, \dots, v_n\}$ is *linearly independent* if no linear combination of them is equal to $\mathbf{0}$, except the trivial combination (empty sum) $0v_1 + \dots + 0v_n$. Since the only scalars are 0 and 1, $S = \{v_1, \dots, v_n\}$ is linearly independent iff no non-empty sum of v_i adds to $\mathbf{0}$. If a set of vectors is not linearly independent, then the set of vectors is said to be *linearly dependent*.

The *span* of $S = \{v_1, \dots, v_n\}$, denoted $\text{span}(S)$ or $\langle S \rangle$ is the collection of all linear combinations that can be made from S . This is always a vector space.

A *basis* for a vector space V is a subset $B \subseteq V$ such that:

- ★ $\langle B \rangle = V$ (B spans V), and
- ★ B is linearly independent.

Every vector space V has a basis. In fact, a vector space usually has many different bases, but all bases of V have the same number of elements. The *dimension* of a vector space is the number of elements in (any) basis.

Eg: $\mathbb{F}_2^n$ is a vector space. A basis is $\theta_1 = 10000 \dots 0$, $\theta_2 = 01000 \dots 0$, $\theta_3 = 00100 \dots 0$, $\dots$, $\theta_n = 00 \dots 001$. So $\dim \mathbb{F}_2^n = n$.

We also consider matrices with entries in $\mathbb{F}_2$.

Abbreviations:

- ★ ero = elementary row operation.
- ★ ref = row echelon form of a matrix.
- ★ rref = reduced row echelon form (after Gauss-Jordan reduction).

The only eros are: adding two rows, and swapping two rows.

4.1.3 Theorem Let A be an $m \times n$ matrix with entries in $\mathbb{F}_2$. All of the following quantities are then equal:

- ★ The number of linearly independent rows in A
- ★ The dimension of the space spanned by the rows of A (row space)
- ★ The number of linearly independent columns in A
- ★ The dimension of the space spanned by the columns of A (column space)
- ★ The number of non-zero rows in any ref of A .

Proof Omitted. □

4.1.4 Definition The number of linearly independent rows of a matrix A is called the *rank* of A . This is equal to the other quantities listed in Theorem 4.1.3. The set of all v such that $Av = \mathbf{0}$ is called the *nullspace* or *kernel* of A , and denoted $\text{NS}(A)$.

Recall¹ the *Rank-Nullity Theorem*, also called the *Fundamental Theorem of Linear Algebra*:

¹Algebraists will recognize as essentially the First Isomorphism Theorem, in a linear algebra context.

4.1.5 Theorem Let $\mathbf{A}$ be an $m \times n$ matrix with entries in $\mathbb{F}_2$. Then

$$\text{rank}(\mathbf{A}) + \dim \text{NS}(\mathbf{A}) = n.$$

Proof Omitted. □

4.2. Basic Properties of Linear Codes

One of the advantages of a linear code is that its distance is easy to find.

4.2.1 Theorem For any linear code $C \neq \{0\}$, the distance of C is equal to the smallest weight of any nonzero codeword.

Proof Let C be a linear code of distance δ , and let c_0 be a nonzero codeword of smallest weight. Since $0, c_0 \in C$ the distance of C is at most $d(c_0, 0) = \|c_0\|$, so $\delta \leq \|c_0\|$. Suppose $\delta < \|c_0\|$. Then there are two codewords $c_1, c_2 \in C$ such that $\delta = d(c_1, c_2) < \|c_0\|$. As C is linear, $c_1 + c_2$ must also be a codeword, of weight $\|c_1 + c_2\| = d(c_1, c_2) < \|c_0\|$. But this contradicts the assumption that c_0 is the nonzero codeword of smallest weight. □

4.2.2 Example Find the distance of the linear code $C = \{0000, 1100, 0011, 1111\}$.

Solution: The non-zero codeword(s) of least weight are 1100, 0011, so $\delta = 2$. ⊗

It is easy to find the distance of a linear code. Other advantages of linear codes include:

- ★ For linear codes, there is a procedure for IMLD which is simpler and faster than we have seen so far (for some linear codes, there are very simple decoding algorithms).
- ★ Encoding using a linear code is faster than for arbitrary non-linear codes.
- ★ It is easy to manufacture linear codes. Just take the span of any set of vectors.
- ★ It is easy to describe the set of error patterns that a linear code will detect.
- ★ It is much easier to describe the set of error patterns a linear code will correct than it is for arbitrary non-linear codes.

Disadvantage: linear codes are not necessarily optimal in terms of their error correcting ability.

Recall that an (n, k, δ) code has 2^k codewords of length n , and minimum distance δ .

A *basis* for a linear code C is a basis for the vector space C . The *dimension* of C is its dimension as a subspace of $\mathbb{F}_2^n$, namely the number of elements in a basis.

For a linear code we have now given two different definitions of dimension. However they agree: Suppose a linear code C has dimension k in the vector space sense. Let $B = \{c_1, c_2, \dots, c_k\}$ be a basis for C . Then each codeword b in C can be written as

$$b = a_1 c_1 + a_2 c_2 + \dots + a_k c_k$$

for a unique choice of scalars $a_1, a_2, \dots, a_k$. Since each a_i is 0 or 1 for $1 \leq i \leq k$, there are 2^k distinct choices for $a_1, a_2, \dots, a_k$, and hence for b .

So the dimension of C in the linear algebra sense agrees with our definition of the dimension k of a code in the general case.

4.3. Bases for Codes

How can we describe a vector space C ? Two approaches are: give a basis, or give a set of equations defining the elements of C .

If $c_1, \dots, c_k$ forms a basis of C , then every element $b \in C$ can be written uniquely in the form $b = a_1c_1 + \dots + a_kc_k$. Let $\mathbf{G}$ be the matrix formed by taking row i to be c_i for $1 \leq i \leq k$. Then expressions $a_1c_1 + \dots + a_kc_k$ are exactly the vectors obtained by multiplying $\mathbf{G}$ by the vector $(a_1, a_2, \dots, a_k)$. That is, C is the image of the matrix $\mathbf{G}$.

Alternatively, one can describe C as all vectors satisfying some linear equations, such as $C = \{x \mid x_1 + x_3 = 0, x_2 + x_4 = 0\}$ etc, where x is the word with bits $x_1x_2 \dots$. This says x is an element of the nullspace of some matrix $\mathbf{H}$.

That is, C may be defined as the image of one map ($\mathbf{G}$) or the kernel of another ($\mathbf{H}$).

In detail, we shall show that associated to each linear code C are two matrices, denoted $\mathbf{G}$ and $\mathbf{H}$:

- ★ $\mathbf{G}$ is used for encoding.
- ★ $\mathbf{H}$ is used for error correction/detection and for finding the distance δ .

[In fact $\mathbf{G}$ is simply a matrix representation of the linear function f that formally defines the code.]

The next sections define $\mathbf{G}$ and $\mathbf{H}$ and explain how they are used. We start with $\mathbf{G}$, which is straightforward. It is a bit harder at first to see the purpose of the matrix $\mathbf{H}$, but it is equally important.

Given a linear code C , We let $\mathbf{G}$ consist of rows that form a basis for C . (Formal definition below.)

First, recall the method of finding a basis for a linear code (vector space):

4.3.1 Algorithm [Algorithm for finding a basis for $C = \langle S \rangle$.]

Let S be a non-empty subset of $\mathbb{F}_2^n$.

- ★ Form the matrix $\mathbf{A}$ whose rows are the words in S .
- ★ Use `eros` to find a `ref` (or `rref`) of $\mathbf{A}$.
- ★ The non-zero rows of the `ref` of $\mathbf{A}$ form a basis for $C = \langle S \rangle$.

Algorithm 4.3.1 works because the rows of $\mathbf{A}$ generate C , and `eros` simply interchange codewords (rows) or replaces one codeword (row) with the sum of two rows (another codeword) giving a new set of codewords which still² generates C . Clearly the non-zero rows in a matrix in `ref` are linearly independent. Note that the basis produced by Algorithm 4.3.1 is not unique, and there is no guarantee that the words in the basis occur in the given set S .

4.3.2 Example Use Algorithm 4.3.1 to find a basis for the linear code $C = \langle S \rangle$ where $S = \{11101, 10110, 01011, 11010\}$.

²Because we are repeatedly replacing $\{u, v\}$ by $\{u, u + v\}$, but $\text{span } \{u, v\} = \text{span } \{u, u + v\}$.

Solution:

$$\mathbf{A} = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix} \rightarrow$$

$$\begin{pmatrix} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad \text{ref.}$$

A basis is $\{11101, 01011, 00111\}$.

Thus C is a $(5, 3)$ code. The complete set of codewords is obtained from all 2^3 linear combinations of basis words, so $C = \{00000, 11101, 01011, 00111, 10110, 11010, 01100, 10001\}$. Recall that δ is the smallest non-zero weight of any word. Thus $\delta = 2$. This means C is 1-error detecting, but 0-error correcting: 10000 is distance 1 from 00000 and from 10001 so cannot be corrected. $\otimes$

After we calculate a basis for C (as above) we put the basis vectors into the rows of a $k \times n$ generating matrix $\mathbf{G}$, so that the rows of $\mathbf{G}$ are linearly independent and form a basis for C .

4.4. Generating Matrices

We now use the material from the previous section to create a matrix that is used in the encoding process for a linear code.

4.4.1 Definition If C is a linear code of length n and dimension k , then any matrix whose rows form a basis for C is called a *generating matrix* for C . Such matrices must have k rows and n columns, and have rank k .

Since a vector space may have many different bases, the matrix $\mathbf{G}$ is not unique. (It is unique if we insist that it is in rref.)

As a consequence of Definition 4.4.1, we have the following theorems.

4.4.2 Theorem A $k \times n$ matrix $\mathbf{G}$ with $n \geq k$ is a generating matrix for some (n, k) linear code C iff the rows of $\mathbf{G}$ are linearly independent (so the rank of $\mathbf{G} = k$).

4.4.3 Theorem If $\mathbf{G}$ is a generating matrix for a linear code C , then any matrix row equivalent to $\mathbf{G}$ is also a generating matrix for C . In particular, every linear code has a unique generating matrix in rref.

To find a generating matrix for a linear code C , we can form the matrix whose rows are the non-zero codewords in C . By the definition of ‘linear’ we must have $C = \langle C \rangle$, so we can use Algorithm 4.3.1 to produce a basis for C . Then the matrix whose rows are these basis vectors forms a generating matrix for C .

4.4.4 Example Find a generating matrix in rref for the linear code $C = \{0000, 1110, 0111, 1001\}$.

Solution:

$$A = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow$$

$$\begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \xrightarrow{\text{ref}} \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \xrightarrow{\text{rref}}$$

So a basis for C is $\{1001, 0111\}$. (Note that $|C| = 4 = 2^2$ so we already knew $\dim C = 2$.) A generating matrix in rref is

$$G = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix}$$

This has rank 2. Every word in C is a sum of (≥ 0) rows of G : $1110 = \text{row } 1 + \text{row } 2$ etc.

Alternative: Since we know $\dim C = 2$, for a basis just pick any 2 linearly independent words of C . ie pick any 2 distinct non-zero words in C for the rows of G . $\circledast$

A $k \times n$ matrix $\mathbf{G}$ is a generating matrix for an (n, k) linear code if the binary words that can be expressed as the sum of a subset of the rows of $\mathbf{G}$ are exactly the codewords of C .

4.4.5 Example Let C be the linear code $C = \{000000, 001110, 010101, 011011, 100011, 101101, 110110, 111000\}$. Show that $\mathbf{G}$ below is a generating matrix for C .

$$\mathbf{G} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}.$$

Solution: Use Algorithm 4.3.1. Or note that

- ★ $|C| = 8$ so $\dim C = 3$, and $\mathbf{G}$ is 3×6 ,
- ★ The rows of $\mathbf{G}$ are linearly independent,
- ★ Each individual row of $\mathbf{G}$ is a codeword.

So the rows of $\mathbf{G}$ generate a 3-dimensional subspace of the 3-dimensional space C , and hence must exactly generate C .

Eg row 1 + row 2 is the codeword 110110, row 1 + row 3 is the codeword 101101, etc. Every codeword in C (and nothing else) is obtained in this way. $\circledast$

Having defined a generating matrix, we can use standard techniques from linear algebra to show how messages of length k are encoded as codewords of length n .

We can no longer avoid discussing the following.

Convention: In coding theory it is (fairly) common to write vectors as row vectors. To multiply a $1 \times k$ vector x by an $k \times n$ matrix $\mathbf{A}$ we must form $x\mathbf{A}$. Of course, $(x\mathbf{A})^\top = \mathbf{A}^\top x^\top$.

$$\text{Recall } \mathbf{A} \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} = \text{1st column of } \mathbf{A}, \quad \mathbf{A} \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix} = \text{2nd column of } \mathbf{A}, \text{ etc.}$$

Transposing and putting $\mathbf{B} = \mathbf{A}^\top$,

$$\begin{aligned}(1, 0, 0, \dots, 0) \mathbf{B} &= \text{1st row of } \mathbf{B} \\ (0, 1, 0, \dots, 0) \mathbf{B} &= \text{2nd row of } \mathbf{B} \\ \therefore (1, 1, 0, \dots, 0) \mathbf{B} &= \text{1st row} + \text{2nd row of } \mathbf{B}\end{aligned}$$

Etc.

Let C be a linear (n, k) -code. If $\mathbf{G}$ is a generating matrix for C and if m is a word of length k written as a row vector, then $c = m\mathbf{G}$ is a codeword in C , since c is a linear combination of the rows of $\mathbf{G}$, and these rows form a basis for C . And every codeword is obtained in this way.

In more detail: let $g_1, g_2, \dots, g_k$ be the rows of $\mathbf{G}$. Let $m = \alpha_1\alpha_2\dots\alpha_k$ be a message with bits α_i :

$$\begin{aligned}m &= \alpha_1(1, 0, 0, \dots, 0) + \alpha_2(0, 1, 0, \dots, 0) + \dots + \alpha_k(0, 0, \dots, 1) \\ m\mathbf{G} &= \left[\alpha_1(1, 0, 0, \dots, 0) + \alpha_2(0, 1, 0, \dots, 0) + \dots + \alpha_k(0, 0, \dots, 1) \right] \mathbf{G} \\ &= \alpha_1 g_1 + \alpha_2 g_2 + \dots + \alpha_k g_k\end{aligned}$$

is a codeword in C .

Thus $m\mathbf{G}$ is the sum of rows of $\mathbf{G}$ corresponding to the places where m is 1. If $m = 101$ then $m\mathbf{G}$ will be row 1 plus row 3 of $\mathbf{G}$, etc.

If $m_1\mathbf{G} = m_2\mathbf{G}$ then we must have $m_1 = m_2$, since each codeword in C is a *unique* linear combination of the codewords in a basis. Thus no codeword $c = m\mathbf{G}$ in C is produced by more than one message word in $\mathbb{F}_2^k$.

This is just restating that the encoding function f (represented by $\mathbf{G}$) is injective.

Summarizing:

4.4.6 Theorem If $\mathbf{G}$ is a generating matrix for a linear (n, k) -code C , then $c = m\mathbf{G}$ ranges over all 2^k codewords in C as m ranges over all 2^k words of length k . Thus $C = \{m\mathbf{G} \mid m \in \mathbb{F}_2^k\}$. Moreover, $m_1\mathbf{G} = m_2\mathbf{G}$ iff $m_1 = m_2$.

Message m is encoded as $c = m\mathbf{G}$, where m and c are represented by row vectors.

4.4.7 Example Let C be the linear $(5, 3)$ -code with generating matrix

$$\mathbf{G} = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{pmatrix}$$

Assign letters to the words in $\mathbb{F}_2^3$ as follows:

$$\begin{array}{cccccccc} 000 & 110 & 010 & 001 & 100 & 101 & 011 & 111 \\ \text{A} & \text{B} & \text{E} & \text{H} & \text{M} & \text{R} & \text{T} & \text{W} \end{array}$$

Using the generating matrix $\mathbf{G}$, encode the message be there .

Solution: We have a $(5, 3)$ -code, so message words are (row vectors of) length 3, and codewords are length 5. Since $B = 110$, form

$$110 \quad \mathbf{G} = (1 \ 1 \ 0) \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{pmatrix} = (1 \ 1 \ 1 \ 0 \ 1).$$

So we encode B as 11101. Etc. We can do all the matrix multiplications at once, by putting each message word into a separate row on the left. So to encode $BE \dots$

$$\begin{array}{l} B \\ E \\ \vdots \end{array} \rightarrow \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ \vdots & \vdots & \vdots \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix}.$$

So $BE \dots$ is encoded as 11101, 01011, $\dots$

⊛

4.5. Systematic and Equivalent Codes

4.5.1 Definition Let $\mathbf{G}$ be a $k \times n$ matrix with $k \leq n$. We say $\mathbf{G}$ is in *standard form* if the first k columns of $\mathbf{G}$ form the $k \times k$ identity matrix $\mathbf{I}_k$. That is, $\mathbf{G} = (\mathbf{I}_k \mid \mathbf{X})$. Any such $\mathbf{G}$ automatically has linearly independent rows and is in rref. Thus $\mathbf{G}$ is a generating matrix for some linear code of length n and dimension k . We say that the code C generated by $\mathbf{G}$ is *systematic*.

Not all linear codes have a generating matrix in standard form.

4.5.2 Example Is the linear code $C = \{00000, 10110, 10101, 00011\}$ systematic?

Solution: We find a generating matrix. There is no need to write down the $\mathbf{0}$ word. ($\mathbf{0}$ is never part of a basis.) So:

$$\begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix} \rightarrow$$

$$\begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad \text{rref}$$

So $\mathbf{G} = \left(\begin{array}{cc|cc} 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{array} \right)$. The unique rref is not in standard form, so C is not systematic. ⊛

Suppose C is a linear (n, k) -code with generating matrix $\mathbf{G}$ in standard form.

To send message m , we send the codeword $c = m\mathbf{G}$. But

$$c = m\mathbf{G} = m(\mathbf{I}_k \mid \mathbf{X}) = (m\mathbf{I}_k \mid m\mathbf{X}) = (m \mid m\mathbf{X}).$$

Thus, *the message bits form the first k bits of the corresponding codeword*. So the encoded message is the original message, followed by $n - k$ extra bits.

The encoding function in this case is as we discussed in the introduction to coding (3.1.1):

$$m = \overbrace{** \dots *}^k \mapsto \overbrace{** \dots *}^k \overbrace{\bullet \bullet \dots \bullet}^{n-k}.$$

Not every code is in standard form, so the encoding process is not always so simple.

We have the following:

4.5.3 Theorem If C is a linear code of length n and dimension k with generating matrix $\mathbf{G}$ in standard form and a message m is to be transmitted, then the first k bits of the codeword $v = m\mathbf{G}$ form the message word $m \in \mathbb{F}_2^k$.

4.5.4 Example Suppose C is the $(7, 4)$ linear code with generating matrix

$$\mathbf{G} = \left(\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{array} \right) = (\mathbf{I}_4 \ \mathbf{X}).$$

- ★ Calculate the codeword corresponding to the message word 0111.
- ★ Which message word corresponds to the codeword 1011000?

Solution:

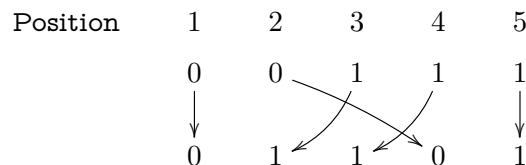
$$(0 \ 1 \ 1 \ 1) \mathbf{G} = \left(\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{array} \right) = (0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1).$$

The message corresponding to 1011000 is just the first 4 bits: 1011. ⊗

Systematic codes are particularly simple to work with. However, (see Example 4.5.2) some codes have no generating matrix in standard form. In this case we can produce an “equivalent” code that is systematic:

4.5.5 Definition If C is any code of length n , we can obtain a new code C' of length n by applying a fixed permutation σ of the n bits. The resulting code $C' = \sigma(C)$ is said to be *equivalent* to C .

4.5.6 Example Let $n = 5$ and $C = \{00000, 00111, 11000, 11111\}$. Rearrange the bits into the order 1, 3, 4, 2, 5, so word $x_1 x_2 x_3 x_4 x_5$ becomes $x_1 x_3 x_4 x_2 x_5$. In algebra notation σ is the permutation $\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & 4 & 2 & 3 & 5 \end{pmatrix} = (2 \ 4 \ 3).$



The resulting code $C' = \{00000, 01101, 10010, 11111\}$ is equivalent to C .

A generating matrix $\mathbf{G}$ for a linear code C contains codewords of C as row vectors. If we change the order of the first two bits, this amounts to changing the order of the first two columns in $\mathbf{G}$. Thus permuting bits in codewords is equivalent to permuting columns in a generating matrix.

Hence we have the following theorem:

4.5.7 Theorem Any linear code C is equivalent to a linear code having a generating matrix in standard form.

Proof If $\mathbf{G}$ is a generating matrix for C , place $\mathbf{G}$ in rref. Since $\mathbf{G}$ has (column) rank k , there are k linearly independent columns: those containing the leading 1s, whose other entries are 0 (rref). Rearrange the columns of the rref so that these columns come first, and hence form an identity matrix. The result is a generating matrix $\mathbf{G}'$ in standard form for a code C' that is equivalent to C . □

4.5.8 Example In Example 4.5.2, we showed that $C = \{00000, 10110, 10101, 00011\}$ is not systematic. Find a systematic code C' that is equivalent to C .

Solution: We found that for this code, we may take $\mathbf{G} = \left(\begin{array}{cc|ccc} 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{array} \right)$. To make this systematic, we need to interchange the second and fourth columns (bits). Thus $C' = \{\overset{\curvearrowright}{00000}, \overset{\curvearrowright}{11100}, \overset{\curvearrowright}{10101}, \overset{\curvearrowright}{01001}\}$ is a systematic code equivalent to C . $\circledast$

Note that C and C' are different codes, although they have many similar properties. They are both linear, both have length 5, dimension 2, and distance 2. However, C' is systematic while C is not.

4.6. Dual Codes

We shall now see how to derive a new code from a given linear code C , using the orthogonal complement. A generating matrix for this new code is the matrix $\mathbf{H}$, which is very useful for error detection and correction in the original code C .

Recall that vectors $v, w \in \mathbb{F}_2^n$ are *orthogonal* if $v \cdot w = 0 \in \mathbb{F}_2$. We write $v \perp w$.

4.6.1 Example $v = 101$ and $w = 111$ are orthogonal since $v \cdot w = 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 = 1 + 0 + 1 = 0$.

If $\emptyset \neq S \subseteq \mathbb{F}_2^n$, the set of vectors orthogonal to every vector in S is denoted $S^\perp$.

$$S^\perp = \{v \in \mathbb{F}_2^n \mid v \cdot s = 0 \text{ for all } s \in S\}.$$

4.6.2 Lemma For any non-empty set $S \subseteq \mathbb{F}_2^n$, the set $S^\perp$ is a vector space (linear code).

Proof The set $S^\perp$ contains $\mathbf{0}$, so is non-empty. We just have to check that $S^\perp$ is closed under addition. But if $u, v \in S^\perp$ and $s \in S$ then $(u + v) \cdot s = u \cdot s + v \cdot s = 0 + 0 = 0$, so $(u + v) \perp s$. And $s \in S$ was arbitrary, so $u + v \in S^\perp$. $\square$

4.6.3 Lemma If $S \subseteq \mathbb{F}_2^n$, then $\langle S \rangle^\perp = S^\perp$.

Proof We have $S \subseteq \langle S \rangle$ so $v \in \langle S \rangle^\perp$ implies $v \in S^\perp$. Thus $\langle S \rangle^\perp \subseteq S^\perp$. Now suppose $v \in S^\perp$. If $u \in \langle S \rangle$, then $u = w_1 + w_2 + \cdots + w_t$ for some $w_1, w_2, \dots, w_t \in S$. Thus, $v \cdot u = v \cdot (w_1 + w_2 + \cdots + w_t) = v \cdot w_1 + v \cdot w_2 + \cdots + v \cdot w_t = 0 + 0 + \cdots + 0 = 0$ (since $v \in S^\perp$ and $w_1, w_2, \dots, w_t \in S$). Since $u \in \langle S \rangle$ was arbitrary, we have shown $v \in \langle S \rangle^\perp$. Thus, $S^\perp \subseteq \langle S \rangle^\perp$ and so $\langle S \rangle^\perp = S^\perp$. $\square$

4.6.4 Lemma Suppose U, V are subspaces of $\mathbb{F}_2^n$, S is a basis for U , T is a basis for V and $s \cdot t = 0$ for all $s \in S$ and $t \in T$. Then $u \cdot v = 0$ for all $u \in U$ and all $v \in V$.

Proof Let $u \in U$ and $v \in V$. Then $u = s_1 + \cdots + s_p$ and $v = t_1 + \cdots + t_q$ where $s_1, \dots, s_p \in S$ and $t_1, \dots, t_q \in T$. Thus, $u \cdot v = (s_1 + \cdots + s_p) \cdot (t_1 + \cdots + t_q)$. Since each term in the expansion of $(s_1 + \cdots + s_p) \cdot (t_1 + \cdots + t_q)$ is 0 we have $u \cdot v = 0$. $\square$

4.6.5 Definition If $C \subseteq \mathbb{F}_2^n$ is a code (linear or not), the set $C^\perp$ is called the *dual code* of C .

Suppose C is a linear code generated by $S \subseteq \mathbb{F}_2^n$. That is, $C = \langle S \rangle$. Then by Lemma 4.6.3 we have $C^\perp = \langle S \rangle^\perp = S^\perp$. Thus,

$$(4.6.1) \quad C = \langle S \rangle \implies C^\perp = S^\perp.$$

Warning: We do not have $\mathbb{F}_2^n = V \cup V^\perp$ in general; in particular a non-zero word may be orthogonal to itself. However we do have the following:

4.6.6 Theorem Let $C \subseteq \mathbb{F}_2^n$ be a linear code. Then $\dim C + \dim C^\perp = n$.

Proof Let $\dim C = k$. Let $S = \{c_1, \dots, c_k\}$ be a basis for C . By Equation 4.6.1

$$x \in C^\perp \iff c_j \cdot x = 0 \quad \text{for } j = 1, \dots, k.$$

Put the vectors c_j in the rows of a $k \times n$ matrix $\mathbf{A}$. Since $\mathbf{A}$ has linearly independent rows, $\text{rank}(\mathbf{A}) = \dim C = k$. Now, $x \in C^\perp \iff \mathbf{A}x^\top = \mathbf{0}$. So $\dim C^\perp = \dim \text{NS}(\mathbf{A})$. Thus, from Theorem 4.1.5, $n = \text{rank}(\mathbf{A}) + \dim \text{NS}(\mathbf{A}) = \dim C + \dim C^\perp$. $\square$

4.6.7 Example Suppose that C is a linear $(9, 4)$ -code. How many codewords are in C ? How many codewords are in $C^\perp$?

Solution: We have $\dim C = 4$ so $|C| = 2^4 = 16$. And $\dim C^\perp = 9 - 4 = 5$, so $|C^\perp| = 2^5 = 32$. Note that there are $2^9 = 512$ possible words, so most words are in neither C nor $C^\perp$. $\otimes$

4.6.8 Lemma If $V \subseteq \mathbb{F}_2^n$ is a subspace then $(V^\perp)^\perp = V$.

Proof It is clear that $V \subseteq (V^\perp)^\perp$ and by Theorem 4.6.6 we have $\dim V + \dim V^\perp = n$ and $\dim V^\perp + \dim (V^\perp)^\perp = n$. Thus, $\dim V = \dim (V^\perp)^\perp$, and this together with $V \subseteq (V^\perp)^\perp$ implies $(V^\perp)^\perp = V$. $\square$

We now introduce the matrix $\mathbf{H}$:

4.6.9 Lemma Let C be a linear (n, k) -code with generating matrix $\mathbf{G}$. Suppose matrix $\mathbf{H}$ is an $n \times (n - k)$ matrix with linearly independent columns. Then the columns of $\mathbf{H}$ form a basis for $C^\perp$ iff $\mathbf{GH} = \mathbf{0}$.

Proof $\implies$ If the columns of $\mathbf{H}$ form a basis for $C^\perp$, then each column h of $\mathbf{H}$ lies in $C^\perp$, and so $c \cdot h = 0$ for all $c \in C$. But this means $\mathbf{GH} = \mathbf{0}$.

$\impliedby$ Suppose $\mathbf{GH} = \mathbf{0}$. Let D be the span of the columns of $\mathbf{H}$. By assumption, the dot product of each row vector of $\mathbf{G}$ with each column vector of $\mathbf{H}$ is 0. Thus $u \cdot v = 0$ for each u in [a basis for] C and v in [a basis for] D . By Lemma 4.6.4, every vector in D is perpendicular to every vector in C , so $D \subseteq C^\perp$. But $\mathbf{H}$ has rank $n - k$, so applying Theorems 4.1.3 and 4.6.6 $\dim D = n - k = n - \dim C = \dim C^\perp$, so $D = C^\perp$. That is, the columns of $\mathbf{H}$ span $C^\perp$, and they are linearly independent, so form a basis. $\square$

Any matrix satisfying the equivalent conditions of Lemma 4.6.9 is called a parity check matrix.

4.6.10 Definition Let C be a linear (n, k) -code. An $n \times (n - k)$ matrix $\mathbf{H}$ is called a *parity check matrix* for C if its columns form a basis for $C^\perp$.

Since the same space may have many bases, the matrix $\mathbf{H}$ is not unique.

If C is a systematic code with generating matrix $\mathbf{G}$ in standard form, then it is easy to find a parity check matrix $\mathbf{H}$. Let

$$\mathbf{G} = (\mathbf{I}_k \ \mathbf{X})$$

where $\mathbf{X}$ is some $k \times (n - k)$ matrix. Let

$$\mathbf{H} = \begin{pmatrix} \mathbf{X} \\ \mathbf{I}_{n-k} \end{pmatrix}.$$

Then: $\mathbf{H}$ is $n \times (n - k)$, and the presence of $\mathbf{I}_{n-k}$ inside $\mathbf{H}$ ensures that the $n - k$ columns of $\mathbf{H}$ are linearly independent. Finally by block multiplication:

$$\mathbf{G}\mathbf{H} = (\mathbf{I}_k \ \mathbf{X}) \begin{pmatrix} \mathbf{X} \\ \mathbf{I}_{n-k} \end{pmatrix} = \mathbf{I}_k \mathbf{X} + \mathbf{X} \mathbf{I}_{n-k} = \mathbf{0}_{k \times (n-k)}.$$

By Lemma 4.6.9, the columns of $\mathbf{H}$ form a basis for $C^\perp$.

4.6.11 Example Let $C = \{000000, 001110, 010101, 011011, 100011, 101101, 110110, 111000\}$. This is easily checked to be a linear code. Find bases for C and $C^\perp$.

Solution:

$$\begin{aligned} \mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \end{pmatrix} &\rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \\ &\rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}. \end{aligned}$$

So a generating matrix for C is

$$\mathbf{G} = \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{array} \right) = (\mathbf{I}_3 \ \mathbf{X}), \quad \mathbf{X} = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}.$$

Thus $\mathbf{G}$ is in standard form, and C is a systematic $(6, 3)$ -code. A basis for C is $\{100011, 010101, 001110\}$. (Taking all sums of these words gives all of C , which proves that C is linear, because it is a span.)

Let

$$\mathbf{H} = \begin{pmatrix} \mathbf{X} \\ \mathbf{I}_{6-3} \end{pmatrix} = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

A basis for $C^\perp$ is $\{011100, 101010, 110001\}$. ⊗

Even if C is not systematic we can still find a parity check matrix $\mathbf{H}$ as follows:

- ★ Find a systematic code C' equivalent to C , via some permutation $C \xrightarrow{\sigma} C'$.
- ★ If $\mathbf{G}$ and $\mathbf{G}'$ are generating matrices for C and C' respectively, then $\mathbf{G} \xrightarrow{\sigma} \mathbf{G}'$ by permuting columns.

- ★ Since $\mathbf{G}'$ is systematic, find corresponding $\mathbf{H}'$ as above. Thus $\mathbf{H}'$ is a parity check matrix for C' .
- ★ Finally apply the inverse permutation σ^{-1} to the rows of $\mathbf{H}'$ to obtain $\mathbf{H}$.

(We shall prove below that $\mathbf{H}$ satisfies Lemma 4.6.9.)

4.6.12 Lemma If $\mathbf{A}$ and $\mathbf{B}$ are matrices, and the columns of $\mathbf{A}$ are permuted with permutation σ to get $\mathbf{A}^*$, and the rows of $\mathbf{B}$ are permuted with the same permutation σ to get $\mathbf{B}^*$, then $\mathbf{AB} = \mathbf{A}^*\mathbf{B}^*$.

Proof Let $\mathbf{A} = [a_{ij}]$, $\mathbf{A}^* = [a_{ij}^*]$, $\mathbf{B} = [b_{jk}]$, $\mathbf{B}^* = [b_{jk}^*]$, and let t be the number of columns in $\mathbf{A}$, which equals the number of rows in $\mathbf{B}$. Then $a_{ij}^* = a_{i\sigma(j)}$ and $b_{jk}^* = b_{\sigma(j)k}$ so

$$\sum_j a_{ij}^* b_{jk}^* = \sum_j a_{i\sigma(j)} b_{\sigma(j)k} = \sum_j a_{ij} b_{jk}$$

and we have $\mathbf{AB} = \mathbf{A}^*\mathbf{B}^*$. □

4.6.13 Algorithm [Finding $\mathbf{G}$ and $\mathbf{H}$] Given linear code C , put the words of C as rows into a matrix $\mathbf{A}$. Use *eros* to convert:

$$\mathbf{A} \rightarrow \left(\begin{array}{c} \mathbf{G} \\ \mathbf{0} \end{array} \right) \text{ in rref.}$$

The rows of $\mathbf{G}$ form a basis for C .

Apply a permutation σ to the columns of $\mathbf{G}$ to form $\mathbf{G} \xrightarrow{\sigma} \mathbf{G}' = (\mathbf{I}_k \mid \mathbf{X})$. Let

$$\mathbf{H}' = \left(\begin{array}{c} \mathbf{X} \\ \mathbf{I}_{n-k} \end{array} \right).$$

Apply the inverse permutation σ^{-1} to the rows of $\mathbf{H}'$ to form $\mathbf{H}' \xrightarrow{\sigma^{-1}} \mathbf{H}$. Then the columns of $\mathbf{H}$ form a basis for $C^\perp$, so $\mathbf{H}^\top$ is a generating matrix for $C^\perp$.

Proof (that the algorithm works) We know $\mathbf{G}'\mathbf{H}' = \mathbf{0}$, as before. Since the permutation σ^{-1} takes $\mathbf{G}' \rightarrow \mathbf{G}$ and also $\mathbf{H}' \rightarrow \mathbf{H}$, if we apply σ^{-1} to the columns of $\mathbf{G}'$ and to the rows of $\mathbf{H}'$ we get $\mathbf{GH} = \mathbf{0}$, by Lemma 4.6.12 with $\mathbf{A} = \mathbf{G}'$, $\mathbf{A}^* = \mathbf{G}$, $\mathbf{B} = \mathbf{H}'$, $\mathbf{B}^* = \mathbf{H}$ and permutation σ^{-1} . So $\mathbf{H}$ satisfies the conditions of Lemma 4.6.9 and the algorithm works. □

4.6.14 Example Consider the code $C = \langle S \rangle$ where $S = \{11010, 10001, 01001, 11000\}$. Find a basis for C and a basis for $C^\perp$.

$$\begin{aligned} \mathbf{A} &= \begin{pmatrix} 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} \\ &\rightarrow \begin{pmatrix} 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \text{ in rref.} \end{aligned}$$

A basis for C is $\{10001, 01001, 00010\}$, so $k = 3$.

$$\text{Now } \mathbf{G} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} \xrightarrow{\sigma} \mathbf{G}' = \left(\begin{array}{ccc|cc} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{array} \right) = (\mathbf{I}_3 \mid \mathbf{X}).$$

$\mathbf{X} = \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$ and the permutation σ swapped columns 3 and 4: $3 \leftrightarrow 4$.

The rows of $\mathbf{X}$ are placed in the first three rows of the $5 \times (5-3)$ matrix $\mathbf{H}'$. The remaining rows of $\mathbf{H}'$ are filled with the 2×2 identity matrix. Thus

$$\mathbf{H}' = \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 0 \\ \hline 1 & 0 \\ 0 & 1 \end{pmatrix} \xrightarrow{\sigma^{-1}} \mathbf{H} = \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}$$

Since σ swapped positions 3 and 4, σ^{-1} swaps positions 4 and 3. So rows 4 and 3 change places in $\mathbf{H}'$ to give $\mathbf{H}$. Thus a basis for $C^\perp$ is $\{00100, 11001\}$. Note that $\dim C = 3$, $\dim C^\perp = 2$ inside $\mathbb{F}_2^5$.

We verify that $\mathbf{G}\mathbf{H} = \mathbf{0}$ as follows.

$$\mathbf{G}\mathbf{H} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

4.6.15 Example If $S = \{101010, 010101, 111111, 000111, 101100\}$, find a basis for $C = \langle S \rangle$, and a basis for $C^\perp$.

Solution:

$$\begin{aligned} \mathbf{A} &= \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \\ &\rightarrow \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}. \end{aligned}$$

So $\mathbf{G} = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$. A basis for C is $\{101010, 010010, 000110, 000001\}$, so $\dim C = 4$ and $|C| = 2^4 = 16$.

$$\mathbf{G}' = \left(\begin{array}{cccc|cc} 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{array} \right) = (\mathbf{I}_4 \quad \mathbf{X}), \quad \mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}.$$

Permutation $\sigma: 3 \mapsto 6, \quad 6 \mapsto 4, \quad 4 \mapsto 3$ also written $\sigma = (3 \ 6 \ 4)$.

So $\sigma^{-1}: 3 \mapsto 4, \quad 4 \mapsto 6, \quad 6 \mapsto 3, \quad \sigma^{-1} = (3 \ 4 \ 6)$, so

$$\mathbf{H}' = \left(\begin{array}{ccc} 1 & 1 & \\ 1 & 0 & \\ 1 & 0 & [3] \\ 0 & 0 & [4] \\ \hline 1 & 0 & \\ 0 & 1 & [6] \end{array} \right) \quad \mathbf{H} = \left(\begin{array}{ccc} 1 & 1 & \\ 1 & 0 & \\ 0 & 1 & [6 \mapsto 3] \\ 1 & 0 & [3 \mapsto 4] \\ 1 & 0 & \\ 0 & 0 & [4 \mapsto 6] \end{array} \right).$$

A basis for $C^\perp$ is $\{110110, 101000\}$, and $\dim C^\perp = 2$, so $|C^\perp| = 2^2 = 4$. ⊗

4.7. Parity Check Matrices

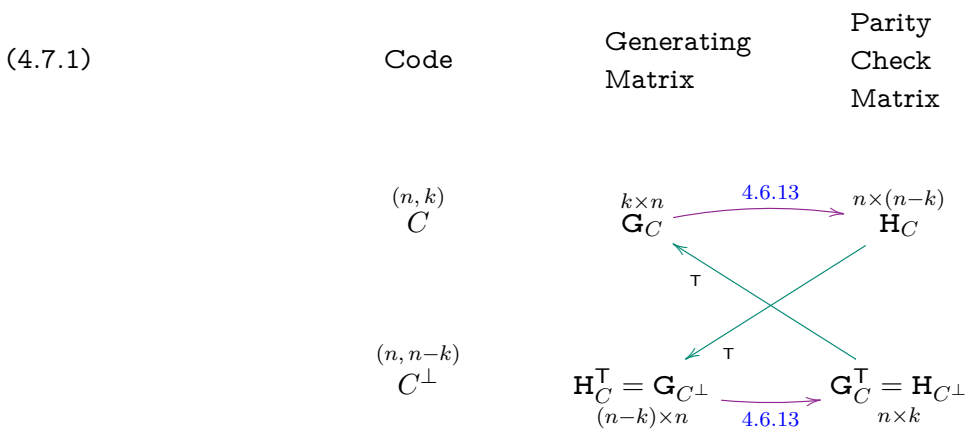
In this section we shall summarise the relationships between the generating and parity check matrices for a code and its dual. In the following sections we discuss how parity check matrices are used in error detection and correction.

First note:

4.7.1 Theorem A matrix $\mathbf{H}$ is a parity check matrix for a linear code C iff $\mathbf{H}^\top$ is a generating matrix for $C^\perp$.

Proof By definition, $\mathbf{H}$ is a parity check matrix for C iff the columns of $\mathbf{H}$ are a basis for $C^\perp$, iff the rows of $\mathbf{H}^\top$ are a basis for $C^\perp$, iff $\mathbf{H}^\top$ is a generating matrix for $C^\perp$. □

So $\mathbf{G}_{C^\perp} = \mathbf{H}_C^\top$. Replacing C by $C^\perp$ and using $(C^\perp)^\perp = C$, we can write $\mathbf{G}_C = \mathbf{H}_{C^\perp}^\top$, so $\mathbf{H}_{C^\perp} = \mathbf{G}_C^\top$. This is summarized in the following important diagram:



Given any one of the generating or parity check matrices for C or $C^\perp$, using diagram 4.7.1 we can find the other three matrices. This means: We can specify C and/or $C^\perp$ by giving $\mathbf{H}$ or $\mathbf{G}$.

Let us look at a comprehensive example.

4.7.2 Example Let C be a linear code with parity check matrix

$$\mathbf{H}_C = \left(\begin{array}{cc|cc} 1 & 1 & & \\ 1 & 1 & & \\ 0 & 1 & & \\ 1 & 0 & & \\ 0 & 1 & & \end{array} \right) = \left(\begin{array}{c} \mathbf{X} \\ \mathbf{I}_2 \end{array} \right).$$

Since $\mathbf{H}_C$ is $5 \times 2 = n \times (n - k)$, $n = 5$, $k = 3$. So C is a $(5, 3)$ -code and $C^\perp$ is a $(5, 2)$ -code.

★

$$\therefore \mathbf{G}_{C^\perp} = \mathbf{H}_C^\top = \left(\begin{array}{ccccc} 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 \end{array} \right).$$

★ Starting with $\mathbf{G}_{C^\perp}$, we use Algorithm 4.6.13 to find $\mathbf{H}_{C^\perp}$. The rref of $\mathbf{G}_{C^\perp}$ is

$$\left(\begin{array}{ccccc} 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \end{array} \right).$$

Apply Algorithm 4.6.13. Swap columns $2 \xleftrightarrow{\sigma} 3$:

$$\mathbf{G}'_{C^\perp} = \left(\begin{array}{cc|cc} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{array} \right) = (\mathbf{I}_2 \mid \mathbf{X}).$$

$$\therefore \mathbf{H}'_{C^\perp} = \left(\begin{array}{ccc|ccc} 1 & 1 & 0 & & & \\ 0 & 1 & 1 & & & \\ 1 & 0 & 0 & & & \\ 0 & 1 & 0 & & & \\ 0 & 0 & 1 & & & \end{array} \right) \xrightarrow{\sigma^{-1}} \left(\begin{array}{ccc|ccc} 1 & 1 & 0 & & & \\ 1 & 0 & 0 & & & \\ 0 & 1 & 1 & & & \\ 0 & 1 & 0 & & & \\ 0 & 0 & 1 & & & \end{array} \right) = \mathbf{H}_{C^\perp}.$$

★

$$\therefore \mathbf{G}_C = \mathbf{H}_{C^\perp}^\top = \left(\begin{array}{ccccc} 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{array} \right).$$

Note that we could have also found a generating matrix for C by starting with $\mathbf{H}_C$ and applying Algorithm 4.6.13 backwards. This would give us the following generating matrix for C .

$$\mathbf{G}'_C = (\mathbf{I}_3 \mid \mathbf{X}) = \left(\begin{array}{ccccc} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{array} \right).$$

Exercise: Check that $\mathbf{G}_C \mathbf{H}_C = \mathbf{G}'_C \mathbf{H}_C = \mathbf{0}$.

Now that we are familiar with the relationship between a generating matrix and a parity check matrix for a linear code C , we will look at how a parity check matrix can be used for error detection.

4.8. Error Detecting using the Parity Check Matrix

The next theorem shows an important use of the parity check matrix. Recall that by definition of $\mathbf{H}$, the column space of $\mathbf{H}$ is $C^\perp$.

4.8.1 Theorem Let C be a linear code with parity check matrix $\mathbf{H}$. Then $C = \{v \in \mathbb{F}_2^n \mid v\mathbf{H} = \mathbf{0}\}$.

Proof Clearly $v\mathbf{H} = \mathbf{0}$ iff v is perpendicular to each column of $\mathbf{H}$ iff v is perpendicular to the column space of $\mathbf{H}$ (see Lemma 4.6.4) iff $v \in (C^\perp)^\perp = C$. $\square$

In particular: If we receive w and $w\mathbf{H} \neq \mathbf{0}$, then error(s) must have occurred.

Some references on coding theory define the parity check matrix $\mathbf{H}'$ to be the transpose of our matrix $\mathbf{H}$, that is, as a $(n - k) \times n$ matrix, and instead of writing $v\mathbf{H} = \mathbf{0}$, write $\mathbf{H}'v^\top = \mathbf{0}^\top$.

4.8.2 Example Let $C = \{0000, 0011, 1100, 1111\}$, so C is a linear $(4, 2)$ -code. Verify that

$$\mathbf{H} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}$$

is a parity check matrix for C .

Solution: $n = 4$, $k = 2$ so $\dim C^\perp = 4 - 2 = 2$ so $C^\perp$ has a basis consisting of 2 vectors in $\mathbb{F}_2^4$.

The columns of $\mathbf{H}$ are linearly independent, so we just have to check that each of them is in $C^\perp$ ie check $1100 \cdot 0000 = 0$, $1100 \cdot 0011 = 0$ etc (exercise). $\square$

(Alternative: we could use Algorithm 4.6.13. But this will only give some parity check matrix for C , not necessarily the given $\mathbf{H}$.)

If we write each codeword as $x = (x_1 \ x_2 \ x_3 \ x_4)$, then $x\mathbf{H} = \mathbf{0}$ iff

$$(x_1 \ x_2 \ x_3 \ x_4) \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix} = (0 \ 0).$$

This holds in $\mathbb{F}_2$ iff

$$x_1 + x_2 = 0, \quad x_1 + x_2 + x_3 + x_4 = 0.$$

The solution to these equations is exactly the set of x with $x\mathbf{H} = \mathbf{0}$, ie exactly the codewords in C . $\circledast$

This explains why the matrix $\mathbf{H}$ is called a *parity check matrix*. Its columns give equations that describe the redundancy in/conditions satisfied by C , as a list of equations mod 2 (parity checks).

For a linear (n, k) -code, the matrix equation $x\mathbf{H} = \mathbf{0}$, where $x = (x_1, x_2, \dots, x_n)$, is a shorthand way of writing a set of $n - k$ simultaneous linear equations in the variables $x_1, x_2, \dots, x_n$. If the j th column of $\mathbf{H}$ has entries of 1 in positions $m_1, m_2, \dots, m_j$, then the bit-sum

$$x_{m_1} + x_{m_2} + \dots + x_{m_j} = 0.$$

In other words, the binary word x satisfies $x\mathbf{H} = \mathbf{0}$ iff the sums (obtained by ordinary addition) of certain of its bits (as determined by $\mathbf{H}$) are even (have even parity). The $n - k$ linear equations are thus referred to as *parity check equations* of the code, and $\mathbf{H}$ is called the *parity check matrix*.

4.8.3 Example The following matrix is a parity check matrix for a linear code C .

$$\mathbf{H}_C = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Which of the following words are codewords of C : 11000, 01100, 11101, 11010?

Solution: Just calculate $v\mathbf{H}$.

$$(1 \ 1 \ 0 \ 0 \ 0) \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = (0 \ 0)$$

so 11000 is a codeword. (Note that $11000 \mathbf{H}$ is just the sum of the first two rows of $\mathbf{H}$.) Similarly $01100 \mathbf{H} = 11$, $11101 \mathbf{H} = 00$, $11010 \mathbf{H} = 10$, so 11000 and 11101 are codewords and the others are not codewords. ⊗

In summary:

4.8.4 Theorem Matrices $\mathbf{G}$ and $\mathbf{H}$ are generating and parity check matrices respectively for a linear (n, k) -code C iff

- ★ The rows of $\mathbf{G}$ are linearly independent,
- ★ The columns of $\mathbf{H}$ are linearly independent,
- ★ $\mathbf{G}$ is $k \times n$, and $\mathbf{H}$ is $n \times (n - k)$, and
- ★ $\mathbf{GH} = \mathbf{0}$.

Then the rows of $\mathbf{G}$ are a basis for C , and the columns of $\mathbf{H}$ are a basis for $C^\perp$. The relationship between $\mathbf{G}$ and $\mathbf{H}$ is summarized in diagram 4.7.1.

We shall see in the next few sections that the parity check matrix can also be used to find the distance of a code, and also for error correction.

4.9. Distance of a Linear Code

We have seen in Theorem 4.2.1 that the distance δ of a linear code is the weight of a non-zero codeword of smallest weight. Note that this is not necessarily the minimum weight row in $\mathbf{G}$ because C contains many other words than just the rows of $\mathbf{G}$ (all the various sums of these rows).

Now we shall see that δ can be found from $\mathbf{H}$.

4.9.1 Theorem Let $\mathbf{H}$ be a parity check matrix for a linear code C . Then δ is equal to the smallest number (> 0) of rows of $\mathbf{H}$ that are linearly dependent.

That is, C has distance δ iff every set of $\delta - 1$ rows of $\mathbf{H}$ is linearly independent, while at least one set of δ rows of $\mathbf{H}$ is linearly dependent.

Proof Let S be a non-empty set of rows of $\mathbf{H}$ of minimal cardinality such that S is linearly dependent. Let $s = |S|$. We must show $s = \delta$.

$$\delta \leq s$$

Since S is dependent, there exists T with $\emptyset \neq T \subseteq S$ such that the sum of elements of T is $\mathbf{0}$. Minimality of S implies $S = T$, so S sums to $\mathbf{0}$.

Form the word b of length n whose i th bit is 1 iff row i of $\mathbf{H}$ is in S . Thus $\|b\| = |S| = s > 0$. And $b\mathbf{H}$ is exactly the sum of words in S . So $b\mathbf{H} = \mathbf{0}$ and $b \in C \setminus \{\mathbf{0}\}$. By Theorem 4.2.1, $\delta \leq \|b\| = s$.

$$s \leq \delta$$

By definition of δ there exists $c \in C \setminus \{\mathbf{0}\}$ with $\|c\| = \delta$. Let R be the set of rows of $\mathbf{H}$ corresponding to the positions where the bits of c are 1. Thus $|R| = \|c\| = \delta$.

Now $c \in C$ implies $\mathbf{0} = c\mathbf{H} =$ the sum of the elements of R . So R is a linearly dependent set of size δ . Minimality of S implies $s \leq \delta$. $\square$

- ★ No set consisting of a single row is dependent, unless $\mathbf{0}$ is a row of $\mathbf{H}$.
- ★ A set of two rows can only be dependent if the two rows are equal, which only happens if $\mathbf{H}$ has repeated rows.
- ★ A set of three rows can only be dependent if one row of $\mathbf{H}$ is the sum of two others.

4.9.2 Example Determine the distance of the linear code C with parity check matrix

$$\mathbf{H} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Solution: Any 1 row is linearly independent, since no row is the zero vector. Any 2 (non-zero) rows will be independent, provided they are not equal. Any 4 vectors in $\mathbb{F}_2^3$ must be dependent, so the question is: is every set of 3 rows of $\mathbf{H}$ an independent set? No: row 2 is row 4 plus row 5. So the smallest dependent set of rows of $\mathbf{H}$ has size 3. So this code has distance $\delta = 3$. It can detect 2 errors and correct 1. $\otimes$

Before we can use $\mathbf{H}$ for error correction, we need one more concept from algebra.

4.10. Cosets

Suppose C is a linear code. During transmission, if error pattern $\mathbf{0}$ occurs, the received word will be in C . More generally, if error pattern e occurs the received word will be in $\{c + e \mid c \in C\}$. In Definition 2.1.10 we abbreviated this set by $C + e$. (Note that $C + \mathbf{0} = C$). The set $C + e$ is called a *coset* of C .

4.10.1 Definition If C is a linear code of length n and $u \in \mathbb{F}_2^n$, then the *coset of C containing u* is the set $\{c + u \mid c \in C\}$. We denote this coset by $C + u$ (or $u + C$).

The coset $(C + e)$ containing e is exactly the set of all possible received words if error pattern e occurred.

4.10.2 Example Let $C = \{000, 111\}$ and $u = 101$. Then

$$C + u = C + 101 = \{000 + 101, 111 + 101\} = \{101, 010\}.$$

Notice that if we consider the coset of C containing a codeword, then we get C .

$$C + 111 = \{000 + 111, 111 + 111\} = \{111, 000\} = C.$$

This occurs because C is closed under addition.

The following theorem gives some important results on cosets.

4.10.3 Theorem [Properties of Cosets] Let C be a linear (n, k) -code. Let $u, v \in \mathbb{F}_2^n$. Then:

- (1) $u \in C + u$.
- (2) $u + v \in C \iff u \in C + v \iff C + u = C + v$.
- (3) Every word in $\mathbb{F}_2^n$ is contained in exactly one coset of C : that is, either $C + u = C + v$ or $(C + u) \cap (C + v) = \emptyset$.
- (4) $|C + u| = |C|$.
- (5) There are 2^{n-k} different cosets of C . Each coset contains 2^k words.
- (6) The code C is itself one of the cosets.

Proof

(1) Because C is linear, it contains the zero vector. So $0 + u \in C + u$ so $u \in C + u$.

(2) We first prove that $u + v \in C \implies u \in C + v$. If $u + v \in C$, then $u + v = c$ for some $c \in C$. Thus, $u = c + v$ and so $u \in C + v$.

We now prove that $u \in C + v \implies C + u = C + v$. If $u \in C + v$, then $u = c + v$ for some $c \in C$. Consider any element $c_1 + u \in C + u$. We have $c_1 + u = c_1 + c + v \in C + v$. So $C + u \subseteq C + v$. Similarly, if $c_2 + v$ is any element of $C + v$, then $c_2 + v = c_2 + c + u \in C + u$. So $C + v \subseteq C + u$ and we have $C + u = C + v$.

We now prove that $C + u = C + v \implies u + v \in C$. If $C + u = C + v$, then $u \in C + u = C + v$. Thus, $u = c + v$ for some $c \in C$ and so $u + v = c \in C$.

(3) We show that the relation $\sim$ on $\mathbb{F}_2^n$ given by $u \sim v \iff u + v \in C$ is an equivalence relation. Reflexive: $u + u = 0 \in C$ so $\sim$ is reflexive. Symmetric: $u + v \in C$ implies $v + u \in C$, so $\sim$ is symmetric. Transitive: $u + v \in C$ and $v + w \in C$ implies $u + w = u + v + v + w \in C$, so $\sim$ is transitive. By (2) the equivalence classes of this relation are the cosets, and this gives us the result.

(4) Consider the function $f : C \rightarrow C + u$ given by $f(c) = c + u$ for all $c \in C$. If $c_1 + u = c_2 + u$, then $c_1 = c_2$ so f is injective, and f is clearly surjective so $|C + u| = |C|$.

(5) The cosets partition $\mathbb{F}_2^n$ by (3). By (4), each coset has size $|C + u| = |C| = 2^k$, so there must be exactly 2^{n-k} of them.

(6) C is the coset $C + 0$. □

4.10.4 Example List all of the cosets of the linear code $C = \{0000, 1011, 0101, 1110\}$.

Solution: We apply Theorem 4.10.3. First, C is itself a coset (Theorem 4.10.3(6)).

Next, each word in $\mathbb{F}_2^4$ is contained in a unique coset (by (3)), so we obtain another coset by picking a word in $\mathbb{F}_2^4 \setminus C$. If we take the word 1000, we have

$$C + 1000 = \{1000, 0011, 1101, 0110\}.$$

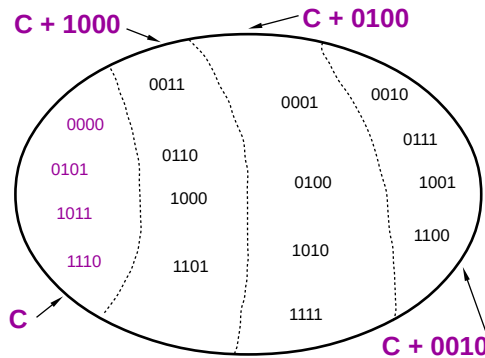
(Note that 1000 is in the coset by (1).)

Now pick another word in $\mathbb{F}_2^4$, and not in either coset so far, say 0100. Then

$$C + 0100 = \{0100, 1111, 0001, 1010\}.$$

Pick another word in $\mathbb{F}_2^4$ not in any coset so far, say 0010.

$$C + 0010 = \{0010, 1001, 0111, 1100\}.$$



This is all the cosets: the code C has dimension $k = 2$, we have listed $2^{n-k} = 2^{4-2} = 4$ cosets each with $2^k = 2^2 = 4$ words (5), and every word in $\mathbb{F}_2^4$ appears in exactly one coset (3). Also, note that two words are in the same coset iff their sum is in C (see (2)). $\circledast$

4.11. IMLD for Linear Codes

One of the fundamental goals of coding theory is to design codes which permit easy and rapid decoding of a received word. We describe such a method for IMLD for a linear code, using the parity check matrix and cosets of the code.

4.11.1 Lemma Let C be a linear code with parity check matrix H . Then $uH = vH \iff C + u = C + v$.

Proof $uH = vH \iff (u + v)H = \mathbf{0} \iff u + v \in C \xrightarrow{4.10.3} C + u = C + v.$ $\square$

Let C be a linear code. Assume the codeword $c \in C$ is transmitted and the word w is received, with an error pattern $e = c + w$. Clearly, $w + e = c \in C$, so $C + w = C + e$. That is, the error pattern e is in the same coset as the received word w by Theorem 4.10.3 (2).

4.11.2 Definition Any word of minimal weight in a coset is called a coset leader for that coset.

There may be several words of equal smallest weight, so the coset leader is not necessarily unique.

In IMLD we always assume the smallest possible weight error pattern. Thus:

4.11.3 Algorithm [Decoding using cosets] Suppose C is a linear code and the word w is received. If the coset $C + w$ has a unique coset leader e , conclude that $c = w + e$ was the word sent.

If a coset has more than one coset leader, IMLD will fail for all words in this coset.

4.11.4 Example Let $C = \{0000, 1011, 0101, 1110\}$ as in Example 4.10.4. The cosets of C are:

Coset 1	Coset 2	Coset 3	Coset 4
0000	1000	0100	0010
1011	0011	1111	1001
0101	1101	0001	0111
1110	0110	1010	1100

Suppose we receive $w = 1101$. This is in the second coset in the above table, so choose $e = 1000$ as the error pattern of least weight. Hence we conclude that $c = w + e = 1101 + 1000 = 0101$ was the codeword most likely sent.

If $w = 1111$ is received, then the words 0100 and 0001 have equal least weight in the corresponding coset. Thus IMLD fails.

The hardest part of this procedure is to find the coset containing the received word w and then to find the word of least weight in the coset. We can use the parity check matrix to do this.

4.11.5 Definition Let C be a linear (n, k) -code with parity check matrix H . For any word $w \in \mathbb{F}_2^n$, define the *syndrome* of w to be wH . This is a word in $\mathbb{F}_2^{n-k}$.

The point of this is that *words from the same coset have the same syndrome*: Words u and v are in the same coset iff $u + v \in C$ (Theorem 4.10.3(2)) iff

$$0 = (u + v)H = uH + vH \iff uH = vH.$$

So the syndrome wH will immediately tell us which coset w is in. The error pattern for w must be one of the words in its coset. Thus the coset leader of this coset will be the error pattern used to correct w .

Notice that H has rank $n - k$ and so the dimension of its row space is $n - k$. That is, the dimension of $\{vH \mid v \in \mathbb{F}_2^n\} \subseteq \mathbb{F}_2^{n-k}$ is $n - k$. Thus, $\{vH \mid v \in \mathbb{F}_2^n\} = \mathbb{F}_2^{n-k}$. Since there are 2^{n-k} cosets, and since words from the same coset have the same syndrome, this means that $C + w \mapsto wH$ is a bijection from the cosets of C to $\mathbb{F}_2^{n-k}$ (the set of all possible syndromes).

4.11.6 Example For the code C defined in Example 4.11.4, a parity check matrix is given by

$$H = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Calculate the syndrome of the received word $w = 1101$. Then calculate the syndrome of the word $e = 1000$, which is the coset leader for the coset $C + w$.

Solution:

$$1101 H = 1 \ 1 \ 0 \ 1 \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = 1 \ 1.$$

Similarly $1000 H = 11$. Notice that the syndromes of w and e are the same because w and e are in the same coset.

IMLD concludes that $v = u + w = 1101 + 1000 = 0101$ was sent, so there was an error in the first bit.

The syndrome eH is equal to the row of H (the first) corresponding to the location of the most likely error. In this way the syndrome indicates the “symptoms” of the error. $\otimes$

In summary:

4.11.7 Theorem Let C be a linear code of length n , H be a parity check matrix for C and $w, e \in \mathbb{F}_2^n$. Then

- ★ $wH = 0$ iff w is a codeword in C .
- ★ $wH = eH$ iff w and e lie in the same coset of C .
- ★ w is corrected using the coset leader e from the coset containing w (if unique).
- ★ If e is the error pattern for a received word w then eH is the sum of the rows of H corresponding to the positions where errors occurred in transmission.

Note that if no errors occur in transmission and w is received then $wH = 0$. But $wH = 0$ does not necessarily imply that no errors occurred, since the codeword w received is not necessarily the same as the codeword sent.

Since words in the same coset have the same syndrome and words in different cosets have different syndromes, we can identify a coset by the syndrome of any word in the coset. Thus for an (n, k) -code, the 2^{n-k} words of length $n - k$ each occur as the syndrome of exactly one of the 2^{n-k} cosets.

4.11.8 Example Let C be the code from Example 4.11.4, with parity check matrix H given in Example 4.11.6. For each coset we can calculate the syndrome using the coset leader:

Coset leader e	Syndrome eH
0000	00
1000	11
0100 or 0001	01
0010	10

Note again that each word of length 2 occurs precisely once as a syndrome.

4.11.9 Definition The table in Example 4.11.8 which matches each syndrome with its coset leader(s) is called a *standard decoding array* or *SDA*. To construct an SDA, first list all of the cosets for the code and choose from each coset the word(s) of least weight as coset leader(s) e . Then find a parity check matrix H for the code and for each coset leader e calculate its syndrome eH .

4.11.10 Example Let $C = \{00000, 10100, 01011, 11111\}$ (this is linear). Construct an SDA for C .

Solution: First we calculate the cosets of C :

$$\begin{aligned}
 C &= \{00000, 10100, 01011, 11111\} \\
 10000 + C &= \{10000, 00100, 11011, 01111\} \\
 01000 + C &= \{01000, 11100, 00011, 10111\} \\
 00010 + C &= \{00010, 10110, 01001, 11101\} \\
 00001 + C &= \{00001, 10101, 01010, 11110\} \\
 11000 + C &= \{11000, 01100, 10011, 00111\} \\
 10010 + C &= \{10010, 00110, 11001, 01101\} \\
 10001 + C &= \{10001, 00101, 11010, 01110\}
 \end{aligned}$$

Next we find a parity check matrix $\mathbf{H}$. We can use generating matrix

$$\mathbf{G} = \left(\begin{array}{cc|ccc} 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{array} \right) = (\mathbf{I} \ \mathbf{X}).$$

By algorithm 4.6.13

$$\mathbf{H} = \left(\begin{array}{c} \mathbf{X} \\ \mathbf{I} \end{array} \right) = \left(\begin{array}{ccccc} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{array} \right).$$

Note that $\delta = 2$, so C cannot always correct a single error.

Our SDA is:

Coset	Coset Leader(s)	Syndrome	Detect?	Correct?
C	00000	000		✓
$10000 + C$	10000, 00100	100	✓	X
$01000 + C$	01000	011	✓	✓
$00010 + C$	00010	010	✓	✓
$00001 + C$	00001	001	✓	✓
$11000 + C$	11000, 01100	111	✓	X
$10010 + C$	10010, 00110	110	✓	X
$10001 + C$	10001, 00101	101	✓	X

Recall: error pattern $e \neq \mathbf{0}$ cannot be detected iff $e \in (C + C) \setminus \{\mathbf{0}\} = C \setminus \{\mathbf{0}\}$ by linearity. $\otimes$

Having constructed an SDA, it is easy to find the codeword corresponding to the received word w using IMLD. We first calculate the syndrome $e\mathbf{H}$, then find the coset leader e in the SDA with the same syndrome (so $e\mathbf{H} = w\mathbf{H}$). Then we conclude that $v = w + e$ was the most likely transmitted codeword, or request retransmission.

Construct a table of syndromes & coset leaders (SDA). If w is received, calculate $s = w\mathbf{H}$. Look up which coset leader(s) correspond to s . Use the coset leader to correct w .

4.11.11 Example Let C be the code given in Example 4.11.10. Use the method of syndrome decoding to find the codewords corresponding to each of the received words $w_1 = 10101$, $w_2 = 01110$ and $w_3 = 00011$.

Solution:

$$s_1 = w_1\mathbf{H} = 1 \ 0 \ 1 \ 0 \ 1 \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = 0 \ 0 \ 1.$$

Similarly, $s_2 = w_2\mathbf{H} = 101$ and $s_3 = w_3\mathbf{H} = 011$.

The error pattern for w_1 is in the same coset as w_1 , so we assume the error pattern is the coset leader for that coset. Since w_1 has the same syndrome as coset leader 00001, w_1 is in the same coset as 00001 and we decode to $w_1 + 00001 = 10100$. Similarly w_3 decodes to $w_3 + 01000 = 01011$. However w_2 is in a coset with 2 coset leaders, so we cannot decode (IMLD).

We check that the method of syndrome decoding agrees with decoding by choosing the closest codeword. We decoded $w_1 = 10101$ to $v_1 = 10100$. By comparing the distance from w_1 to each of the codewords: $d(00000, 10101) = 3$, $d(10100, 10101) = 1$, $d(01011, 10101) = 4$ and $d(11111, 10101) = 2$, we see that v_1 is the closest codeword to w_1 .

We requested retransmission for the received word $w_2 = 01110$. This is the correct outcome since $d(11111, 01110) = 2$ and $d(01011, 01110) = 2$. $\otimes$

An SDA for an arbitrary linear code has 2^{n-k} syndromes and coset leaders to store, which may still require a lot of memory. However, for certain linear codes we can get away with storing much less:

4.11.12 Example Let C be the code with generating matrix $\mathbf{G}$ and parity check matrix $\mathbf{H}$ where

$$\mathbf{G} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix} \quad \mathbf{H} = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Calculate n , k and δ for this code, and find all the coset leaders.

Write down the results of the encoding, transmission and decoding processes, starting with the message word $m = 0101$ and assuming that a single error in position 4 occurs during transmission.

Solution: For future use, note that the 7 rows of $\mathbf{H}$ are all possible non-zero binary vectors of length 3, each occurring once.

Since $\mathbf{G}$ is a 4×7 matrix, $n = 7$ and $k = 4$. To find δ , find the smallest dependent set among the rows of $\mathbf{H}$. The rows in $\mathbf{H}$ are all non-zero (so $\text{rank} \geq 1$) and distinct (so $\text{rank} \geq 2$). But row 6 plus row 7 is row 4, so $\delta = 3$ (Theorem 4.9.1).

So C is a $(7, 4, 3)$ -code. It is 2-error detecting, and 1-error correcting. So all error patterns of weight ≤ 1 can be corrected. There are exactly 8 such patterns (words of length 7 with weight ≤ 1). Therefore all 8 of these words must be in distinct cosets, where they are coset leaders.

Now $|C| = 2^4$ inside $\mathbb{F}_2^7$, so there are $2^{7-4} = 8$ cosets, each with $|C| = 16$ elements. But we have 8 coset leaders, so there is exactly 1 coset leader in each coset, and each coset leader is weight 0 or 1. So the code corrects all error patterns of weight ≤ 1 , and nothing else.

For example, error pattern $e = 0001000$ must be a coset leader. Its syndrome is $e\mathbf{H} = 011$, so all words with syndrome 011 are corrected using e . In fact, if $e = 0001000$ then $e\mathbf{H}$ will be the 4th row of $\mathbf{H}$. So all words whose syndrome is the 4th row of $\mathbf{H}$ will be corrected by altering their 4th bit.

Thus for this code:

- ★ Receive word w . Calculate its syndrome $s = w\mathbf{H}$.
- ★ If $s = \mathbf{0}$, w requires no correction.
- ★ Otherwise, the syndrome will be the j th row of $\mathbf{H}$, for some unique j . Correct bit j of w .

This works because the error pattern corresponding to row j of $\mathbf{H}$ has 1 in the j th position and 0s elsewhere, and is a coset leader.

For this code we do not even have to calculate and store the SDA. We only need $\mathbf{G}$ and $\mathbf{H}$.

To encode, note that $\mathbf{G} = (\mathbf{I} \ \mathbf{X})$, so C is a systematic code. Thus, to encode m , calculate $m\mathbf{G}$:

$$m\mathbf{G} = 0 \ 1 \ 0 \ 1 \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix} = 0 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1$$

So m is encoded as $c = 0101101$. Note that m occurs as the first 4 bits of c .

If an error occurs in position 4, then the received word is $w = c + 0001000 = 0100101$.

We receive w . Calculate its syndrome

$$s = w\mathbf{H} = 0 \ 1 \ 0 \ 0 \ 1 \ 0 \ 1 \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = 0 \ 1 \ 1.$$

Since the syndrome is not 000, one or more errors have occurred. The syndrome is the 4th row of $\mathbf{H}$. By the discussion above, the coset leader corresponding to this syndrome is $e = 0001000$. Thus we decode to $w + e = 0001000 = 0101101$. Because the code is systematic, the message word was 0101. *

This code (called a *Hamming code*) is an optimal example of a 1-error correcting code, in a certain sense. We shall come back to a more detailed discussion of it in a later section.

4.12. Encoding and Decoding Summary

We summarize the information transmission process for a linear (n, k) -code. The possible messages are the 2^k words of length k , that is, all the vectors in the vector space $\mathbb{F}_2^k$. The codewords are 2^k of the possible 2^n words of length n , and the set of all these codewords, the code, is a subspace of the vector space $\mathbb{F}_2^n$. The received words are all the possible words of length n . Let $\mathbf{G}$ and $\mathbf{H}$ be a generating matrix and parity check matrix, respectively.

- ★ Message word m of length k
- ★ Codeword $c = m\mathbf{G}$ of length n
- ★ Received word w of length n
- ★ Compute $w\mathbf{H}$
- ★ Most likely error pattern e is the coset leader of the coset with syndrome $w\mathbf{H}$.
- ★ Most likely codeword is $c' = w + e$

4.13. Reliability of IMLD for Linear Codes

Let C be a linear (n, k) -code, transmitted over a BSC of reliability p with $\frac{1}{2} < p < 1$, decoded via IMLD.

Let $Q_C(c)$ be the probability that if codeword c was sent then the received word it is corrected to is c , ie that the error correcting “works properly”. This will be an expression in p . The overall reliability of the code is then³

$$\frac{1}{|C|} \sum_{c \in C} Q_C(c).$$

In general this is a large calculation, because for each c we must consider all possible error patterns and their probabilities, and determine which patterns are corrected properly, and this will depend on c .

For linear codes we shall show that $Q_C(c)$ is independent of c . Thus we can write Q_C instead of $Q_C(c)$, and then the reliability of the code is exactly Q_C .

Let $\mathcal{E}$ be the set of unique coset leaders of C . Suppose c is sent. Let R_c be the set of words that are strictly closer to c than to any other codeword. These are the words that will be correctly dealt with under IMLD and corrected back to c .

Then

$$\begin{aligned} u \in R_c &\iff d(u, c) < d(u, c') \quad \forall c' \in C \setminus \{c\} \\ &\iff \|u + c\| < \|u + c'\| \quad \forall c' \in C \setminus \{c\} \\ &\iff c + u \in C + u \text{ has smaller weight than any other word in } C + u \\ &\iff e = c + u \text{ is the unique coset leader of } C + u \\ &\iff e = c + u \in \mathcal{E} \\ &\iff u \in c + \mathcal{E}. \\ \therefore R_c &= c + \mathcal{E} \end{aligned}$$

Thus:

$$\begin{aligned} Q_C(c) &= \sum_{w \in R_c} \mathbb{P}_p(c \rightarrow w) \\ &= \sum_{\substack{w=c+e \\ e \in \mathcal{E}}} p^{n-\|e\|} (1-p)^{\|e\|} \quad \text{by Equation 3.5.1} \\ &= \sum_{e \in \mathcal{E}} p^{n-\|e\|} (1-p)^{\|e\|} \end{aligned}$$

We see that this final expression is independent of c .

4.13.1 Example Let C be the linear code from Example 4.11.10. Determine the reliability of IMLD for the code C using a BSC with reliability $p = 0.97$.

Solution: The set of unique coset leaders is $\mathcal{E} = \{00000, 01000, 00010, 00001\}$.

So $Q_C = p^5(1-p)^0 + 3p^4(1-p) = p^4(p + 3 - 3p) = p^4(3 - 2p) \simeq 0.938$. *

³This assumes every codeword is equally likely to be sent. This could be the case if messages are compressed and/or encrypted before sending. If not, one could take a sum weighted by the probability of each word being sent.

4.14. Generalising to larger fields: q -ary codes

We adjust the definition of a binary block code given earlier, to define a q -ary code.

4.14.1 Definition Let $n \geq k$ be positive integers. A (q -ary block) code consists of:

- ★ A set $M \subseteq \mathbb{F}_q^k$ of message words together with
- ★ An injective function called the *encoding function* $f: M \rightarrow \mathbb{F}_q^n$.

We write $C = f(M)$, the image of f , and (abusing terminology) call C the *code*. Elements in C are called *codewords*. The *length* of the code is n , and the *message length* or *dimension* of the code is k .

A code C is an (n, k) q -ary code if it has codewords of length n and $|C| = q^k$.

Note that the distance between two words x and y in $\mathbb{F}_q^n$ is the number of places in which they differ. The relationships between distance and error detection and correction are the same for q -ary codes as for binary codes. Note that when codeword c is transmitted and error pattern e occurs, then the received word is $w = c + e$, but that for non-binary codes, this means $e = w - c$ and $c = w - e$.

The concept of a generating matrix, parity check matrix, dual code, and syndrome decoding all generalise naturally from binary to q -ary codes. Note however, the following important difference in determining a parity check matrix from a generating matrix in standard form. Let

$$\mathbf{G} = (\mathbf{I}_k \ \mathbf{X})$$

where $\mathbf{X}$ is some $k \times (n - k)$ matrix, be a generating matrix for an (n, k) q -ary code. Then

$$\mathbf{H} = \begin{pmatrix} -\mathbf{X} \\ \mathbf{I}_{n-k} \end{pmatrix}$$

is a parity check matrix for this code. By block multiplication, we see that

$$\mathbf{G} \mathbf{H} = (\mathbf{I}_k \ \mathbf{X}) \begin{pmatrix} -\mathbf{X} \\ \mathbf{I}_{n-k} \end{pmatrix} = -\mathbf{I}_k \mathbf{X} + \mathbf{X} \mathbf{I}_{n-k} = \mathbf{0}_{k \times (n-k)}.$$

Furthermore, when the most likely error pattern has been determined, it must be **subtracted** from the received word in order to determine the most likely codeword.

4.14.2 Example Consider the ternary code with generating matrix

$$\mathbf{G} = \begin{pmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 2 & 2 \end{pmatrix}$$

Here $n = 4$, $k = 2$, and we have $3^2 = 9$ codewords.

The code and its cosets are as follows.

$$\begin{aligned} C &= \{0000, 1012, 0122, 2021, 0211, 1101, 1220, 2110, 2202\} \\ C + 1000 &= \{1000, 2012, 1122, 0021, 1211, 2101, 2220, 0110, 0202\} \\ C + 0100 &= \{0100, 1112, 0222, 2121, 0011, 1201, 1020, 2210, 2002\} \\ C + 0010 &= \{0010, 1022, 0102, 2001, 0221, 1111, 1200, 2120, 2212\} \\ C + 0001 &= \{0001, 1010, 0120, 2022, 0212, 1102, 1221, 2111, 2200\} \\ C + 2000 &= \{2000, 0012, 2122, 1021, 2211, 0101, 0220, 1110, 1202\} \\ C + 0200 &= \{0200, 1212, 0022, 2221, 0111, 1001, 1120, 2010, 2102\} \\ C + 0020 &= \{0020, 1002, 0112, 2011, 0201, 1121, 1210, 2100, 2222\} \\ C + 0002 &= \{0002, 1011, 0121, 2020, 0210, 1100, 1222, 2112, 2201\} \end{aligned}$$

(a) Encode the message words $m_1 = 12$ and $m_2 = 21$.

- (b) Determine the distance of this code.
 (c) Construct a standard decoding array for this code and determine the most likely codeword corresponding to each of the following received words 2021, 1110, 0102.

Solution:

- (a) The message word m_1 corresponds to the codeword 1220 since

$$m_1 \mathbf{G} = (1 \ 2) \begin{pmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 2 & 2 \end{pmatrix} = (1 \ 2 \ 2 \ 0)$$

The message word m_2 corresponds to the codeword 2110 since

$$m_2 \mathbf{G} = (2 \ 1) \begin{pmatrix} 1 & 0 & 1 & 2 \\ 0 & 1 & 2 & 2 \end{pmatrix} = (2 \ 1 \ 1 \ 0)$$

- (b) The minimum weight non-zero codeword has weight 3, so the distance of the code is 3. It is 1-error correcting.
 (c) A parity check matrix for this code is

$$\mathbf{H} = \begin{pmatrix} 2 & 1 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Using this parity check matrix, an SDA is as follows.

coset leader	syndrome
0000	00
1000	21
0100	11
0010	10
0001	01
2000	12
0200	22
0020	20
0002	02

The received word 2021 corresponds to codeword 2021 since

$$(2 \ 0 \ 2 \ 1) \begin{pmatrix} 2 & 1 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = (0 \ 0).$$

The received word 1110 corresponds to codeword 2110 since

$$(1 \ 1 \ 1 \ 0) \begin{pmatrix} 2 & 1 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = (1 \ 2), \text{ and } 1110 - 2000 = 2110.$$

The received word 0102 corresponds to codeword 0122 since

$$(0 \ 1 \ 0 \ 2) \begin{pmatrix} 2 & 1 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = (1 \ 0), \text{ and } 0102 - 0010 = 0122.$$

⊗

Bounds on Codes

We now look at the problem of determining how the parameters n , k , δ of a linear code are constrained. Given values for two of these three parameters, what range of values is possible for the third? This problem has not been solved in general, although it has been settled for many small values of n , k and δ .

5.0.1 Example Consider a linear $(40, 20)$ -code. How large can δ be? As of 2008 it was known that the maximal δ value is either 9 or 10. A construction of a $(40, 20, 9)$ code is known, and theoretical results imply $\delta \leq 10$, but it is not known if a $(40, 20, 10)$ code exists.

5.1. The Singleton Bound

We develop some bounds. First we have a fairly intuitive observation: given a linear (n, k, δ) code it is always possible to “degrade” it to produce a worse code. (This is mostly needed so that we can state some results more cleanly later on.)

5.1.1 Lemma [Spoiling Lemma] If there exists a linear (n, k, δ) code, then there exists a linear (n, k, d) code for $d = 1, 2, \dots, \delta$.

Proof If $\delta = 1$ the result is trivial, so let C be an (n, k, δ) code with $\delta > 1$. It suffices to construct a $(n, k, \delta - 1)$ code, since by iterating this construction we obtain the stated result.

For any word $u \in \mathbb{F}_2^n$, let $u(j)$ denote the j th bit of the word u . Fix a codeword $c^* \in C$ of exact weight δ . Let i be a fixed index for which $c^*(i) = 1$. Define a function $f: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ that sets bit i to 0, and leaves all other bits unchanged:

$$f(b_1 b_2 \cdots b_{i-1} b_i b_{i+1} \cdots b_n) = b_1 b_2 \cdots b_{i-1} 0 b_{i+1} \cdots b_n.$$

Thus, for each $u \in \mathbb{F}_2^n$, we have $\|f(u)\| = \|u\|$ or $\|u\| - 1$.

Let $C' = f(C)$. We show that C' is a linear $(n, k, \delta - 1)$ -code.

It is clear that C' has length n . So it remains to show that C' is linear, has dimension k , and has distance $\delta - 1$.

- (a) Let $u', v' \in C'$. Then $u' = f(u)$ and $v' = f(v)$ for some $u, v \in C$. In each position $j \neq i$ we have $u'(j) + v'(j) = u(j) + v(j)$. And in position $j = i$ we have $u'(j) + v'(j) = 0 + 0 = 0$. Thus, $u' + v' = f(u + v)$. Since C is linear $u + v \in C$, and so we have $u' + v' = f(u + v) \in C'$. Thus, C' is linear.
- (b) If $f(b) = f(c)$, then $d(b, c) \leq 1$, and this implies $b = c$ because that $\delta > 1$. Thus, f is injective and so the restriction of the map f to C is a bijection from C to C' . Thus, $\dim C' = \dim C = k$.
- (c) We noted above that for each $u \in \mathbb{F}_2^n$, we have $\|f(u)\| = \|u\|$ or $\|u\| - 1$. Thus, since C' is linear and since the distance of a linear code equals the minimum weight of a non-zero

codeword, the distance of C' is at least $\delta - 1$. But $\|f(c^*)\| = \delta - 1$ so the distance of C' is exactly $\delta - 1$. $\square$

5.1.2 Theorem [Singleton Bound] In any code (linear or not), $|C| \leq 2^{n-\delta+1}$. In particular, if $|C| = 2^k$, then

$$(5.1.1) \quad n \geq k + \delta - 1.$$

Proof Let C be an (n, k, δ) code. Consider the (projection) map $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{n-\delta+1}$ that deletes the first $\delta - 1$ bits of every vector. Since the distance of C is δ , the images under f of distinct codewords remain distinct (in other words, $f|_C$ is injective). In particular $|C| = |f(C)|$. Hence $|C| \leq 2^{n-\delta+1}$.

Putting $|C| = 2^k$, and taking logs on both sides shows $k \leq n - \delta + 1$, which gives the result $n \geq k + \delta - 1$. $\square$

The Singleton Bound is not very sharp (we can rarely find codes which meet this bound). If we divide the bound through by n we get a relativized version with the rate r and relative distance ∂ :

$$(5.1.2) \quad r + \partial \leq 1 + \frac{1}{n}.$$

For large n we essentially have $r + \partial \leq 1$.

5.2. The Hamming Bound

We develop a very useful bound, with a bit of counting.

Observe that the number of distinct words of length n and weight t is

$$\binom{n}{t} = \frac{n!}{t!(n-t)!}.$$

5.2.1 Definition Let $v \in \mathbb{F}_2^n$ and let $r \geq 0$ be an integer. The (closed) *ball* of radius r centred at v is the set

$$B_r(v) = \{w \in \mathbb{F}_2^n \mid d(v, w) \leq r\}.$$

5.2.2 Lemma For any $r \leq n$ and $v \in \mathbb{F}_2^n$

$$|B_r(v)| = \binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{r}.$$

Proof Given a fixed word v of length n , every word of distance i from v is of the form $v + e$ where $\|e\| = i$. So there are $\binom{n}{i}$ words at distance i from v . Considering the range $0 \leq i \leq r$ gives the required result. $\square$

5.2.3 Theorem [Hamming Bound] If C is a code of length n (linear or not) and distance δ then

$$|C| \leq \frac{2^n}{\binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{d}} \quad \text{where } d = \left\lfloor \frac{\delta - 1}{2} \right\rfloor.$$

Proof For any distinct codewords c_1 and c_2 , we must have $B_d(c_1) \cap B_d(c_2) = \emptyset$, since otherwise the minimum distance of the code would be at most $2d < \delta$. Considering the balls centred at each codeword gives the bound $|C| \cdot |B_d(c)| \leq 2^n$. The result follows from Lemma 5.2.2. $\square$

The easiest application of the Hamming bound is when n and δ are given. One obtains an upper bound on k . If $|C| = 2^k$ we get

$$(5.2.1) \quad \binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{d} \leq 2^{n-k}.$$

Taking logs in (5.2.1) gives a bound $k \leq n - (\cdots)$, hence $k \leq \lfloor n - (\cdots) \rfloor$. By (2.3.1): $m \in \mathbb{Z} \implies \lfloor m - \theta \rfloor = m - \lceil \theta \rceil$. Hence

$$(5.2.2) \quad k \leq n - \left\lceil \log_2 \left(\sum_{j=0}^{\lfloor \frac{\delta-1}{2} \rfloor} \binom{n}{j} \right) \right\rceil.$$

The Hamming bound is usually better than the Singleton bound, but not always.

5.2.4 Example If $\delta = 2$ the Hamming bound gives the trivial bound $|C| \leq 2^n$, so $k \leq n$. The Singleton bound gives $k \leq n - 1$ in this case.

5.2.5 Example We wish to design a code with 2^8 codewords, capable of correcting 2 errors. Use the Singleton and Hamming bounds to estimate how large n must be.

Solution: To correct 2 errors we need $\lfloor \frac{\delta-1}{2} \rfloor \geq 2$ so $\delta \geq 5$. Since $k = 8$, the Singleton bound gives $n \geq k + \delta - 1 = 12$. This does not mean that there necessarily exists a $(12, 8, 5)$ code, only that n must be ≥ 12 . So there are no $(11, 8, 5)$, $(10, 8, 5)$ etc codes.

For a code of these parameters the Hamming bound is: $2^8 \left[\binom{n}{0} + \binom{n}{1} + \binom{n}{2} \right] \leq 2^n$. Rearranging, we get that $n^2 + n + 2 \leq 2^{n-7}$. The rhs will eventually be much larger than the lhs. By inspection, the smallest value of n for which the inequality holds is 15. So the Hamming bound shows that (in addition to the parameters ruled out by the Singleton bound) no $(14, 8, 5)$, $(13, 8, 5)$ or $(12, 8, 5)$ codes exist.

(In fact it is known that there does exist a non-linear $(15, 8, 5)$ code and a linear $(16, 8, 5)$ code, but no linear $(15, 8, 5)$ code.) *

5.2.6 Example Consider the Hamming bound for the $(7, 4, 3)$ code C with generator matrix

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}$$

which was discussed at the end of the Section 4.11. We have $n = 7$, $\delta = 3$. Thus

$$|C| \leq \frac{2^7}{\binom{7}{0} + \binom{7}{1}} = \frac{128}{8} = 16.$$

And in fact $|C| = 16$, so the bound is sharp for this code. This means $\mathbb{F}_2^7$ is exactly covered by the balls of radius 1 centred on the 16 words of C .

5.3. The Griesmer Bound

There are many further bounds known, some of which are quite complicated. We give one more, which gives a bound on n if k , δ are known.

5.3.1 Theorem [Griesmer Bound] Let C be a linear (n, k, δ) code. Then

$$(5.3.1) \quad n \geq \sum_{j=0}^{k-1} \left\lceil \frac{\delta}{2^j} \right\rceil.$$

Proof Let $L(k, \delta)$ denote the minimum length of a linear code of dimension k and distance δ . Roughly speaking we relate $L(k, \delta)$ to $L(k-1, \delta/2)$, and use induction on k .

Heuristically for fixed k , $L(k, d)$ should be an increasing function of d . Indeed there exists a linear $(L(k, d'), k, d')$ code by definition of L , and hence if $d' \geq d > 0$ there exists a linear $(L(k, d'), k, d)$ code by the Spoiling Lemma 5.1.1. So the minimum length of a linear code of dimension k and distance d must be $\leq L(k, d')$. That is:

$$(5.3.2) \quad d' \geq d > 0 \implies L(k, d') \geq L(k, d).$$

Fix k and δ . Let C be a linear code of dimension k and distance δ of minimal length $n = L(k, \delta)$. By Theorem 4.2.1 there is a word $c \in C$ of exact weight δ , and no smaller weight non-zero word. Since equivalent codes have the same parameters, wlog let $c = (\underbrace{11 \cdots 1}_{\delta} \mid \underbrace{00 \cdots 0}_{n-\delta}) = (\mathbf{1}_\delta \mid \mathbf{0}_{n-\delta})$,

where $\mid$ denotes concatenation.

Extend $\{c\}$ to a basis for C . Let $\mathbf{G}$ be a corresponding generator matrix for this basis, with first row c . Thus C has a generating matrix

$$\mathbf{G} = \left(\begin{array}{ccc|ccc} 1 & \cdots & 1 & 0 & \cdots & 0 \\ * & * & * & & & \\ \vdots & \vdots & \vdots & & \mathbf{G}' & \end{array} \right).$$

Let the remaining rows of $\mathbf{G}$ be $(g_2 \mid g'_2), \dots, (g_k \mid g'_k)$, where each g_i is length δ , and each g'_i length $n - \delta$. We now show that $\{g'_2, \dots, g'_k\}$ is a linearly independent set. If not, $\sum_i g'_i = \mathbf{0}_{n-\delta}$ for some non-empty sum of the g'_i , so $\sum_i (g_i \mid g'_i) = (g \mid \mathbf{0}_{n-\delta}) = s$, say. Since s is a sum of rows of $\mathbf{G}$, $s \in C$, with $\|s\| = \|g\| \leq \delta$. If $0 < \|g\| < \delta$ then $0 < \|s\| < \delta$, contradiction. If $\|g\| = 0$ then $g = \mathbf{0}_\delta$ so $s = \mathbf{0}_n$ which says that the rows of $\mathbf{G}$ are linearly dependent, a contradiction. If $\|g\| = \delta$ then $g = \mathbf{1}_\delta$, so $s = c$ and $s + c = \mathbf{0}_n$. Since s does not involve the first row of $\mathbf{G}$, $s + c = \mathbf{0}$ is also a non-trivial linear combination of rows of $\mathbf{G}$, contradiction. We have proved that the $k-1$ rows of $\mathbf{G}'$ are linearly independent.

Let $f: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{n-\delta}$ be the linear (projection) map defined by deleting the first δ components of a vector. The code $f(C)$ is generated by $\{f(r) \mid r \text{ is a row of } \mathbf{G}\}$. But $f(c) = \mathbf{0}$ so we can remove it from the list of generators for $f(C)$. Thus $f(C)$ is generated by the rows of $\mathbf{G}'$, and these are linearly independent. Hence $\mathbf{G}'$ is a generating matrix for $f(C)$, so the parameters of the linear code $f(C)$ are $(n - \delta, k - 1, \delta')$ for some (unknown) δ' . Thus

$$(5.3.3) \quad n - \delta \geq L(k - 1, \delta')$$

We want to bound δ' in terms of δ .

Let $w \in f(C)$ be a non-zero word of minimal weight. So there exists $u \in C$ with $u = (v \mid w)$ and $\|w\| = \delta'$. Then

$$\begin{aligned}
 \delta &\leq \|u + c\| && u, c \in C, \quad u \neq c \implies 0 \neq u + c \in C \\
 &= \|(v \mid w) + (\mathbf{1}_\delta \mid \mathbf{0}_{n-\delta})\| \\
 &= \|(v + \mathbf{1}_\delta) \mid (w + \mathbf{0}_{n-\delta})\| \\
 &= \|v + \mathbf{1}_\delta\| + \|w + \mathbf{0}_{n-\delta}\| \\
 &= \delta - \|v\| + \|w\| && \text{Equation 2.2.2} \\
 &\leq \|u\| - \|v\| + \|w\| && 0 \neq u \in C \implies \|u\| \geq \delta \\
 &= \|v\| + \|w\| - \|v\| + \|w\| && u = (v \mid w) \\
 &= 2\|w\| = 2\delta'.
 \end{aligned}$$

So $\delta' \geq \delta/2$ so

$$(5.3.4) \quad \delta' \geq \left\lceil \frac{\delta}{2} \right\rceil.$$

$$\begin{aligned}
 \therefore \quad L(k, \delta) - \delta &= n - \delta && \text{Definition of } n \\
 &\geq L(k-1, \delta') && \text{Equation 5.3.3} \\
 &\geq L(k-1, \lceil \delta/2 \rceil) && \text{Equations 5.3.4, 5.3.2}
 \end{aligned}$$

Thus

$$(5.3.5) \quad L(k, \delta) \geq L(k-1, \lceil \delta/2 \rceil) + \delta.$$

The proof of Equation 5.3.1 now follows by induction on k .

For $k = 1$, the bound holds trivially: the rhs of 5.3.1 is $\lceil \delta \rceil = \delta$, and clearly $n \geq \delta$.

Assume Equation 5.3.1 holds for $k-1$, and that C is any linear (n, k, δ) code. Then

$$\begin{aligned}
 n &\geq L(k, \delta) && \text{Definition of } L(k, \delta) \\
 &\geq L(k-1, \lceil \delta/2 \rceil) + \delta && \text{Equation 5.3.5} \\
 &\geq \sum_{j=0}^{k-2} \left\lceil \frac{\lceil \delta/2 \rceil}{2^j} \right\rceil + \delta && \text{Inductive hypothesis } k-1 \\
 &= \sum_{j=0}^{k-2} \left\lceil \frac{\delta}{2^{j+1}} \right\rceil + \delta && \text{Lemma 2.3.7} \\
 &= \sum_{i=0}^{k-1} \left\lceil \frac{\delta}{2^i} \right\rceil && \text{Put } i = j+1.
 \end{aligned}$$

□

5.3.2 Example Repeat Example 5.2.5 using the Griesmer bound. In a linear $(n, 8, 5)$ code, how large must n be?

Solution: We have $n \geq \lceil 5 \rceil + \lceil 5/2 \rceil + \lceil 5/4 \rceil + \lceil 5/8 \rceil + \lceil 5/16 \rceil + \lceil 5/32 \rceil + \lceil 5/64 \rceil + \lceil 5/128 \rceil = 5 + 3 + 2 + 1 + 1 + 1 + 1 + 1 = 15$. This is the same result we achieved with the Hamming bound, but the calculation was simpler. $\otimes$

5.4. The Gilbert–Varshamov Bound

All of the bounds we have seen so far rule out the existence of codes with certain parameters. The next bound is different in that it gives an existence result.

The case $n = \delta$ is not interesting: if $n = \delta$ then any two codewords differ in every bit. The largest such linear code is $\{0, 1\}$. Similarly, $\delta = 1$ has no error correcting/detecting ability: also uninteresting.

5.4.1 Theorem [Gilbert–Varshamov (G–V)] Let $n, k, \delta \in \mathbb{N}$, with $n > \delta \geq 2$. If

$$(5.4.1) \quad \binom{n-1}{0} + \binom{n-1}{1} + \cdots + \binom{n-1}{\delta-2} < 2^{n-k}$$

then there exists a linear (n, k, δ) -code.

Proof Suppose that the integers n , k and δ satisfy 5.4.1. We will construct an $n \times (n - k)$ matrix H with linearly independent columns and in which no set of $\delta - 1$ or fewer rows is linearly dependent. This matrix is a parity check matrix for a linear (n, k, d) -code for some $d \geq \delta$, by Theorem 4.9.1. Thus, by the Spoiling Lemma 5.1.1, there exists a linear (n, k, δ) -code.

First note that the number of linear combinations of at most x vectors chosen from a set of y vectors is:

$$\binom{y}{0} + \binom{y}{1} + \binom{y}{2} + \cdots + \binom{y}{x}$$

since any such linear combination is the sum of i vectors from the set of y vectors, where $0 \leq i \leq x$.

Start building the matrix H by choosing the first $n - k$ rows of H to be the rows of the matrix I_{n-k} . This guarantees that the columns of H are linearly independent. Furthermore, we are guaranteed that so far, we have $n - k$ rows, no set of $\delta - 1$ of which are linearly dependent.

Now we repeatedly add rows, subject to the condition that each added row is a vector of length $n - k$ that is not a linear combination of $\delta - 2$ or fewer of the existing rows. Thus, at each stage we have an $i \times (n - k)$ matrix in which no set of $\delta - 1$ rows is linearly dependent. Then, for the $(i + 1)$ th row, we can choose any vector that is not a linear combination of $\delta - 2$ or fewer of the existing rows. That is, for the $(i + 1)$ th row, we can choose any one of

$$2^{n-k} - \left[\binom{i}{0} + \binom{i}{1} + \cdots + \binom{i}{\delta-2} \right]$$

vectors. So, provided that

$$(5.4.2) \quad 2^{n-k} > \left[\binom{i}{0} + \binom{i}{1} + \cdots + \binom{i}{\delta-2} \right],$$

we can extend our $i \times (n - k)$ matrix to an $(i + 1) \times (n - k)$ matrix in which no set of $\delta - 1$ rows is linearly dependent.

By our assumption 5.4.1, and since the right side of 5.4.2 increases with i , we can thus build the required $n \times (n - k)$ matrix. $\square$

We restate the G–V result as a sort of converse of the Hamming bound Equation 5.2.1.

5.4.2 Corollary If $n > \delta \geq 2$ then there exists a linear code C with length n and distance δ with

$$(5.4.3) \quad |C| \geq \frac{2^{n-1}}{\binom{n-1}{0} + \binom{n-1}{1} + \cdots + \binom{n-1}{\delta-2}}.$$

Proof The G–V bound guarantees the existence of a linear (n, k, δ) -code provided

$$\binom{n-1}{0} + \binom{n-1}{1} + \cdots + \binom{n-1}{\delta-2} < 2^{n-k}.$$

That is, if

$$(5.4.4) \quad 2^k < \frac{2^n}{\binom{n-1}{0} + \binom{n-1}{1} + \cdots + \binom{n-1}{\delta-2}}.$$

Let k' be the largest value of k for which inequality 5.4.4 is satisfied, so (by G–V) there does exist a linear (n, k', δ) code, C . We show that C satisfies inequality 5.4.3.

Inequality 5.4.4 is not satisfied for $k' + 1$, so

$$\begin{aligned} 2^{k'+1} &\geq \frac{2^n}{\binom{n-1}{0} + \binom{n-1}{1} + \cdots + \binom{n-1}{\delta-2}} \\ \therefore |C| = 2^{k'} &\geq \frac{2^{n-1}}{\binom{n-1}{0} + \binom{n-1}{1} + \cdots + \binom{n-1}{\delta-2}}. \quad \square \end{aligned}$$

Taking logs in (5.4.3) gives a bound $k \geq n - 1 - (\cdots)$, hence $k \geq \lceil n - 1 - (\cdots) \rceil$. By (2.3.1): $m \in \mathbb{Z} \implies \lceil m - \theta \rceil = m - \lfloor \theta \rfloor$. Hence there exists a linear (n, k, δ) code with

$$(5.4.5) \quad k \geq n - 1 - \left\lceil \log_2 \left(\sum_{j=0}^{\delta-2} \binom{n-1}{j} \right) \right\rceil.$$

Compare Equation 5.2.2.

Codes satisfying the G–V bound are rather trivial, and not of use in applications. The G–V bound is approximately what one might expect from a “random” linear code. We would hope to do considerably better in practice.

We generally use all of the previously discussed bounds together to determine a range of interesting parameters for coding theory. For example: given n and δ , G–V gives a range of values of k for which a code is known to exist, and the other bounds give a range where no code can exist. The gap in between these values is of interest to coding theorists.

5.4.3 Example What is the maximum rate of a 2-error correcting linear code with $n = 9$?

Solution: We have $\delta = 5$. The Hamming and G–V bounds (Equation 5.4.3) give:

$$2.7 < \frac{256}{93} = \frac{2^8}{\binom{8}{0} + \binom{8}{1} + \binom{8}{2} + \binom{8}{3}} \leq |C| \leq \frac{2^9}{\binom{9}{0} + \binom{9}{1} + \binom{9}{2}} = \frac{512}{46} < 11.2.$$

[Meaning: there exists C satisfying the left inequality, and all C must satisfy the right inequality.] So by G–V, there exists a code satisfying $|C| \geq 2.7$. Such a linear code must have at least 4 codewords, so $k \geq 2$. So in particular there exists a $(9, 2, 5)$ -code. Since $|C|$ is a power of 2 the Hamming bound shows $|C| \leq 8$. The remaining question is: can $k = 3$?

Apply the Griesmer bound with $n = 8$ and $k = 3$: $n \geq \lceil 5 \rceil + \lceil 5/2 \rceil + \lceil 5/4 \rceil = 5 + 3 + 2 = 10$. So a $(9, 3, 5)$ -code does not exist. Thus the largest linear code with $n = 9$ and $\delta = 5$ has $|C| = 4$. So the best rate is (only) $2/9$. ⊗

5.4.4 Example A linear $(11, 4, 5)$ -code is known to exist. How does this compare to the code in the previous example?

Solution: It has rate $4/11 \approx 0.364$ compared to $2/9 \approx 0.222$. This is about a 64% improvement in the rate, with the same total error correcting ability. However the relative error correcting ability has decreased from $2/9$ to $2/11 \approx 0.182$, which is about an 18% reduction. $\otimes$

5.5. Perfect Codes

A code is called perfect if the Hamming bound is achieved. In this case, there is some radius r such that every word in $\mathbb{F}_2^n$ lies in exactly one closed ball of radius r centred on a codeword. This means every word has a unique closest codeword, and we never have to request retransmission under IMLD. (This does not mean that the code never fails. It is possible—though improbable—that so many errors occur that a received word is closer to some codeword other than the one sent.)

5.5.1 Definition A code C of length n and odd distance $\delta = 2t + 1$ is called a *perfect code* if C attains the Hamming bound given in Theorem 5.2.3. That is, C is perfect iff

$$|C| = \frac{2^n}{\binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{t}}.$$

5.5.2 Theorem If C is a perfect code of length n and distance $\delta = 2t + 1$ then C will correct all error patterns of weight less than or equal to t , and no other error patterns. Every word in $\mathbb{F}_2^n$ lies in exactly one closed ball of radius t centred on a codeword.

Proof Suppose C is a perfect code of length n and distance $\delta = 2t + 1$. By Theorem 3.3.4 C will correct all error patterns of weight $\leq t$. This means that the balls of radius t centred on codewords are disjoint. By Lemma 5.2.2, each such ball contains

$$\binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{t}$$

words, and so the number of words occurring in all of these balls is

$$|C| \left(\binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{t} \right).$$

Since C is perfect, this is all 2^n words of $\mathbb{F}_2^n$, and it follows that no other error pattern is corrected. $\square$

Any code of distance $\delta = 2t + 1$ will correct all words that are distance $\leq t$ from a codeword (all words with error pattern of weight $\leq t$). However some words may not be within distance t of any codeword. Such words may still be closer to one codeword than any other, or they may be equally close to 2 or more different codewords.

A perfect code of distance $\delta = 2t + 1$ has the property that every word in $\mathbb{F}_2^n$ is within distance t of a unique codeword. Thus every received word will be corrected. We never have to request retransmission.

There are some trivial perfect codes, and we discuss these now.

5.5.3 Example Suppose there is a perfect code of distance 1, so $t = 0$. Then $\binom{n}{0} = 1 = 2^0$, so $|C| = \frac{2^n}{\binom{n}{0}} = 2^n$. The only code with 2^n codewords and length n is $\mathbb{F}_2^n$, so $\mathbb{F}_2^n$ is a perfect code. (But this code provides no error detection or correction.)

5.5.4 Example Consider the code of length $n = 2t + 1$ and distance $2t + 1$ with 2 codewords: $\mathbf{0}$ and $\mathbf{1}$. So $k = 1$. If $w \in \mathbb{F}_2^{2t+1}$ and $\|w\| \leq t$ then $w \in B_t(\mathbf{0})$. Otherwise $\|w\| \geq t + 1$ so $d(w, \mathbf{1}) = \|w + \mathbf{1}\| = 2t + 1 - \|w\| \leq t$. So every word is contained in exactly one of the balls $B_t(\mathbf{0})$, $B_t(\mathbf{1})$.

Or directly:

$$\begin{aligned} 2^{2t+1} &= \left[\binom{2t+1}{0} + \binom{2t+1}{1} + \cdots + \binom{2t+1}{t} \right] + \left[\binom{2t+1}{t+1} + \cdots + \binom{2t+1}{2t+1} \right] \\ &= 2 \left[\binom{2t+1}{0} + \binom{2t+1}{1} + \cdots + \binom{2t+1}{t} \right] \\ \therefore 2^{n-k} &= 2^{2t} = \binom{2t+1}{0} + \binom{2t+1}{1} + \cdots + \binom{2t+1}{t}. \end{aligned}$$

The codes from the two previous examples are called the *trivial* perfect codes.

If $t = 1$, then $\delta = 3$ and a perfect (n, k) -code must satisfy $2^{n-k} = \binom{n}{0} + \binom{n}{1} = n + 1$. The case $n + 1 = 2^2$ is the trivial $\{\mathbf{0}, \mathbf{1}\}$ code above. The smallest possible non-trivial perfect code would be $n + 1 = 8 = 2^3$ so $n = 7$ and $n - k = 3$, ie a $(7, 4, 3)$ -code. And indeed, we showed that the $(7, 4, 3)$ Hamming code of Example 5.2.6 is perfect: $\binom{7}{1} + \binom{7}{0} = 2^3$ and $\frac{2^7}{2^3} = 16$, which is the number of words in the Hamming code.

The following theorem completely determines the parameters for which a perfect (n, k, δ) -code exists.

5.5.5 Theorem There exists a nontrivial perfect (n, k, δ) -code if and only if $(n, k, \delta) = (2^r - 1, 2^r - 1 - r, 3)$ for some integer $r \geq 3$ or $(n, k, \delta) = (23, 12, 7)$. Moreover, any code with these parameters is perfect.

Proof In 1973, Tietavainen and van Lint proved that any perfect code has these parameters. This part of the proof is omitted. Perfect codes with the stated parameters will be constructed in later sections. So it remains to show that any code with these parameters is perfect.

First, let C be a $(2^r - 1, 2^r - 1 - r, 3)$ -code. We have $t = 1$ and

$$\frac{2^n}{\binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{t}} = \frac{2^{2^r-1}}{1 + 2^r - 1} = 2^{2^r-1-r} = 2^k = |C|.$$

Thus, any $(2^r - 1, 2^r - 1 - r, 3)$ -code is perfect.

Now let C be a $(23, 12, 7)$ -code. We have $t = 3$ and

$$\frac{2^{23}}{\binom{23}{0} + \binom{23}{1} + \binom{23}{2} + \binom{23}{3}} = \frac{2^{23}}{2048} = \frac{2^{23}}{2^{11}} = 2^{12} = |C|.$$

Thus, any $(23, 12, 7)$ -code is perfect. □

5.5.6 Example If there exists a perfect $(23, k, 7)$ -code, then what is the value of k ?

Solution: The Hamming bound for a $(23, k, 7)$ -code is

$$2^{23-k} \leq \binom{23}{0} + \binom{23}{1} + \binom{23}{2} + \binom{23}{3} = 2048 = 2^{11}.$$

So if $k = 12$ we have equality in the Hamming bound. Thus, if there exists a perfect $(23, k, 7)$ -code, then $k = 12$. ⊗

5.5.7 Example Show that a $(90, 78, 5)$ code would be a 2-error correcting perfect code.

Solution: We have $n = 90$, $k = 78$, $\delta = 5 = 2t + 1$ so $t = 2$. The Hamming bound is

$$2^{90-78} = 4096 \leq \binom{90}{0} + \binom{90}{1} + \binom{90}{2} = 1 + 90 + 4005 = 4096$$

and equality is achieved.

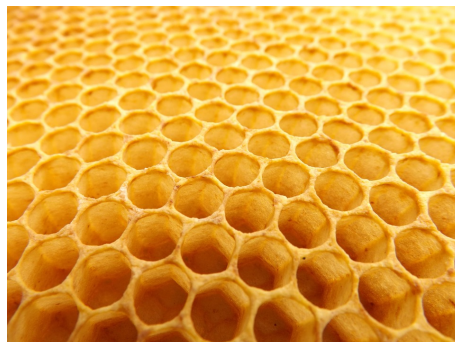
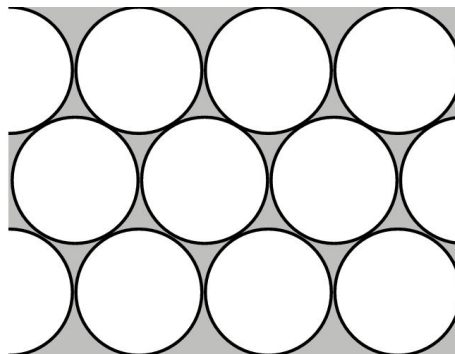
However by Theorem 5.5.5 no such code exists. ⊗

5.6. Interlude: Sphere Packing

A perfect code has the property that it partitions $\mathbb{F}_2^n$ precisely into identical non-overlapping spheres (really, balls). In this case 100% of the elements of $\mathbb{F}_2^n$ are covered by spheres. Clearly(?) this is impossible in $\mathbb{R}^n$ ($n \geq 2$).

5.6.1 Question Let $n \geq 2$. What is the limiting maximal density of space that can be covered by non-overlapping unit (hyper)-balls in $\mathbb{R}^n$?

(To make this rigorous one considers any packing of spheres, calculates the percentage of volume of a cube of side length s that is occupied by the packing, and considers the limit as $s \rightarrow \infty$.)



Honeycomb



Hexagonal close packing with Hales

n	Optimal Density	Structure	Proof
2	$\frac{\pi}{2\sqrt{3}} \simeq 0.9069$	Hexagonal lattice (honeycomb)	1940, Toth
3	$\frac{\pi}{3\sqrt{2}} \simeq 0.7405$	Hexagonal close packed (cannon balls)	(Conjectured by Kepler 1611) 1998, Hales
≥ 4	?	Unknown before 2016	
8	$\frac{\pi^4}{384} \simeq 0.2537$	E_8 lattice	2016, Viazovska
24	$\frac{\pi^{12}}{12!} \simeq 0.001930$	Leech lattice	2016, Viazovska et al

The existence of the Leech lattice in some way “explains” the existence of the unique perfect code with $\delta > 3$, namely the $(23, 12, 7)$ code that we shall construct later.

5.7. Constructing new codes from old

In this section we summarise several recursive constructions that allow us to create new linear codes by adapting or combining codes we already have.

Shortening

Let C be an (n, k, δ) linear code. Choose a value of $i \in \{1, 2, \dots, n\}$. For each value of i , we can obtain a new code S_i , called a shortening of C , where S_i is obtained by taking the codewords of C that have a 0 in position i , and shortening those words by deleting the 0 in position i .

5.7.1 Theorem If there exists a linear (n, k, δ) -code, then there exists a linear $(n - 1, k - 1, \delta)$ -code.

5.7.2 Example Let C be the $(5, 3, 2)$ -code with generating matrix $\mathbf{G} = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{pmatrix}$.

Thus $C = \{00000, 10110, 01011, 00101, 11101, 10011, 01110, 11000\}$.

The codewords with a zero in position 2 are $\{00000, 10110, 00101, 10011\}$, so shortening on position 2 gives the code

$$S_2 = \{0000, 1110, 0101, 1011\}.$$

Projection

Let C be an (n, k, δ) linear code. Choose a value of $i \in \{1, 2, \dots, n\}$. For each value of i , we can obtain a new code P_i , called a projection of C , where P_i is obtained by deleting the bit in position i from each codeword in C .

5.7.3 Theorem If there exists a linear (n, k, δ) -code with $\delta > 1$, then there exists a linear $(n - 1, k, \delta - 1)$ -code.

5.7.4 Example Let C be the $(5, 3, 2)$ -code from the previous example. The projection obtained from C by deleting bit 2 is

$$P_2 = \{0000, 1110, 0011, 0101, 1101, 1011, 0110, 1000\}.$$

The projection obtained from C by deleting bit 4 is

$$P_4 = \{00000, 1010, 0101, 0011, 1111, 1001, 0110, 1100\}.$$

C has $\delta = 2$ but P_2 has $\delta = 1$ while P_4 has $\delta = 2$.

Extension

Sometimes increasing the length of a code by one bit gives a new code with improved error detection or correction, which justifies the (slightly) lower rate.

5.7.5 Definition Let C be a linear code of length n . If $c \in C$, let c^* be the word of length $n + 1$ formed by adding one extra bit to c to make $\|c^*\|$ even. Let $C^* = \{c^* \mid c \in C\}$. This is a code of length $n + 1$, called the *extended code* of C .

Note that $c^* = (c \mid b)$ where $c \in C$, $b \in \mathbb{F}_2$ and $b \equiv \|c\| \pmod{2}$. So if $c = (c_1, \dots, c_n)$ then $b = c_1 + c_2 + \dots + c_n$.

5.7.6 Example Let $C = \mathbb{F}_2^3$ be the trivial code (all words of length 3). Then $C^* \subseteq \mathbb{F}_2^4$ is a *raid5* code, Example 1.1.2.

5.7.7 Lemma If C is linear so is C^* .

Proof Let $c_1, c_2 \in C$ with corresponding words $c_1^* = (c_1 \mid b_1)$, $c_2^* = (c_2 \mid b_2) \in C^*$. Then $b_1 \equiv \|c_1\| \pmod{2}$, $b_2 \equiv \|c_2\| \pmod{2}$ and $c_1^* + c_2^* = (c_1 + c_2 \mid b)$ where $b = b_1 + b_2 \equiv \|c_1\| + \|c_2\| \pmod{2} \equiv \|c_1 + c_2\| \pmod{2}$ by Lemma 2.2.5. This says $c_1^* + c_2^* = (c_1 + c_2)^* \in C^*$, so C^* is closed under addition. $\square$

5.7.8 Theorem If a linear code C has a $k \times n$ generating matrix $\mathbf{G}$ then the extended code C^* has a $k \times (n+1)$ generating matrix

$$\mathbf{G}^* = \begin{pmatrix} \mathbf{G} & \mathbf{b} \end{pmatrix},$$

where $\mathbf{b}$ is a $k \times 1$ column vector such that each row of $\mathbf{G}^*$ has even weight.

A parity check matrix for C^* can be constructed from $\mathbf{G}^*$ using Algorithm 4.6.13. However, the following theorem provides a quicker method to find a parity check matrix for C^* .

5.7.9 Theorem If $\mathbf{H}$ is a parity check matrix for C , then a parity check matrix for C^* is given by

$$\mathbf{H}^* = \begin{pmatrix} \mathbf{H} & \mathbf{1}^T \\ \mathbf{0} & 1 \end{pmatrix},$$

where $\mathbf{1}^T$ is a column vector containing all 1s, and $(\mathbf{0} \ 1)$ is a single extra row.

Proof We know $\mathbf{H}$ is an $n \times (n-k)$ matrix with (column) rank $(n-k)$. The last column in $\mathbf{H}^*$ is clearly independent from the first $(n-k)$, so $\mathbf{H}^*$ is an $(n+1) \times (n+1-k)$ matrix with (column) rank $(n-k+1)$. Moreover,

$$\mathbf{G}^* \mathbf{H}^* = \begin{pmatrix} \mathbf{G} & \mathbf{b} \end{pmatrix} \begin{pmatrix} \mathbf{H} & \mathbf{1}^T \\ \mathbf{0} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{G}\mathbf{H} & \mathbf{G}\mathbf{1}^T + \mathbf{b} \end{pmatrix}.$$

But $\mathbf{G}\mathbf{H} = \mathbf{0}$, and $\mathbf{G}\mathbf{1}^T$ sums the ones in each row of $\mathbf{G}$, so from the definition of $\mathbf{b}$ it follows that $\mathbf{G}\mathbf{1}^T + \mathbf{b} = \mathbf{0}$. $\square$

5.7.10 Example Let C be the linear code with generating matrix $\mathbf{G}$ and parity check matrix $\mathbf{H}$, where

$$\mathbf{G} = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix} \quad \text{and} \quad \mathbf{H} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Find $\mathbf{G}^*$ and $\mathbf{H}^*$ for the extended code C^* .

Solution:

$$\mathbf{G}^* = \left(\begin{array}{ccccc|c} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \end{array} \right), \quad \mathbf{H}^* = \left(\begin{array}{cc|c} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ \hline 0 & 0 & 1 \end{array} \right).$$

⊗

Let C be an (n, k, δ) linear code, and let C^* have parameters (n^*, k^*, δ^*) . Clearly $n^* = n+1$ and $k^* = k$. Thus, the rate r^* of C^* is $r^* = k^*/n^* = k/(n+1)$. Notice that C^* is a slightly less efficient code than C since

$$r = \frac{k}{n} > \frac{k}{n+1} = r^*.$$

Let $c \in C \setminus \{0\}$ have minimal weight, and let $c^* = (c \mid b) \in C^* \setminus \{0\}$ where $b \in \{0, 1\}$. Clearly $\delta^* \leq \|c^*\|$.

Now let $u^* = (u \mid v) \in C^* \setminus \{0\}$ be any word, where $u \in C$ and $v \in \{0, 1\}$. Note that $u \neq 0$ else $u^* = 0$.

If $\|u\| = \|c\|$ then $v = b$ so $\|u^*\| = \|c^*\|$. Otherwise by definition of c , $\|u\| > \|c\|$ so $\|u^*\| \geq \|u\| \geq \|c\| + 1 \geq \|c^*\|$. So every word in $C^* \setminus \{0\}$ has weight $\geq \|c^*\|$, so $\delta^* \geq \|c^*\|$ also. Thus

$$\delta^* = \|c^*\| = \begin{cases} \delta, & \text{if } \delta \text{ is even,} \\ \delta + 1, & \text{if } \delta \text{ is odd.} \end{cases}$$

Thus an extended code is only useful when the distance of the original code is odd, in which case the extended code corrects no more errors than the original code, but it detects one more error.

5.7.11 Theorem Let C be a linear (n, k, δ) -code in which δ is odd. Then there exists a linear $(n+1, k, \delta+1)$ -code.

Note that Theorems 5.7.3 and 5.7.11 give us the result that a linear (n, k, δ) -code with δ odd exists if and only if a linear $(n+1, k, \delta+1)$ -code exists.

Residual

Let C be a linear (n, k, δ) -code and let $v \in C$ be a codeword of weight δ . By considering equivalent codes, we can assume that v has 1 in the first δ positions, and 0 in the remaining $n - \delta$ positions. Choose a generating matrix G for code C , such that G has v as its first row. Delete the first δ columns of G as well as the row of zeros that results, leaving a $(k-1) \times (n-\delta)$ matrix. The residual code R of C is the code with this matrix as its generating matrix.

5.7.12 Theorem Let C be a linear (n, k, δ) -code. The residual code is a linear $(n - \delta, k - 1, \lceil \frac{\delta}{2} \rceil)$ -code.

$(a|a+b)$ Construction

Let C_1 be a linear (n, k_1, δ_1) -code and let C_2 be a linear (n, k_2, δ_2) -code. For each $c_1 \in C_1$ and $c_2 \in C_2$, we construct a word of length $2n$ whose first n bits are the word c_1 and whose last n bits are the word $c_1 + c_2$. These words of length $2n$ are a new linear code and we refer to this construction the $(a|a+b)$ construction. Note that $a \in C_1$ and $b \in C_2$.

5.7.13 Example Let C_1 be the linear $(4, 2, 1)$ -code, $C_1 = \{0000, 1000, 0111, 1111\}$. Let C_2 be the linear $(4, 2, 2)$ -code, $C_2 = \{0000, 1100, 0011, 1111\}$.

The $(a|a+b)$ construction with $a \in C_1$ and $b \in C_2$ gives codewords:

0000 0000	1000 1000	0111 0111	1111 1111
0000 1100	1000 0100	0111 1011	1111 0011
0000 0011	1000 1011	0111 0100	1111 1100
0000 1111	1000 0111	0111 1000	1111 0000

This code has $n = 8$, $k = 4$ and $\delta = 2$.

The $(a|a+b)$ construction with $a \in C_2$ and $b \in C_1$ gives codewords:

0000 0000	1100 1100	0011 0011	1111 1111
0000 1000	1100 0100	0011 1011	1111 0111
0000 0111	1100 1011	0011 0100	1111 1000
0000 1111	1100 0011	0011 1100	1111 0000

This code has $n = 8$, $k = 4$ and $\delta = 1$.

5.7.14 Theorem Let C_1 be a linear (n, k_1, δ_1) -code and let C_2 be a linear (n, k_2, δ_2) -code. Then there exists a linear $(2n, k_1 + k_2, d)$ -code where $d = \min\{2\delta_1, \delta_2\}$.

Let C_1 be a linear (n, k_1, δ_1) -code with generating matrix $\mathbf{G}_{C_1}$ and parity check matrix $\mathbf{H}_{C_1}$, let C_2 be a linear (n, k_2, δ_2) -code with generating matrix $\mathbf{G}_{C_2}$ and parity check matrix $\mathbf{H}_{C_2}$, and let C be the code constructed from C_1 and C_2 using the $(a|a+b)$ construction with $a \in C_1$ and $b \in C_2$. Then $\mathbf{G}_C$ and $\mathbf{H}_C$ are a generating matrix and parity check matrix, respectively, for C , where

$$\mathbf{G}_C = \begin{pmatrix} \mathbf{G}_{C_1} & \mathbf{G}_{C_1} \\ \mathbf{0} & \mathbf{G}_{C_2} \end{pmatrix} \quad \mathbf{H}_C = \begin{pmatrix} \mathbf{H}_{C_1} & \mathbf{H}_{C_2} \\ \mathbf{0} & \mathbf{H}_{C_2} \end{pmatrix}.$$

CHAPTER 6

Linear Codes II

We have bounds on what is possible to achieve with linear codes, and some construction techniques for constructing good linear codes (those whose parameters are close to the bounds). In this chapter we investigate some important families of linear codes.

6.1. Reed-Muller Codes

Mariner 9 was successfully launched on 30 May, 1971, and became the first artificial satellite of Mars when it arrived on 13 November, 1971 and went into orbit. It transmitted pictures back to Earth for nearly a year and completed its final transmission on 27 October, 1972.

Upon arrival, Mariner 9 observed that a great dust storm was obscuring the whole globe of the planet. Ground controllers sent commands to the spacecraft to wait until the storm had abated, the dust had settled, and the surface was clearly visible before compiling its global mosaic of high-quality images of the Martian surface. The storm persisted for a month, but after the dust cleared, Mariner 9 proceeded to reveal a very different planet than expected – one that boasted gigantic volcanoes and a grand canyon stretching 4,800 kilometers across its surface. More surprisingly, the relics of ancient riverbeds were carved in the landscape of this seemingly dry and dusty planet. Mariner 9 exceeded all primary photographic requirements by photo-mapping 100 percent of the planet's surface. The spacecraft also provided the first close-up pictures of the two small, irregular Martian moons: Phobos and Deimos.

Mariner 9 had a radio transmitter with a power of only 20 watts, and was transmitting over a distance of 84 million miles. Despite this, near-perfect pictures were obtained. Each picture transmitted by Mariner 9 is made up of over half a million tiny picture elements forming a rectangular array. Each picture element is a uniform shade of grey, the precise shade being specified by a 9-bit binary number. Thus each picture was represented using 5,250,000 bits, in grey-scale. Given the large distances, low power, poor reliability of electronics in the transmitter and receiver, and the general background noise of space, many errors occurred in transmission. Hence it was necessary to perform extensive error correction. Each message was divided into packets of 6 bits, and then each string of 6 bits was encoded into a 32 bit word. Thus a picture represented by 5 and a quarter million bits, was transmitted using over 5 times that number of bits.

The code used on the Mariner 9 was a *first-order Reed-Muller Code of length 32*. These are linear codes that were introduced in the 1950s by I.S. Reed and D.E. Muller. Rather than maintaining a fixed distance (and hence error correction capacity, like the Hamming codes that we encounter in the next section), Reed-Muller codes give an increased distance as the length of codewords increases. In this section we describe the first-order Reed-Muller codes and their properties.

6.1.1 Example The first-order Reed-Muller code of length 4 has the following generating matrix.

$$G = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

It has $2^3 = 8$ codewords, namely

$$\{0000, 0101, 1010, 0011, 1100, 0110, 1001, 1111\},$$

and it has distance 2 (easily seen from the minimum weight of the non-zero codewords). The code consists of all even-weight words of length 4.

6.1.2 Example The first-order Reed-Muller code of length 8 has the following generating matrix.

$$G = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

It has $2^4 = 16$ codewords, given by multiplying G by each of the 2^4 binary words of length 4. This code has one word of weight 0, one word of weight 8, and all the other codewords have weight 4 (but it does not contain all the weight 4 binary words of length 8). It has distance 4.

Recall that the *binary representation* of a non-negative integer x is the binary sequence $x_t x_{t-1} \dots x_2 x_1 x_0$ such that

$$x = x_t \cdot 2^t + x_{t-1} \cdot 2^{t-1} + \cdots + x_2 \cdot 2^2 + x_1 \cdot 2^1 + x_0 \cdot 2^0.$$

For example, the binary representation of 1543 is 11000000111. Notice that the generating matrix G for the Reed-Muller code of length 8 has the binary representations of the integers from 8 to 15 as its columns. If we ignore the top row, we have the binary representations of the integers from 0 to 7.

6.1.3 Definition The first-order Reed-Muller code of length 2^m , denoted $RM(m)$, is the linear code given by the generating matrix G_m whose columns are the binary representations of the integers from 2^m to $2^{m+1} - 1$.

The codes $RM(2)$ and $RM(3)$ are given in Examples 6.1.1 and 6.1.2 respectively.

6.1.4 Example The first-order Reed-Muller codes $RM(4)$ and $RM(5)$ have generating matrices

$$G_4 = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

and

[illegible]

6.1.5 Theorem $RM(m)$ is a $(2^m, m+1, 2^{m-1})$ -code.

Proof Since G_m has $(2^{m+1} - 1) - (2^m - 1) = 2^m$ columns, the code $RM(m)$ has length 2^m . The number of bits in the binary representation of $2^{m+1} - 1$ is $m + 1$, so G_m has $m + 1$ rows, which are easily seen to be linearly independent. Thus, $RM(m)$ has dimension $m + 1$. Clearly, $RM(m)$ contains $\mathbf{0}_{2^m}$ and $\mathbf{1}_{2^m}$, and it can be shown that every other codeword in $RM(m)$ has weight 2^{m-1} . Thus, $RM(m)$ has distance 2^{m-1} . $\square$

The code used by the Mariner 9 was $RM(5)$, which has 64 codewords, each of length 32. Having distance 16, it can correct any received word containing 7 or fewer errors.

6.2. Hamming Codes

In this section we construct a perfect single error correcting linear code of length $2^r - 1$ for all $r \geq 2$. By Theorem 5.5.5, this is every length for which a perfect single error correcting code exists.

Suppose C is a perfect single error correcting linear code, and let H be a parity check matrix for C . Since H has rank $n - k$, and since any word of length n is a possible received word, every word of length $n - k$ can occur as a syndrome. There must be a unique error pattern of weight ≤ 1 for each such syndrome. But the syndrome of a weight 1 word is a row of H , and so the rows of H must include every word of length $n - k$. This motivates the following definition.

6.2.1 Definition For $r \geq 2$ let $H = H_r$ denote a $(2^r - 1) \times r$ matrix whose rows consist of all non-zero vectors of length r , each occurring exactly once. The corresponding linear code is called a *Hamming code* of length $2^r - 1$.

The matrix H_r must contain each word of weight one as a row and hence must have r linearly independent columns. Hence H_r is a parity check matrix for some linear code and by definition this code is a Hamming code.

6.2.2 Example One possibility for a parity check matrix H for a Hamming code of length 7 (so $r = 3$) is:

$$H = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

By Algorithm 4.6.13, a generating matrix G for a Hamming code of length 7 is therefore

$$(6.2.1) \quad G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

Thus the code has dimension 4 and contains $2^4 = 16$ codewords. The distance of the code is $\delta = 3$. The information rate is $4/7$. This is the code examined in Example 4.11.12.

In general, H_r is an $n \times (n - k) = (2^r - 1) \times r$ matrix, so $n = 2^r - 1$ and $k = n - r = 2^r - 1 - r$. No row of H_r is the zero word, and no two rows of H_r are equal, so no set of ≤ 2 rows of H_r is linearly dependent. But 3 rows of H_r can be linearly dependent: $100 \dots 0$, $0100 \dots 0$, $1100 \dots 0$. Thus (Theorem 4.9.1) every Hamming code has distance $\delta = 3$.

Thus, for all $r \geq 2$ the Hamming code with parity check matrix $H = H_r$ is a $(2^r - 1, 2^r - 1 - r, 3)$ linear code.

r	$n = 2^r - 1$	k	$ C = 2^k$
2	3	1	2
3	7	4	16
4	15	11	2048
5	31	26	67108864
6	63	57	$144115188075855872 \simeq 1.4 \times 10^{17}$

For $n = 2^r - 1$ and $\delta = 3 = 2t + 1$ (so $t = 1$) the Hamming bound is

$$\frac{2^n}{\binom{n}{0} + \cdots + \binom{n}{t}} = \frac{2^n}{\binom{n}{0} + \binom{n}{1}} = \frac{2^n}{1 + n} = \frac{2^{2^r-1}}{1 + 2^r - 1} = 2^{2^r-1-r} = 2^k = |C|.$$

Theorem 5.5.2 implies Hamming codes are perfect, single error correcting codes.

It is trivial to construct an SDA for a Hamming code. All error patterns of weight 1 are corrected, so every word of length $2^r - 1$ and weight one must be a coset leader. If e is an error pattern then eH sums the rows of the parity check matrix H corresponding to positions in which errors occurred. Hence, since H has $2^r - 1$ rows, an SDA for a Hamming code must be given by:

coset leader	syndrome
000...0	000...0
I_{2^r-1}	H

Since Hamming codes have distance 3 (odd), we can gain some extra error detection capability by forming the extended Hamming code. The *extended Hamming code of length 2^r* is a $(2^r, 2^r - 1 - r, 4)$ linear code. Thus, an extended Hamming code is 1-error correcting and 3-error detecting code.

6.2.3 Definition The dual of the Hamming code of length $2^r - 1$ is called the *simplex* code of length $2^r - 1$.

6.2.4 Example The simplex code of length 7.

Recall that $G_{C^\perp} = H_C^\top$ (see 4.7.1). So we take $G = H^\top$, where H contains each possible non-zero row:

$$G = G_{C^\perp} = (H_C)^\top = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

This has each possible non-zero vector of length 3 in exactly one column.

6.3. The Extended Golay Code

Hamming codes are perfect $(2^r - 1, 2^r - 1 - r, 3)$ -codes. By Theorem 5.5.5, any other non-trivial perfect code is a $(23, 12, 7)$ -code. A $(23, 12, 7)$ -code exists and is called the *Golay code*. We shall construct this code, and its extended code C^* . Actually we construct C^* first, and then later obtain C by dropping the last bit from C^* .

The extended Golay code was used by the Voyager space probes. The Voyager space mission (Voyager 1 was launched on 1 September 1977 and Voyager 2 was launched on 20 August 1977) successfully explored the planets Jupiter, Saturn, Uranus and Neptune and the two spacecraft continue their exploration as they move towards interstellar space. The Galileo spacecraft was launched on 18 October 1989, entered into orbit around Jupiter in 1995, and conducted a thorough exploration of Jupiter and its moons before making a mission-ending plunge into the planet in

2003. To send data back to Earth, each of these spacecraft used a pair of codes, one of which was the Extended Golay Code (a 3-error correcting code).

6.3.1 Definition Let B be the 12×12 matrix:

$$B = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \end{pmatrix}.$$

and let G be the 12×24 matrix $G = (I_{12} \ B)$. The linear code with generating matrix G is the *extended Golay code*, which we will denote C_{24} .

The matrix B can be constructed as follows. We first construct an 11×11 matrix B' , and then obtain B from B' by adding $(\mathbf{1}_{11} | 0)$ as row 12 and $(\mathbf{1}_{11} | 0)^T$ as column 12. For the first row of B' , we place a 1 in columns 1, 2, 4, 5, 6 and 10, and a zero in the remaining columns 3, 7, 8, 9, 11. Each subsequent row is obtained from the row above by shifting each entry down and to the left, except the entry in column 1 which gets shifted down and into column 11. Thus, for $1 \leq i \leq 11$ and $1 \leq j \leq 11$ we have $B'(i+1, j) = B'(i, j+1)$, where the row and column indices are calculated modulo 11 on the residues $1, 2, \dots, 11$. In other words, each row is a left cyclic shift of the row above.

If we were to index the rows and columns of B' from 0 to 10 instead of from 1 to 11, then the 0th (top) row would have 1s in column 0 and columns 1, 3, 4, 5, 9 and 0s in columns 2, 6, 7, 8, 10. Thus, the indices of columns containing 1 are 0 and the quadratic residues modulo 11, and the indices of the columns containing 0 are the quadratic non-residues modulo 11.

6.3.2 Lemma The matrix B has the following properties.

- (α) $B^T = B$
- (β) $B^2 = I_{12}$ (over $\mathbb{F}_2$)
- (γ) Every row of B has weight 7, except the bottom row which has weight 11.
- (δ) The distance between any 2 distinct rows of B is 6.
- (ϵ) Each row and column of B has odd weight.

Proof Each of properties $\alpha, \beta, \gamma, \delta, \epsilon$ can be checked by direct observation and calculation. Properties α and γ are easily verified by noting the way the rows of B' are constructed via cyclic shifting. Property ϵ follows immediately from α and γ .

For properties β and δ , we observe that the positions in which the 1s occur in rows 1 to 11 of $\mathbf{B}$ are as follows.

row 1 :	1	2	4	5	6	10	12
row 2 :	2	3	5	6	7	11	12
row 3 :	3	4	6	7	8	1	12
row 4 :	4	5	7	8	9	2	12
row 5 :	5	6	8	9	10	3	12
row 6 :	6	7	9	10	11	4	12
row 7 :	7	8	10	11	1	5	12
row 8 :	8	9	11	1	2	6	12
row 9 :	9	10	1	2	3	7	12
row 10 :	10	2	1	3	4	8	12
row 11 :	11	3	2	4	5	9	12

For $1 \leq i < i' \leq 11$, if we look at the positions of the 1s in rows i and i' , then we see that there are exactly 4 positions in common. For example, rows 3 and 5 both have 1s in positions 3, 6, 8 and 12, and nowhere else.

Thus, for $1 \leq i < i' \leq 11$, rows i and i' of $\mathbf{B}$ are at distance 6 (because the weight of each of these rows is 7) and are orthogonal (because 4 is even). For $1 \leq i \leq 11$, row i and row 12 are also at distance 6, so δ holds, and row i and row 12 of $\mathbf{B}$ are orthogonal (because row i has exactly six 1s in columns 1 to 11). Finally, since each row of $\mathbf{B}$ has odd weight, the dot product of any row with itself is 1. Thus, since $\mathbf{B}^T = \mathbf{B}$, we have $\mathbf{B}^2 = \mathbf{I}_{12}$. So β holds. $\square$

6.3.3 Theorem [Properties of C_{24}] The code C_{24} has the following properties:

- (a) C_{24} has length 24 and dimension 12.
- (b) C_{24} has parity check matrix $\mathbf{H} = \mathbf{G}^T = \begin{pmatrix} \mathbf{I}_{12} \\ \mathbf{B} \end{pmatrix}$.
- (c) Another parity check matrix for C_{24} is $\mathbf{H}' = \begin{pmatrix} \mathbf{B} \\ \mathbf{I}_{12} \end{pmatrix}$.
- (d) Another generating matrix for C_{24} is $\mathbf{G}' = \begin{pmatrix} \mathbf{B} & \mathbf{I}_{12} \end{pmatrix}$.
- (e) Any two rows (distinct or not) of $\mathbf{G}$ are orthogonal.
- (f) C_{24} is self-dual: $(C_{24})^\perp = C_{24}$.
- (g) The distance of C_{24} is 8, so C_{24} is a 3-error correcting code.
- (h) C_{24} has one word of weight 0, namely $\mathbf{0}_{24}$, one word of weight 24, namely $\mathbf{1}_{24}$, and every other word has weight 8, 12, or 16 (in fact, the weight distribution for the codewords of C_{24} is

weight	0	4	8	12	16	20	24
# words	1	0	759	2576	759	0	1

but we don't need this and won't be proving it).

Proof (a) is clear. (b) $\mathbf{H}$ is a 24×12 matrix with linearly independent columns, and since $\mathbf{B}^2 = \mathbf{I}_{12}$ (see Lemma 6.3.2 (β)), we have

$$\mathbf{G}\mathbf{H} = \begin{pmatrix} \mathbf{I}_{12} & \mathbf{B} \end{pmatrix} \begin{pmatrix} \mathbf{I}_{12} \\ \mathbf{B} \end{pmatrix} = \mathbf{I}_{12}^2 + \mathbf{B}^2 = \mathbf{I}_{12} + \mathbf{I}_{12} = \mathbf{0}.$$

- (c) $\mathbf{H}'$ is a 24×12 matrix with linearly independent columns, and

$$\mathbf{G}\mathbf{H}' = \begin{pmatrix} \mathbf{I}_{12} & \mathbf{B} \end{pmatrix} \begin{pmatrix} \mathbf{B} \\ \mathbf{I}_{12} \end{pmatrix} = \mathbf{B} + \mathbf{B} = \mathbf{0}.$$

(d) $\mathbf{G}'$ has the correct size, its rows are linearly independent, and

$$\mathbf{G}'\mathbf{H} = \begin{pmatrix} \mathbf{B} & \mathbf{I}_{12} \end{pmatrix} \begin{pmatrix} \mathbf{I}_{12} \\ \mathbf{B} \end{pmatrix} = \mathbf{B} + \mathbf{B} = \mathbf{0}.$$

(e) $\mathbf{G}$ is a generating matrix and $\mathbf{G}^\top = \mathbf{H}'$ is a parity check matrix, so $\mathbf{G}\mathbf{G}^\top = \mathbf{G}\mathbf{H}' = \mathbf{0}$ (as in (c)). So every row of $\mathbf{G}$ is orthogonal to every column of $\mathbf{G}^\top$.

(f) Since $\mathbf{G}_{C^\perp} = \mathbf{H}_C^\top$, a generating matrix for $(C_{24})^\perp$ is $(\mathbf{H})^\top = (\mathbf{I}_{12}^\top \ \mathbf{B}^\top) = (\mathbf{I}_{12} \ \mathbf{B}) = \mathbf{G}$ (see Lemma 6.3.2 (α)), so C_{24} and $(C_{24})^\perp$ have equal generating matrices.

(g) There are two steps:

(1) The distance of C_{24} is 6, 7 or 8.

Proof By Lemma 6.3.2 (γ), there are rows of $\mathbf{G}$ with weight 8, so $d(C_{24}) \leq 8$.

Suppose there exists a codeword c with $0 < \|c\| \leq 5$. Using our two generating matrices,

$$c = u_1 (\mathbf{I}_{12} \ \mathbf{B}) = u_2 (\mathbf{B} \ \mathbf{I}_{12})$$

for some u_1 and $u_2 \in \mathbb{F}_2^{12}$. Write $c = (c_1 \mid c_2)$ where c_1 and c_2 have length 12. Since $\|c\| \leq 5$, at least one of c_1, c_2 has weight ≤ 2 . But $c_1 = u_1$ and $c_2 = u_2$. So one of u_1 or u_2 has weight ≤ 2 . Thus c is the sum of either 1 or 2 rows of a generating matrix. However, by Lemma 6.3.2 (γ) and (δ), any such c must satisfy $\|c\| \geq 6$, contradiction.

(2) If $w \in C_{24}$, then $\|w\|$ is a multiple of 4.

Proof Let r_i denote row i of $\mathbf{G}$. Any codeword w can be expressed as the sum of at most 12 rows of $\mathbf{G}$, so $w = r_{i_1} + \cdots + r_{i_m}$, where $m \leq 12$. Perform induction on m :

If $m = 0$ the result is trivial. If $m = 1$ then $\|w\| = \|r_{i_1}\| = 8$ or 12 (see Lemma 6.3.2 (γ)).

If $m \geq 1$, assume the hypothesis is true for m and consider $m + 1$.

We know $w = (r_{i_1} + \cdots + r_{i_m}) + r_{i_{m+1}} = c + r$, where $r = r_{i_{m+1}}$ and c is a codeword which is a sum of m rows of $\mathbf{G}$. By inductive hypothesis $\|c\|$ is a multiple of 4, and so is $\|r\|$ (see Lemma 6.3.2 (γ)). Also since the rows of $\mathbf{G}$ are mutually orthogonal (by (e) above), $c \perp r$ (see Lemma 4.6.4). Thus, by Lemma 2.2.5 $\|w\| = \|c + r\| \equiv \|c\| + \|r\| \equiv 0 + 0 \pmod{4}$. So, by induction, (2) is true.

It follows from (1) and (2) that $d(C_{24}) = 8$.

(h) By Lemma 6.3.2 (ε), the sum of all rows of $\mathbf{G}$ is $\mathbf{1}$, so $\mathbf{1} \in C_{24}$. Thus, if $c \in C_{24}$, then $c + \mathbf{1} \in C_{24}$. Hence the number of words of weight r equals the number of words of weight $24 - r$. Clearly, there is 1 word of weight 0 and 1 word of weight 24. There are no words of weight 4 (as $\delta = 8$), so there are no words of weight 20. Thus, all words have weight 0, 8, 12, 16 or 24 (recall that we have already seen that the weight of any word in C_{24} is a multiple of 4). $\square$

6.4. Decoding the Extended Golay Code

In this section we give an algorithm for IMLD for C_{24} . We will use the following notation throughout.

w	Received word
c	Closest codeword to w
e	Error pattern ($e = c + w$)
$e = (e_1 \mid e_2)$	e_1, e_2 of length 12
θ_j	The word of length 12 with a 1 in the j th position and zeros elsewhere
b_j	The j th row of matrix $\mathbf{B}$
$\mathbf{G} = (\mathbf{I}_{12} \mid \mathbf{B})$	The generating matrix for C_{24}
$\mathbf{H} = \begin{pmatrix} \mathbf{I}_{12} \\ \mathbf{B} \end{pmatrix}$ and $\mathbf{H}' = \begin{pmatrix} \mathbf{B} \\ \mathbf{I}_{12} \end{pmatrix}$	Parity check matrices for C_{24}
s	The syndrome $w\mathbf{H}$
s'	The syndrome $w\mathbf{H}'$

Before describing the decoding algorithm, we present a few results.

6.4.1 Lemma The syndrome $s = e_1 + e_2\mathbf{B}$ and the syndrome $s' = s\mathbf{B} = e_1\mathbf{B} + e_2$.

Proof We have

$$s = w\mathbf{H} = e\mathbf{H} = e_1\mathbf{I}_{12} + e_2\mathbf{B} \quad \therefore s = e_1 + e_2\mathbf{B}.$$

Similarly,

$$s' = w\mathbf{H}' = e\mathbf{H}' = e_1\mathbf{B} + e_2\mathbf{I}_{12} \quad \therefore s' = e_1\mathbf{B} + e_2$$

and so

$$\begin{aligned} s' &= e_1\mathbf{B} + e_2\mathbf{I}_{12} \\ &= e_1\mathbf{B} + e_2\mathbf{B}^2 \\ &= (e_1 + e_2\mathbf{B})\mathbf{B} \\ &= s\mathbf{B}. \end{aligned}$$

□

6.4.2 Lemma If u is a non-zero word of length 12, then $\|u\mathbf{B}\| \geq 8 - \|u\|$.

Proof Since C_{24} has distance 8, we have $\|u\mathbf{G}\| \geq 8$. But $\mathbf{G} = (\mathbf{I}_{12} \mid \mathbf{B})$, so

$$8 \leq \|u\mathbf{G}\| = \|u(\mathbf{I}_{12} \mid \mathbf{B})\| = \|(u \mid u\mathbf{B})\| = \|u\| + \|u\mathbf{B}\|.$$

Thus, $\|u\mathbf{B}\| \geq 8 - \|u\|$.

□

6.4.3 Lemma If $\|e\| \leq 3$, then the following are equivalent.

- (a) $e_2 = \mathbf{0}$.
- (b) $e = (s \mid \mathbf{0})$.
- (c) $\|s\| \leq 3$.

Proof We show that (a) implies (b), (b) implies (c), and (c) implies (a).

(a) $\implies$ (b) If $e_2 = \mathbf{0}$, then $s = e_1 + e_2\mathbf{B} = e_1$ and so $e = (s \mid \mathbf{0})$.

(b) $\implies$ (c) If $e = (s \mid \mathbf{0})$, then $\|s\| \leq 3$ (because $\|e\| \leq 3$).

(c) $\implies$ (a) Suppose $\|s\| \leq 3$. Then

$$\|e_2\mathbf{B}\| = \|e_1 + e_1 + e_2\mathbf{B}\| \leq \|e_1\| + \|e_1 + e_2\mathbf{B}\| = \|e_1\| + \|s\| \leq \|e_1\| + 3.$$

But $\|e_1\| + \|e_2\| \leq 3$ so we have $\|e_2\mathbf{B}\| \leq 6 - \|e_2\|$ which contradicts Lemma 6.4.2 unless $e_2 = \mathbf{0}$.

Thus, we have shown $\|s\| \leq 3$ implies $e_2 = \mathbf{0}$, as required.

□

6.4.4 Lemma Let $j \in \{1, 2, \dots, 12\}$. If $\|e\| \leq 3$, then the following are equivalent.

- (a) $e_2 = \theta_j$.
- (b) $e = (s + b_j \mid \theta_j)$.
- (c) $\|s + b_j\| \leq 2$.

Proof We show that (a) implies (b), (b) implies (c), and (c) implies (a).

(a) $\implies$ (b) If $e_2 = \theta_j$, then $s = e_1 + \theta_j \mathbf{B} = e_1 + b_j$. Thus, $e = (e_1 \mid e_2) = (s + b_j \mid \theta_j)$.

(b) $\implies$ (c) If $e = (s + b_j \mid \theta_j)$, then $\|s + b_j\| \leq 2$ (because $\|\theta_j\| = 1$ and $\|e\| \leq 3$).

(c) $\implies$ (a) Suppose $\|s + b_j\| \leq 2$. For a contradiction, assume that $e_2 \neq \theta_j$. Then by Lemma 6.4.2 we have

$$\begin{aligned}
 8 - \|e_2 + \theta_j\| &\leq \|(e_2 + \theta_j)\mathbf{B}\| \\
 &= \|e_2\mathbf{B} + \theta_j\mathbf{B}\| \\
 &= \|e_2\mathbf{B} + b_j\| \\
 &= \|e_1 + e_2\mathbf{B} + b_j + e_1\| \\
 &\leq \|e_1 + e_2\mathbf{B} + b_j\| + \|e_1\| \\
 &= \|s + b_j\| + \|e_1\| \\
 &\leq 2 + \|e_1\|.
 \end{aligned}$$

Thus, $6 \leq \|e_1\| + \|e_2 + \theta_j\| \leq \|e_1\| + \|e_2\| + \|\theta_j\| \leq 4$, a contradiction. We conclude that $e_2 = \theta_j$, as required. $\square$

6.4.5 Lemma If $\|e\| \leq 3$, then the following are equivalent.

- (a) $e_1 = \mathbf{0}$.
- (b) $e = (\mathbf{0} \mid s')$.
- (c) $\|s'\| \leq 3$.

Proof This can be proved using similar arguments to those used in the proof of Lemma 6.4.3. $\square$

6.4.6 Lemma Let $j \in \{1, 2, \dots, 12\}$. If $\|e\| \leq 3$, then following are equivalent.

- (a) $e_1 = \theta_j$.
- (b) $e = (\theta_j \mid s' + b_j)$.
- (c) $\|s' + b_j\| \leq 2$.

Proof This can be proved using similar arguments to those used in the proof of Lemma 6.4.4. $\square$

We are now ready to present the algorithm for IMLD for the extended Golay code C_{24} .

6.4.7 Algorithm IMLD for the Extended Golay Code, C_{24}

- (a) Compute the syndrome $s = w\mathbf{H} = w \begin{pmatrix} \mathbf{I}_{12} \\ \mathbf{B} \end{pmatrix}$.
- (b) If $\|s\| \leq 3$ then $e = (s \mid 0)$. Stop.
- (c) If $\|s + b_j\| \leq 2$ for some row b_j of $\mathbf{B}$ then $e = (s + b_j \mid \theta_j)$. Stop.
- (d) Compute the second syndrome $s' = s\mathbf{B}$.
- (e) If $\|s'\| \leq 3$ then $e = (0 \mid s')$. Stop.
- (f) If $\|s' + b_j\| \leq 2$ for some row b_j of $\mathbf{B}$ then $e = (\theta_j \mid s' + b_j)$. Stop.
- (g) If e is not yet determined, then more than 3 errors have occurred so request retransmission.

Of course, once e is determined, w is decoded to $w + e$, and the message is the first 12 bits of $(w + e)$.

We now explain why this algorithm works when $\|e\| \leq 3$. Suppose $\|e\| \leq 3$. Step (a) involves calculating the syndrome $s = wH$.

If $e_2 = \mathbf{0}$, then by Lemma 6.4.3 we have $\|s\| \leq 3$ and $e = (s \mid \mathbf{0})$, and the correct evaluation of e is determined at step (b) of the algorithm. If $e_2 = \theta_j$ for some j , then by Lemma 6.4.4 we have $e = (s + b_j \mid \theta_j)$ and $\|s + b_j\| \leq 2$, and the correct evaluation of e is determined at step (c) of the algorithm. Note that the algorithm does indeed proceed past step (b) to step (c) when $e_2 = \theta_j$. This is because $e_2 = \theta_j$ implies $e_2 \neq \mathbf{0}$ and so by Lemma 6.4.3 we have $\|s\| > 3$. Thus, step (b) is bypassed when $e_2 = \theta_j$.

If $\|e_2\| > 1$, then by Lemmas 6.4.3 and 6.4.4 we have $\|s\| > 3$ and $\|s + b_j\| > 2$ for all j . So the algorithm proceeds to step (d) where the syndrome $s' = wH'$ is calculated via $s' = sB$ (see Lemma 6.4.1).

Now, with $\|e_2\| > 1$ we have $\|e_1\| \leq 1$. Thus, by similar reasoning to that of the preceding paragraph, if $e_1 = \mathbf{0}$ or $e_1 = \theta_j$ for some j , then the correct evaluation of e is determined at step (e) or step (f) respectively (by Lemmas 6.4.5 and 6.4.6).

Finally, when $\|e\| \leq 3$ it must be the case that at least one of e_1 and e_2 has weight at most 1. That is, we have one of

- (i) $e_2 = \mathbf{0}$;
- (ii) $e_2 = \theta_j$ for some j ;
- (iii) $e_1 = \mathbf{0}$;
- (iv) $e_1 = \theta_j$ for some j .

By Lemmas 6.4.3-6.4.6, this means that the algorithm terminates are one of steps (b), (c), (e) or (f). Thus, if the algorithm proceeds to step (g), then it is indeed the case that more than 3 errors have occurred.

6.4.8 Example Decode the received word $w = 101\ 111\ 101\ 111\ 010\ 010\ 010\ 010$.

Solution: The syndrome is

$$s = wH = 100\ 000\ 000\ 001$$

which has weight 2. Since $\|s\| \leq 3$, we find that $e = (s \mid \mathbf{0}) = (100\ 000\ 000\ 001 \mid \mathbf{0})$, and conclude that $c = w + e = 001\ 111\ 101\ 110\ 010\ 010\ 010\ 010$ was the codeword sent.

Because $G = \begin{pmatrix} I_{12} & B \end{pmatrix}$ is in standard form, the actual message sent is the first 12 bits of the corresponding codeword c . Hence 001 111 101 110 was the message sent. ⊛

6.4.9 Example Decode the received word $w = 001\ 001\ 001\ 101\ 101\ 000\ 101\ 000$.

Solution: The syndrome is

$$s = wH = 110\ 001\ 001\ 001$$

which has weight 5. Proceeding to Step (c) of Algorithm 6.4.7, we compute

$$\begin{aligned}
110\ 001\ 001\ 001 &+ 110\ 111\ 000\ 101 = s + b_1 = 000\ 110\ 001\ 100 \\
110\ 001\ 001\ 001 &+ 101\ 110\ 001\ 011 = s + b_2 = 011\ 111\ 000\ 010 \\
110\ 001\ 001\ 001 &+ 011\ 100\ 010\ 111 = s + b_3 = 101\ 101\ 011\ 110 \\
110\ 001\ 001\ 001 &+ 111\ 000\ 101\ 101 = s + b_4 = 001\ 001\ 100\ 100 \\
110\ 001\ 001\ 001 &+ 110\ 001\ 011\ 011 = s + b_5 = 000\ 000\ 010\ 010
\end{aligned}$$

Then, since $\|s + b_5\| \leq 2$, we have that

$$e = (s + b_5 \mid \theta_5) = (000\ 000\ 010\ 010 \mid 000\ 010\ 000\ 000)$$

and conclude that $c = w + e = 001\ 001\ 011\ 111\ 101\ 010\ 101\ 000$ was the codeword sent, so $001\ 001\ 011\ 111$ was the message sent. $\otimes$

6.5. The Golay Code

6.5.1 Definition The *Golay Code* C_{23} is obtained by deleting the last bit from each codeword of C_{24} .

We saw that every word in C_{24} has even weight. It follows that C_{24} is the extended code of C_{23} .

6.5.2 Theorem C_{23} has the following properties:

- (a) C_{23} is a linear code. A generating matrix is $(I_{12} \mid \widehat{B})$, where $\widehat{B}$ is obtained from the matrix B , as defined above, by deleting the last column.
- (b) C_{23} is a $(23, 12, 7)$ -code.
- (c) C_{23} is a perfect 3-error correcting code.

Proof (a) and (b) are clear, and (c) follows by Theorem 5.5.5. $\square$

Thus C_{23} will correct all error patterns of weight ≤ 3 and no other error patterns (Theorem 5.5.2).

The decoding algorithm for C_{23} is as follows.

6.5.3 Algorithm IMLD for the Golay Code C_{23}

- (a) Form $w^* = (w \mid 0)$ or $(w \mid 1)$ such that $\|w^*\|$ is odd.
- (b) Find the closest codeword $c^* \in C_{24}$ to w^* using Algorithm 6.4.7.
- (c) Remove the last bit from c^* to obtain $c \in C_{23}$.

We now explain why this algorithm works. We know that for any received word w , there exists a unique $c \in C_{23}$ such that $d(w, c) \leq 3$. We need to show that w is decoded to c . Let $c^* \in C_{24}$ be the word obtained by adding a bit to the end of c such that $\|c^*\|$ is even (we do not know what c^* is, but we know it exists). Since $d(w, c) \leq 3$, we have $d(w^*, c^*) \leq 4$. However, c^* has even weight (because $c^* \in C_{24}$) and w^* has odd weight (by definition). This means that $d(w^*, c^*)$ is odd and it follows that $d(w^*, c^*) \leq 3$. The decoding algorithm for C_{24} thus decodes w^* to c^* , and so deleting the last bit of c^* yields c .

The extended Golay code is a $(24, 12, 8)$ code, hence providing 3-error correction and 7-error detection with rate $12/24 = 0.5$. The Golay code is a $(23, 12, 7)$ code, hence providing 3-error correction and 6-error detection with rate $12/23 \simeq 0.52$.

We have now met all of the non-trivial perfect binary codes: the Hamming codes and the Golay code. There are no others.

CHAPTER 7

Cyclic Codes

Until now, we have assumed that errors occurring in transmission over a binary symmetric channel are randomly distributed. However, in some channels, it may be likely that errors occur very close to each other. For example, a compact disc may become scratched, or solar radiation may cause a group of errors which occur close together. In this chapter we consider codes that are designed to cope with this type of error pattern.

7.1. Introduction to burst errors

7.1.1 Definition A *burst* is a word that begins and ends with a 1 or is the empty word. A *burst error pattern* is a word of the form $\mathbf{0}_r b \mathbf{0}_s$ where $r, s \geq 0$ and b is a burst. (Note that if $r = 0$ there are no 0s on the left, and, if $s = 0$, none on the right.)

7.1.2 Definition Let $\mathfrak{B}_n(x)$ denote the set of all burst error patterns belonging to $\mathbb{F}_2^n$ with burst of length at most x , and let $\overline{\mathfrak{B}}_n(x)$ denote the set of all cyclic shifts of words in $\mathfrak{B}_n(x)$. The words in $\overline{\mathfrak{B}}_n(x)$ are called *cyclic burst error patterns*.

Note that the empty word $\{\}$ is a burst, so $\mathbf{0} \in \mathfrak{B}_n(x)$ always.

Also

$$(7.1.1) \quad \mathfrak{B}_n(x) \subseteq \overline{\mathfrak{B}}_n(x) \subseteq \{\text{all error patterns of weight } x\}.$$

7.1.3 Example Determine $\mathfrak{B}_5(3)$ and $\overline{\mathfrak{B}}_5(3)$.

Solution: The bursts of length at most 3 are $\{\}$, 1, 11, 111 and 101. Note that a word containing $\{\}$ must be $\mathbf{0}_r \{\} \mathbf{0}_s = \mathbf{0}$. Thus we have

$$\mathfrak{B}_5(3) = \{00000, 10000, 01000, 00100, 00010, 00001, 11000, 01100, 00110, 00011, 10100, 01010, 00101, 11100, 01110, 00111\}.$$

Most of the cyclic shifts of words in $\mathfrak{B}_5(3)$ are already in $\mathfrak{B}_5(3)$. For $\overline{\mathfrak{B}}_5(3)$ we add the missing shifts:

$$\overline{\mathfrak{B}}_5(3) = \mathfrak{B}_5(3) \cup \{10001, 01001, 10010, 11001, 10011\}.$$

⊗

7.1.4 Example Determine $|\overline{\mathfrak{B}}_{15}(3)|$.

Solution: The bursts of length at most 3 are $\{\}$, 1, 11, 111 and 101.

Burst b	# burst patterns containing b	
$\{\}$	1	$\mathbf{0}$
1	15	$r = 0, 1, \dots, 14$
11	14	$r = 0, 1, \dots, 13$
101	13	$r = 0, 1, \dots, 12$
111	13	$r = 0, 1, \dots, 12$

There are $1 + 15 + 14 + 13 + 13 = 56$ burst patterns. As in Example 7.1.3 there are another 5 shifts of these burst patterns not already counted, so $|\overline{\mathfrak{B}}_{15}(3)| = 61$. $\circledast$

7.1.5 Example Consider the words $c = 1101010$, $w_1 = 0100010$ and $w_2 = 0011010$. Assume c was transmitted. If w_1 was received the error pattern $e_1 = 1001000$ occurred, which has weight 2 and burst error length 4. If w_2 was received, the error pattern $e_2 = 1110000$ occurred, which has weight 3 and burst error length 3.

Thus if errors occur randomly, then w_1 is more likely than w_2 to be the received word. However if we assume errors occur in bursts, then w_2 may be more likely than w_1 to be the received word.

When applying IMLD to a linear code with the assumption of random errors, we constructed an SDA with coset leaders that were the words of smallest weight in the cosets, and said that such a code is t -error correcting precisely when all the words of weight at most t are in different cosets of the code. If errors are likely to occur in bursts, then we construct an SDA with coset leaders that are the error patterns with smallest burst length in each coset.

7.1.6 Definition A linear code C of length n is an x -burst error correcting code if all the words in $\mathfrak{B}_n(x)$ are in different cosets of C . It is an x -cyclic burst error correcting code if all the words in $\overline{\mathfrak{B}}_n(x)$ are in different cosets.

From (7.1.1), t -error correcting $\implies t$ -cyclic burst correcting $\implies t$ -burst correcting.

This is because a t -error correcting code must correct any error pattern of weight $\leq t$, including, in particular, all the bursts of length $\leq t$.

7.1.7 Example Show that there does not exist a 3-error correcting linear $(15, 9)$ -code, but that the same arguments do not rule out a 3-cyclic burst error correcting linear $(15, 9)$ -code.

Solution: In fact it is not even possible to have 3-error correcting $(19, 9)$ -code (linear or not). Such a code has $\delta = 7$, but the Hamming bound for a $(19, 9, 7)$ -code is

$$512 = 2^9 = |C| \leq \frac{2^{19}}{\binom{19}{0} + \dots + \binom{19}{3}} = \frac{2^{19}}{1160} \leq 452,$$

a contradiction. So there is no linear $(19, 9, 7)$ -code. The Hamming bound similarly rules out the existence of an $(n, 9, 7)$ -code for any $n \leq 19$.

However, the number of cosets of a $(15, 9)$ -code is $2^{15}/2^9 = 64$. By Example 7.1.4, $|\overline{\mathfrak{B}}_{15}(3)| = 61$. So a similar argument does not rule out the possibility that all the cyclic burst error patterns of length at most 3 could be in different cosets. $\circledast$

The previous example shows the reason for considering cyclic burst error correction. There are many fewer t cyclic burst error patterns than there are arbitrary weight t error patterns. Therefore, if we assume the errors only occur in bursts, we can correct higher weight error patterns.

How well can we do? Recall the Singleton bound 5.1.1 $\delta \leq n - k + 1$. We prove a similar result for burst errors.

7.1.8 Lemma If C is a t -burst error correcting code of length n and dimension k , then $t \leq n - k$.

Proof The code C must correct all errors whose non-zero entries occur in the first t places. There are 2^t of these, all of which must be in different cosets. But there are 2^{n-k} cosets in all. Hence $2^t \leq 2^{n-k}$ and so $t \leq n - k$. $\square$

Note: the result above looks very similar to the Singleton bound, that bound is for δ , not for the number of errors that can be corrected, $\lfloor \frac{\delta-1}{2} \rfloor$. With more work it can be shown that in a linear t -burst error correcting code we actually have $t \leq \frac{1}{2}(n - k)$, much as in the Singleton bound. This improved result is called the *Rieger bound*.

7.2. Words Viewed as Polynomials

Cyclic linear codes are a large family of codes that have a straightforward decoding algorithm for correcting cyclic burst error patterns. Examples include Reed-Solomon codes (used in CDs, DVDs, Blu-ray Discs, QR Codes, wireless communications).

In order to define cyclic linear codes, it is convenient to represent words (binary vectors) of length n as polynomials of degree at most $n - 1$ over $\mathbb{F}_2$.

Recall that the set of all polynomials¹ over $\mathbb{F}_2$ is denoted $\mathbb{F}_2[x]$.

7.2.1 Definition The polynomial $f(x) = \alpha_0 + \alpha_1 x + \cdots + \alpha_{n-1} x^{n-1}$ of degree at most $n - 1$ over $\mathbb{F}_2$ may be regarded as the word $v = \alpha_0 \alpha_1 \cdots \alpha_{n-1}$ of length n in $\mathbb{F}_2^n$.

Note that the leftmost bit corresponds to the constant term, that is, the x^0 term.

If we fix the length n of words, then every word of length n corresponds to exactly one polynomial over $\mathbb{F}_2$ of degree less than or equal to $n - 1$. Thus any code C of length n can be represented as a set of polynomials over $\mathbb{F}_2$ of degree at most $n - 1$.

7.2.2 Example If $C = \{00000, 11100, 00111, 11011\}$, then the words of C are represented by polynomials as follows.

$$00000 \mapsto 0 \quad 11100 \mapsto 1 + x + x^2 \quad 00111 \mapsto x^2 + x^3 + x^4 \quad 11011 \mapsto 1 + x + x^3 + x^4.$$

If we have a higher degree polynomial, we can reduce it to a lower degree one, by taking the remainder after dividing by some fixed polynomial.

7.2.3 Theorem [Division Algorithm] If $f(x), p(x) \in \mathbb{F}_2[x]$ with $p(x) \neq 0$ then there exist unique polynomials

$$q(x), r(x) \in \mathbb{F}_2[x] \quad \text{such that} \quad f(x) = q(x)p(x) + r(x),$$

with $\deg r(x) < \deg p(x)$. Here $q(x)$ is called the *quotient* and $r(x)$ is called the *remainder*.

In particular, if we fix a polynomial $p(x) \in \mathbb{F}_2[x]$ of degree $n > 0$, then for each $f(x) \in \mathbb{F}_2[x]$ we can write

$$f(x) = q(x)p(x) + r(x), \quad \text{with} \quad \deg r(x) \leq n - 1.$$

¹It is convenient to consider the 0 polynomial to have degree $-\infty$ (or some value < 0) to make the statement of the division algorithm work even if $r = 0$

In this way, to each polynomial $f(x)$ we associate a unique polynomial $r(x)$ of degree at most $n-1$, and hence a unique word of length n . Thus, given any polynomial $f(x) \in \mathbb{F}_2[x]$, we can represent $f(x)$ by a unique representative of degree less than n from the set of residue (equivalence) classes of $\mathbb{F}_2[x]$ modulo $p(x)$. We say that $f(x)$ is equivalent to $r(x)$ modulo $p(x)$, and write

$$f(x) \equiv r(x) \pmod{p(x)}.$$

Note that $f(x) \equiv 0 \pmod{p(x)}$ exactly when $f(x)$ is a multiple of $p(x)$. We can avoid polynomial long division when finding $r(x) \pmod{p(x)}$ in the following way.

$$\begin{aligned} p(x) &= p_0 + p_1x + \cdots + p_{n-1}x^{n-1} + x^n \\ \implies x^n &= 1 \cdot p(x) + \left(p_0 + p_1x + \cdots + p_{n-1}x^{n-1} \right) && \text{Add } p(x) + x^n \text{ both sides} \\ \implies x^n &\equiv p_0 + p_1x + \cdots + p_{n-1}x^{n-1} \pmod{p(x)}. \end{aligned}$$

Multiplying by x : $x^{n+1} \equiv p_0x + p_1x^2 + \cdots + p_{n-1}x^n \pmod{p(x)}$. If $p_{n-1} \neq 0$, substitute in for x^n using the previous equation. And so on.

7.2.4 Example Let $p(x) = 1 + x^2 + x^5$. Compute the word v of length $n = 5$ corresponding to $f(x) = 1 + x^4 + x^9 \pmod{p(x)}$.

Solution: Mod $p(x)$ we have $1 + x^2 + x^5 \equiv 0$, so

$$\begin{aligned} x^5 &\equiv 1 + x^2 \\ x^6 &\equiv x + x^3 && \text{multiplying by } x \\ x^7 &\equiv x^2 + x^4 \\ x^8 &\equiv x^3 + (1 + x^2) && \text{since } x^5 \equiv 1 + x^2 \\ &\equiv 1 + x^2 + x^3 \\ x^9 &\equiv x + x^3 + x^4 \end{aligned}$$

So $1 + x^4 + x^9 \equiv 1 + x^4 + (x + x^3 + x^4) \equiv 1 + x + x^3 \mapsto 11010$. *

7.2.5 Definition From now on, we fix $p(x) = 1 + x^n$ to make the correspondence between polynomials and words of length n .

Hence $x^n \equiv 1 \pmod{p(x)}$.

7.3. Introduction to cyclic codes

7.3.1 Definition Let v be a word of length n . The *cyclic shift* of v , denoted $S(v)$, is the word obtained by moving the last bit of v to the beginning.

7.3.2 Example $S(010111) = 101011$, $S(S(010111)) = S(101011) = 110101$.

Note that the cyclic shift preserves the weight of words: $\|v\| = \|S(v)\|$. Also $S(v + w) = S(v) + S(w)$.

7.3.3 Definition A code C is *cyclic* if it closed under cyclic shifts: $(v \in C) \implies (S(v) \in C)$.

7.3.4 Example

- ★ $\{000, 110, 101, 011\}$ is both cyclic and linear.
- ★ $\{000, 100, 011, 111\}$ is linear but not cyclic.
- ★ $\{010, 100, 001\}$ is cyclic but not linear.

7.3.5 Example Find the smallest cyclic linear code C containing the word 001.

Solution: Since C is cyclic, it must contain 010 and 100. But C is linear and contains a basis for $\mathbb{F}_2^3$, so $C = \mathbb{F}_2^3$. ⊛

We are using $p(x) = 1 + x^n$ so $x^n \equiv 1 \pmod{1 + x^n}$. The main reason for using $1 + x^n$ is that if the word v is represented by the polynomial $f(x)$, then the word $S(v)$ is represented by the polynomial $xf(x) \pmod{(x^n + 1)}$:

$$\begin{aligned}
 \text{Word } v &= f_0 & f_1 & f_2 & \cdots & f_{n-1} \\
 f(x) &= f_0 + f_1x + f_2x^2 + \cdots + f_{n-1}x^{n-1} \\
 xf(x) &= f_0x + f_1x^2 + \cdots + f_{n-2}x^{n-1} + f_{n-1}x^n \\
 &\equiv f_0x + f_1x^2 + \cdots + f_{n-2}x^{n-1} + f_{n-1} \cdot 1 \\
 &\hspace{15em} (\text{mod } x^n + 1) \\
 &= f_{n-1} + f_0x + f_1x^2 + \cdots + f_{n-2}x^{n-1} \\
 \text{Word } &f_{n-1} & f_0 & f_1 & \cdots & f_{n-2} &= S(v).
 \end{aligned}$$

7.3.6 Example Let $f(x) = 1 + x + x^5$ and use $p(x) = 1 + x^7$ to make the correspondence between polynomials and words of length 7. Then

$$\begin{aligned}
 f(x) = 1 + x + x^5 &\mapsto 1100010 = w \\
 xf(x) = x + x^2 + x^6 &\mapsto 0110001 = S(w) \\
 x^2f(x) = 1 + x^2 + x^3 &\mapsto 1011000 = S(S(w))
 \end{aligned}$$

A set of polynomials corresponds to a linear code if it is closed under addition.

By definition of cyclic, a set of polynomials modulo $p(x) = 1 + x^n$ corresponds to a cyclic code if the set is closed under multiplication by x .

A code C is both linear and cyclic if it is closed under both addition, and under multiplication by x . In this case, if $f = f_0 + f_1x + \cdots + f_tx^t$, $f_i \in \{0, 1\}$ is any word (not necessarily a codeword) and $c = c(x) \in C$ then $xc \in C$, and hence $x^2c = x(xc) \in C$, and so $x^3c = x(x^2c) \in C$ etc. Thus each of f_0c , f_1xc , f_2x^2c , $\dots$ are in C . Since C is also closed under addition, $f_0c + f_1xc + f_2x^2c + \cdots = f \cdot c \in C$. Thus a linear cyclic code is closed under addition, and also under multiplication by any polynomial (not just by polynomials in C).

From now on, we shall refer to the set of polynomials corresponding to a code as the code itself and will use polynomial notation and usual bitwise notation interchangeably.

7.3.7 Definition Let $C \neq \{0\}$ be a cyclic code of length n . A generator $g(x)$ for C is a non-zero polynomial in C of least degree.7.3.8 Example Let C be the cyclic linear code $C = \{0000, 1010, 0101, 1111\}$, so

$$C = \{0, 1 + x^2, x + x^3, 1 + x + x^2 + x^3\}.$$

By definition, $1 + x^2 = 1010$ is a generator for C . Show that every word in C is a multiple mod $(1 + x^4)$ of this generator.

Solution:

$$\begin{aligned} 0 &= 0(1 + x^2) \\ 1 + x^2 &= 1(1 + x^2) \\ x + x^3 &= x(1 + x^2) \\ 1 + x + x^2 + x^3 &= (1 + x)(1 + x^2) \end{aligned}$$

Note that $x^2(1 + x^2) = x^2 + x^4 \equiv 1 + x^2$ so multiplying $(1 + x^2)$ by any x^n will produce a word in C , and since C is linear, multiplying $(1 + x^2)$ by any $f(x)$ will still produce a word in C . $\circledast$

7.3.9 Theorem Let $C \neq \{0\}$ be a cyclic linear code of length n . Then:

- (a) C has a unique generator $g(x)$. The generator satisfies $\deg g(x) \leq n - 1$.
- (b) Every polynomial $f(x) \in C$ can be written in the form $f(x) = q(x)g(x)$ for a unique $q(x) \in \mathbb{F}_2[x]$, and $\deg q(x) \leq n - 1 - \deg g(x)$.

Proof (a) C is cyclic and contains at least one non-zero polynomial, so we can select from C a nonzero polynomial $g(x)$ of smallest degree, say degree $m < n$. By definition, $g(x)$ is a generator for C .

Assume C contains another polynomial $h(x)$ of degree m . Since $x^m + x^m = 0$, we have $\deg(g(x) + h(x)) < m$. But as C is linear, $g(x) + h(x) \in C$. Thus, as $g(x)$ has minimal degree among non-zero polynomials in C we must have $g(x) + h(x) = 0$ (the zero polynomial), so $g(x) = h(x)$ and C has a unique generator.

(b) Now let $f(x)$ be any polynomial in C . By the Division Algorithm there exist unique polynomials $q(x)$ and $r(x)$ in $\mathbb{F}_2[x]$ such that

$$f(x) = q(x)g(x) + r(x)$$

with $\deg(r(x)) < \deg(g(x))$. Since C is cyclic and linear, $q(x)g(x) \in C$. Since C is linear and hence closed under addition, $r(x) = f(x) + q(x)g(x) \in C$. By the minimality of the degree of the generator $g(x)$ it follows that $r(x)$ is the zero polynomial $r(x) = 0$ so $f(x) = q(x)g(x)$. Thus $\deg f(x) = \deg q(x) + \deg g(x)$, and since $f(x) \in C$, we have $\deg f(x) \leq n - 1$. Hence $\deg q(x) \leq (n - 1) - \deg g(x)$. $\square$

The codes corresponding with generators $g(x) = 1$ and $g(x) = 1 + x^n = \mathbf{0}$ are called *improper*. The code generated by 1 contains $1000 \cdots$ and its shifts, and linear combinations of these, so is all of $\mathbb{F}_2^n$. The code generated by $1 + x^n$ is the code generated by the zero word, namely $\{0\}$.

7.3.10 Example Find the smallest linear cyclic code C of length 6 containing $x^2 + x^5$. Find a generator $g(x)$ for C , and represent each word in C as a multiple of $g(x)$.

Solution: C contains $x^2 + x^5$, corresponding to 001001. C is also closed under shifts, so contains 100100, 010010. These 3 vectors are linearly independent (distinct, non-zero, and the 3 of them do not sum to $\mathbf{0}$), so C contains all 8 linear combinations of them. So far:

$$C \supseteq \{000000, 001001, 100100, 010010, 101101, 011011, 110110, 111111\}.$$

Since this set is still closed under shifts, it is a cyclic linear code, so this is all of C .

In polynomial terms, $C = \{0, x^2 + x^5, 1 + x^3, x + x^4, 1 + x^2 + x^3 + x^5, x + x^2 + x^4 + x^5, 1 + x + x^3 + x^4, 1 + x + x^2 + x^3 + x^4 + x^5\}$. So the generator is $g(x) = 1 + x^3$.

$q(x)$	$q(x)g(x)$
0	0
1	$1 + x^3$
x	$x + x^4$
$1 + x$	$1 + x + x^3 + x^4$
x^2	$x^2 + x^5$
$1 + x^2$	$1 + x^2 + x^3 + x^5$
$x + x^2$	$x + x^2 + x^4 + x^5$
$1 + x + x^2$	$1 + x + x^2 + x^3 + x^4 + x^5$

⊛

7.4. Generating matrices for linear cyclic codes

We now find a generating matrix for a linear cyclic code C .

7.4.1 Theorem If a linear cyclic code C of length n has generator $g(x)$ of degree $n-k$, then $\{g(x), xg(x), \dots, x^{k-1}g(x)\}$ forms a basis for C , and hence $\dim C = k$.

Proof First note that the polynomials $g(x), xg(x), \dots, x^{k-1}g(x)$ are all distinct modulo $(1+x^n)$, since each has degree one greater than its predecessor and $x^{k-1}g(x)$ has degree $n-1$ (so no reduction mod $x^n + 1$ occurs.) To see that these polynomials form a basis for C , we must show that they both span C and are linearly independent.

To see that they span C , let $f(x) \in C$. Then by Theorem 7.3.9, $f(x) = q(x)g(x)$ for some $q(x) \in \mathbb{F}_2[x]$ where $\deg q(x) \leq n-1 - \deg g(x)$ so $\deg q(x) \leq k-1$. Let

$$q(x) = q_0 + q_1x + \dots + q_{k-1}x^{k-1}$$

where $q_0, q_1, \dots, q_{k-1} \in \mathbb{F}_2$, and $q_{k-1} = 0$ is allowed. Then

$$\begin{aligned} f(x) &= q(x)g(x) \\ &= (q_0 + q_1x + \dots + q_{k-1}x^{k-1})g(x) \\ &= q_0g(x) + q_1xg(x) + q_2x^2g(x) + \dots + q_{k-1}x^{k-1}g(x). \end{aligned}$$

Thus any $f(x) \in C$ can be written as a linear combination of the elements of the set

$$\{g(x), xg(x), \dots, x^{k-1}g(x)\}$$

so this set spans C .

To see that the set of polynomials is linearly independent, let

$$g(x) = \alpha_0 + \alpha_1x + \dots + \alpha_{n-k-1}x^{n-k-1} + x^{n-k},$$

and associate with each polynomial in $\{g(x), xg(x), \dots, x^{k-1}g(x)\}$ the corresponding word of length n . Writing these words as the rows of a matrix gives

$$\begin{pmatrix} \alpha_0 & \alpha_1 & \alpha_2 & \dots & \dots & \dots & \alpha_{n-k-1} & 1 & 0 & 0 & \dots & 0 \\ 0 & \alpha_0 & \alpha_1 & \dots & \dots & \dots & \alpha_{n-k-2} & \alpha_{n-k-1} & 1 & 0 & \dots & 0 \\ 0 & 0 & \alpha_0 & \alpha_1 & \dots & \dots & \dots & \alpha_{n-k-2} & \alpha_{n-k-1} & 1 & \dots & 0 \\ \vdots & & & & \vdots & \vdots & & & & & & \vdots \\ 0 & 0 & \dots & \dots & \alpha_0 & \alpha_1 & \dots & \dots & \dots & \alpha_{n-k-2} & \alpha_{n-k-1} & 1 \end{pmatrix}.$$

It is clear from the form of the matrix that the rows are linearly independent and so $\{g(x), xg(x), \dots, x^{k-1}g(x)\}$ is a linearly independent set.

Thus $\{g(x), xg(x), \dots, x^{k-1}g(x)\}$ is a basis for C , so the dimension of C is k . $\square$

7.4.2 Corollary Let C be a linear cyclic code of length n with generator polynomial $g(x)$ of degree $n - k$. Then the $k \times n$ matrix

$$\mathbf{G} = \begin{pmatrix} \leftarrow & g(x) & \rightarrow \\ \leftarrow & xg(x) & \rightarrow \\ \leftarrow & x^2g(x) & \rightarrow \\ \vdots & & \\ \leftarrow & x^{k-1}g(x) & \rightarrow \end{pmatrix}$$

is a generating matrix for C (where the polynomials are interpreted as row vectors).

7.4.3 Example Let C be the linear cyclic code of length $n = 7$, with generator $g(x) = 1 + x + x^3$. Find a generating matrix for C .

Solution: The degree of g is $n - k = 3$, so $k = 4$, and a basis for C is

$$\begin{aligned} g(x) &= 1 + x + x^3 \\ xg(x) &= x + x^2 + x^4 \\ x^2g(x) &= x^2 + x^3 + x^5 \\ x^3g(x) &= x^3 + x^4 + x^6 \end{aligned}$$

so a generating matrix for C is

$$\mathbf{G} = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}.$$

Check that C is equivalent to a $(7, 4, 3)$ Hamming code, Example 5.2.6. $\otimes$

It turns out that all the Hamming codes are (equivalent to) cyclic codes. It can also be shown that the Golay code C_{23} is equivalent to the linear cyclic code with generator $1 + x^2 + x^4 + x^5 + x^6 + x^{10} + x^{11}$.

The previous exercise shows one benefit of using a cyclic code. Rather than storing the entire generating matrix for the code, it is only necessary to store the generator polynomial.

Let C be a linear cyclic (n, k) code with generator $g(x) = g_0 + g_1x + \dots + g_{n-k}x^{n-k}$. The message word $a = a_0a_1 \dots a_{k-1}$ of length k can be represented by the message polynomial $a(x) = a_0 + a_1x + \dots + a_{k-1}x^{k-1}$.

To encode a , we form $a\mathbf{G}$:

$$\begin{aligned} (a_0 \ a_1 \ a_2 \ \dots \ a_{k-1}) & \begin{pmatrix} g_0 & g_1 & g_2 & \dots & g_{n-k-1} & g_{n-k} & 0 & 0 & \dots & 0 \\ 0 & g_0 & g_1 & \dots & g_{n-k-2} & g_{n-k-1} & g_{n-k} & 0 & \dots & 0 \\ 0 & 0 & g_0 & g_1 & \dots & g_{n-k-2} & g_{n-k-1} & g_{n-k} & \dots & 0 \\ \vdots & & & & \vdots & & & & & \vdots \\ 0 & 0 & 0 & \dots & g_0 & g_1 & \dots & g_{n-k-2} & g_{n-k-1} & g_{n-k} \end{pmatrix} \\ &= (a_0g_0 \quad a_1g_0 + a_0g_1 \quad a_2g_0 + a_1g_1 + a_0g_2 \quad \dots \quad a_{k-1}g_{n-k}). \end{aligned}$$

This is the polynomial

$$\begin{aligned} & a_0g_0 + (a_1g_0 + a_0g_1)x + (a_2g_0 + a_1g_1 + a_0g_2)x^2 + \dots + a_{k-1}g_{n-k}x^{n-1} \\ &= (a_0 + a_1x + \dots + a_{k-1}x^{k-1}) (g_0 + g_1x + \dots + g_{n-k}x^{n-k}). \end{aligned}$$

Thus $a(x)$ is encoded as $a(x)g(x)$ to give the codeword $c(x)$ of length n . If we receive codeword $c(x)$, we divide $c(x)$ by $g(x)$ to recover $a(x)$.

7.4.4 Example Consider the linear cyclic code C from Example 7.4.3 with messages encoded using the procedure above. Determine the codeword corresponding to the message word 1010. Determine the message word corresponding to the codeword 1100101.

Solution: $g(x) = 1 + x + x^3$, $a(x) = 1 + x^2$ $a(x)g(x) = 1 + x + x^2 + x^5$, so the codeword is 1110010.

1100101 corresponds to $1 + x + x^4 + x^6 = (1 + x^3)(1 + x + x^3)$, so the message word was $1 + x^3$, so 1001. *

We can also find the generating matrix in standard form of a linear cyclic code.

7.4.5 Theorem Let $g(x)$ be the generator of a linear cyclic code C of length n . For $n - k \leq i \leq n - 1$, let r_i be the length $n - k$ word with $x^i \equiv r_i(x) \pmod{g(x)}$. Then a generating matrix for C is:

$$\mathbf{G} = \left(\begin{array}{c|c} & r_{n-k} \\ & \vdots \\ \mathbf{I}_k & r_{n-1} \end{array} \right).$$

Proof By the Division Algorithm, $x^i = q(x)g(x) + r_i(x)$, so $x^i + r_i(x) = q(x)g(x)$. Thus $g(x)$ divides $x^i + r_i(x)$, so $c_i(x) = x^i + r_i(x)$ is a polynomial corresponding to a codeword in C . Note that $\deg r_i(x) < \deg g(x) = n - k$ so r_i “fits” in the space in the matrix $\mathbf{G}$. Also, since $\deg r_i(x) < \deg g(x)$ and $\deg x^i \geq \deg g(x)$, we have $c_i(x) = x^i + r_i(x) \neq 0$. Listing the $c_i(x)$ in a matrix shows they are linearly independent:

$$\left(\begin{array}{c|c} r_{n-k} & \\ \vdots & \\ r_{n-1} & \mathbf{I}_k \end{array} \right).$$

As this matrix has k linearly independent rows, each of which is a codeword of C , it follows that it is a generating matrix for C . But C is cyclic, so we can perform a cyclic shift k times on this matrix. Hence another generating matrix for C is the one given in the statement of the theorem. □

7.4.6 Example Find $\mathbf{G}$ in standard form for the linear cyclic code C of length 7 generated by $g(x) = 1 + x + x^3$ (Exercise 7.4.3).

Solution: Here $n = 7$, $n - k = 3$, so $k = 4$ and

$$\begin{array}{ll} x^{n-k} = x^3 \equiv 1 + x \pmod{g(x)} & r_3 = 110 \\ x^4 = x(1 + x) \equiv x + x^2 \pmod{g(x)} & r_4 = 011 \\ x^5 = x(x + x^2) = x^2 + x^3 \equiv 1 + x + x^2 \pmod{g(x)} & r_5 = 111 \\ x^6 = x(1 + x + x^2) = x + x^2 + x^3 \equiv 1 + x^2 \pmod{g(x)} & r_6 = 101. \end{array}$$

Hence a generating matrix in standard form is:

$$\mathbf{G} = \left(\begin{array}{ccccccc} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{array} \right).$$

*

7.5. Finding a generator polynomial for a linear cyclic code

It turns out that only some $g(x)$ can ever occur as generators. The following theorem shows how to find all linear cyclic codes of a given length, by determining which polynomials are generators of a linear cyclic code of that length.

7.5.1 Theorem The polynomial $g(x)$ is a generator for a linear cyclic code of length n iff

$$g(x) \text{ divides } (1 + x^n).$$

Proof $\Rightarrow$ Assume that C is a linear cyclic code of length n with generator $g(x)$. By the Division Algorithm, there exist unique polynomials $q(x)$ and $r(x)$ such that

$$1 + x^n = q(x)g(x) + r(x),$$

where $r(x) = 0$ or $\deg r(x) < \deg g(x)$. Since algebra is done modulo $(1 + x^n)$ in C , we have $r(x) \equiv q(x)g(x)$. Therefore $r(x) \in C$. By the minimality of the degree of the generator $g(x)$, we must have $r(x) = 0$. Thus $1 + x^n \equiv q(x)g(x)$, so $g(x)$ divides $(1 + x^n)$.

$\Leftarrow$ Conversely, suppose that $g(x)$ divides $(1 + x^n)$, say $g(x)q(x) = 1 + x^n$. The case $g(x) = 1 + x^n$ is trivial, as noted previously. Otherwise $\deg g(x) = n - k$, for some k with $0 < k \leq n$. Thus $\{g(x), xg(x), \dots, x^{k-1}g(x)\}$ is a linearly independent set (Theorem 7.4.1). Define C to be the span of $\{g(x), xg(x), \dots, x^{k-1}g(x)\}$. Thus C is a linear code. To see that C is cyclic, let $f(x)$ be a linear combination of the above polynomials, so

$$f(x) = f_0 g(x) + f_1 xg(x) + f_2 x^2 g(x) + \dots + f_{k-1} x^{k-1} g(x), \quad f_i \in \{0, 1\}.$$

Then

$$xf(x) = f_0 xg(x) + f_1 x^2 g(x) + f_2 x^3 g(x) + \dots + f_{k-2} x^{k-1} g(x) + f_{k-1} x^k g(x).$$

All the terms are in C except possibly $x^k g(x)$. However,

$$x^k g(x) \equiv (1 + x^n) + x^k g(x) = q(x)g(x) + x^k g(x) = (q(x) + x^k)g(x),$$

and

$$\deg q(x) = n - \deg g(x) = k,$$

so $\deg(q(x) + x^k) \leq k - 1$. Therefore $f_{k-1} x^k g(x)$ is a linear combination of the polynomials in our basis for C and so $xf(x) \in C$. Therefore, C is cyclic. $\square$

Thus, to find all linear cyclic codes of a given length n , find all factors of $1 + x^n$. Each such factor is a generator for a linear cyclic code of length n . (The codes corresponding to the trivial factors 1 and $1 + x^n$ of $1 + x^n$ are not interesting, as noted previously.)

7.5.2 Definition The linear cyclic codes of length n generated by 1 and by $1 + x^n$ are called the *improper cyclic codes*. Other cyclic linear codes of length n are called *proper cyclic codes*.

7.5.3 Definition A polynomial $p(x) \in \mathbb{F}_2[x]$ is *irreducible* if $p(x) = q(x)s(x)$ implies either $q(x)$ or $s(x)$ has degree 0 (that is, the only factors of $p(x)$ are 1 and itself).

To find all factors of $1 + x^n$, we first find all irreducible factors of $1 + x^n$, then we form all possible products of these factors (except for the products 1 and $1 + x^n$). Each such product is the generator for a proper linear cyclic code of length n .

Here is a table of all small degree irreducible polynomials in $\mathbb{F}_2[x]$.

degree	all irreducible polynomials of this degree
1	$x, \quad x + 1$
2	$x^2 + x + 1$
3	$x^3 + x^2 + 1, x^3 + x + 1$
4	$x^4 + x^3 + x^2 + x + 1, \quad x^4 + x^3 + 1, \quad x^4 + x + 1$
5	$x^5 + x^2 + 1, \quad x^5 + x^3 + 1,$ $x^5 + x^a + x^b + x^c + 1$, for $(a, b, c) = (3, 2, 1), (4, 2, 1), (4, 3, 1), (4, 3, 2)$.

If n is even, say $n = 2m$, then

$$1 + x^n = 1 + x^{2m} = (1 + x^m)^2.$$

So we only need to factorize $1 + x^n$ with n odd.

Here is a table of factorizations of $(1 + x^n)$ into irreducible polynomials, for small odd values of n :

$$\begin{aligned}
1 + x &= 1 + x \\
1 + x^3 &= (1 + x)(1 + x + x^2) \\
1 + x^5 &= (1 + x)(1 + x + x^2 + x^3 + x^4) \\
1 + x^7 &= (1 + x)(1 + x + x^3)(1 + x^2 + x^3) \\
1 + x^9 &= (1 + x)(1 + x + x^2)(1 + x^3 + x^6) \\
1 + x^{15} &= (1 + x)(1 + x + x^2)(1 + x + x^4)(1 + x^3 + x^4)(1 + x + x^2 + x^3 + x^4) \\
1 + x^{23} &= (1 + x)(1 + x + x^5 + x^6 + x^7 + x^9 + x^{11})(1 + x^2 + x^4 + x^5 + x^6 + x^{10} + x^{11})
\end{aligned}$$

7.5.4 Example Find all proper linear cyclic codes of length $n = 3$.

Solution: For $n = 3$, the factorization of $1 + x^3$ into irreducible factors is

$$1 + x^3 = (1 + x)(1 + x + x^2).$$

Thus there are two proper cyclic codes of length 3. One has generator $g(x) = 1 + x$ so $n - k = 1$ and $k = 2$. A generating matrix is

$$\mathbf{G} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}.$$

This code is $C = \{000, 110, 011, 101\}$.

The other code has generator $g(x) = 1 + x + x^2$, so $n - k = 2$ and $k = 1$. A generating matrix is $\begin{pmatrix} 1 & 1 & 1 \end{pmatrix}$, so the code is $C = \{000, 111\}$. ⊗

7.5.5 Example Find the dimensions of all proper linear cyclic codes of length $n = 6$.

Solution: We have the following factorization into irreducible polynomials:

$$x^6 + 1 = (x^3 + 1)^2 = (x + 1)^2(x^2 + x + 1)^2.$$

Thus $g(x)$ can have any degree from 0 to 6. So there is a proper linear cyclic code of length 6 with dimensions 1, 2, 3, 4, 5. ⊗

7.5.6 Example Does there exist a linear cyclic code of length 15 and dimension 4?

Solution: If $k = 4$ then $n - k = 11$ so we need a factor of $(1 + x^{15})$ of degree 11. The irreducible factors of $(1 + x^{15})$ have degrees 1, 2, 4, 4, 4, so take $1 + 2 + 4 + 4 = 11$. For example, $g(x) = (1 + x)(1 + x + x^2)(1 + x + x^4)(1 + x^3 + x^4) = 1 + x + x^5 + x^6 + x^{10} + x^{11}$. So, yes. $\otimes$

7.5.7 Example Construct a linear cyclic code of length 15 with 4 codewords.

Solution: If $k = 2$ then $n - k = 13$ so we need a factor of $(1 + x^{15})$ of degree 13. Take factors of degrees 4, 4, 4, 1:

$$\begin{aligned} g(x) &= (1 + x)(1 + x + x^4)(1 + x^3 + x^4)(1 + x + x^2 + x^3 + x^4) \\ &= 1 + x + x^3 + x^4 + x^6 + x^7 + x^9 + x^{10} + x^{12} + x^{13}. \end{aligned}$$

Modulo $g(x)$ we have:

$$\begin{aligned} x^{13} &\equiv 1 + x + x^3 + x^4 + x^6 + x^7 + x^9 + x^{10} + x^{12} \\ x^{14} &\equiv (x + x^2 + x^4 + x^5 + x^7 + x^8 + x^{10} + x^{11}) + \\ &\quad (1 + x + x^3 + x^4 + x^6 + x^7 + x^9 + x^{10} + x^{12}) \\ &\equiv 1 + x^2 + x^3 + x^5 + x^6 + x^8 + x^9 + x^{11} + x^{12} \end{aligned}$$

$$\therefore \mathbf{G} = \left(\begin{array}{cc|cccccccccccccc} 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \end{array} \right).$$

So $C = \{101101101101101, 011011011011011, 110110110110110, 000000000000000\}$ is a linear cyclic code of length 15 with 4 codewords. $\otimes$

7.6. Error detection and correction using cyclic codes

We now have enough background to discuss burst error correction. Cyclic codes are suitable for correcting cyclic burst error patterns.

A burst is a pattern $b = 1 * \dots * 1$. A burst error pattern has the form $0 \dots 0 b 0 \dots 0$. Thus:

7.6.1 Definition

★ Suppose $e(x) = x^j f(x)$ for some j and for some polynomial

$$f(x) = 1 + \alpha_1 x + \alpha_2 x^2 + \dots + \alpha_{m-1} x^{m-1} + x^m.$$

Then the *burst length* of $e(x)$ is $\deg f(x) + 1 = m + 1$.

★ $e(x)$ has *cyclic burst length* ℓ if the minimum degree of $x^j f(x) \bmod (1 + x^n)$ for $j = 0, 1, \dots, n - 1$ is $\ell - 1$.

7.6.2 Example For the word $e = 00010100$ we have the polynomial $e(x) = x^3 + x^5 = x^3(1 + x^2)$, so we have $j = 3$ and $f(x) = 1 + x^2$. Thus e has burst length $2 + 1 = 3$.

The word $e = 1000110$ has polynomial form $e(x) = 1 + x^4 + x^5$ with $j = 0$ and $f(x) = 1 + x^4 + x^5$ so e has burst length 6. However, $x^3(1 + x^4 + x^5) \bmod (1 + x^7) = 1 + x + x^3$ (or equivalently 1101000) so e has cyclic burst length 4.

As noted earlier (Theorem 7.4.5), any linear cyclic code C with generator polynomial $g(x)$ of degree $n - k$ has a generating matrix of the form:

$$\mathbf{G} = \left(\begin{array}{c|ccc} & r_{n-k} & & \\ \mathbf{I}_k & \vdots & & \\ & r_{n-1} & & \end{array} \right)$$

where $x^i \equiv r_i(x) \pmod{g(x)}$. Thus, by Algorithm 4.6.13, a parity check matrix is

$$(7.6.1) \quad \mathbf{H} = \left(\begin{array}{c} r_{n-k} \\ r_{n-k+1} \\ \vdots \\ r_{n-1} \\ \hline \mathbf{I}_{n-k} \end{array} \right), \quad r_i(x) \equiv x^i \pmod{g(x)}.$$

In Example 7.1.7 we considered the existence of a linear code of length 15 and dimension 9 that is 3 cyclic burst error correcting (but not 3-error correcting). We now construct such a code.

7.6.3 Example We construct a cyclic linear code of length 15 and dimension 9. As generator we need a polynomial $g(x)$ of degree $n - k = 15 - 9 = 6$. The factors of $1 + x^{15}$ are of degree 1, 2, 4, 4, 4, so $g(x)$ must be the product of the degree 2 and a degree 4 factor of $1 + x^{15}$. We shall take C to be the cyclic linear code of length 15 with generator

$$g(x) = (1 + x + x^2)(1 + x^3 + x^4) = 1 + x + x^2 + x^3 + x^6.$$

We find a parity check matrix for C .

We have $n - k = 6$. We find $x^i \pmod{g(x)}$ for $i = n - k$ to $n - 1$, so for $i = 6, 7, \dots, 14$:

$$\begin{aligned} x^6 &\equiv 1 + x + x^2 + x^3 && \rightarrow r_6 = 111100 \\ x^7 &\equiv x + x^2 + x^3 + x^4 && \rightarrow r_7 = 011110 \\ x^8 &\equiv x^2 + x^3 + x^4 + x^5 && \rightarrow r_8 = 001111 \\ x^9 &\equiv x^3 + x^4 + x^5 + (1 + x + x^2 + x^3) \\ &= 1 + x + x^2 + x^4 + x^5 && \rightarrow r_9 = 111011 \quad \text{etc.} \end{aligned}$$

$$\therefore \mathbf{H} = \left(\begin{array}{cccccc} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 \\ \hline & & & & \mathbf{I}_6 & \end{array} \right)$$

Claim: C is a 3 cyclic burst error correcting code.

The proof is a lengthy calculation. We require all error patterns of length 15 with cyclic burst length at most 3 to have different syndromes (hence be in different cosets). The syndrome is the sum of the rows of $\mathbf{H}$ corresponding to the locations of the 1s in the error pattern. So we require all these sums to be different.

An error pattern with cyclic burst length 1 has syndrome equal to a row of $\mathbf{H}$. (Eg $100 \dots 0 \longleftrightarrow x^0$ whose syndrome is row 1 of $\mathbf{H}$.) Check that all the rows of $\mathbf{H}$ are different. So all cyclic burst length 1 patterns are correctable.

Error patterns with cyclic burst length 2 contain the burst 11 so have syndromes equal to sums of 2 (cyclically) adjacent rows of H . Eg the error pattern $1100 \cdots 0 \longleftrightarrow 1 + x$ has syndrome the sum of the first two rows of H , etc. A calculation reveals (check) that the error patterns with cyclic burst length 2 and their syndromes are:

error pattern	syndrome	error pattern	syndrome
$1 + x$	100010	$x^8 + x^9$	011001
$x + x^2$	010001	$x^9 + x^{10}$	110000
$x^2 + x^3$	110100	$x^{10} + x^{11}$	011000
$x^3 + x^4$	011010	$x^{11} + x^{12}$	001100
$x^4 + x^5$	001101	$x^{12} + x^{13}$	000110
$x^5 + x^6$	111010	$x^{13} + x^{14}$	000011
$x^6 + x^7$	011101	$1 + x^{14}$	111101
$x^7 + x^8$	110010		

Error patterns with cyclic burst length 3 contain burst 111 or 101, so have syndromes either the sum of 3 (cyclically) adjacent rows of H , or the sum of a row of H with the row that is 2 rows (cyclically) below it.

Check that the error patterns with cyclic burst length 3 and their syndromes are:

error pattern	syndrome	error pattern	syndrome
$1 + x + x^2$	101101	$1 + x^2$	110011
$x + x^2 + x^3$	101010	$x + x^3$	100101
$x^2 + x^3 + x^4$	010101	$x^2 + x^4$	101110
$x^3 + x^4 + x^5$	110110	$x^3 + x^5$	010111
$x^4 + x^5 + x^6$	011011	$x^4 + x^6$	110111
$x^5 + x^6 + x^7$	110001	$x^5 + x^7$	100111
$x^6 + x^7 + x^8$	100100	$x^6 + x^8$	101111
$x^7 + x^8 + x^9$	010010	$x^7 + x^9$	101011
$x^8 + x^9 + x^{10}$	001001	$x^8 + x^{10}$	101001
$x^9 + x^{10} + x^{11}$	111000	$x^9 + x^{11}$	101000
$x^{10} + x^{11} + x^{12}$	011100	$x^{10} + x^{12}$	010100
$x^{11} + x^{12} + x^{13}$	001110	$x^{11} + x^{13}$	001010
$x^{12} + x^{13} + x^{14}$	000111	$x^{12} + x^{14}$	000101
$1 + x^{13} + x^{14}$	111111	$1 + x^{13}$	111110
$1 + x + x^{14}$	100011	$x + x^{14}$	011111

Check that all of these 61 syndromes are different. (Which 3 words are not syndromes?) So C is a 3 cyclic burst error correcting code. Compare Example 7.1.7.

Currently there do not seem to be very efficient algorithms to actually find cyclic burst error correcting codes with good properties. Most examples have been found by computer search.

As the example shows, the creation and storage of an SDA can be costly. Luckily, the structure of cyclic linear codes allows us to decode received words containing cyclic burst errors without

an SDA. We use the parity check matrix (Equation 7.6.1):

$$\mathbf{H} = \begin{pmatrix} r_{n-k} \\ r_{n-k+1} \\ \vdots \\ r_{n-1} \\ \hline \mathbf{I}_{n-k} \end{pmatrix}.$$

For a received word w with error pattern e and syndrome s , let $w(x)$, $e(x)$, $s(x)$ be the corresponding polynomials. For $0 \leq i \leq n-1$, let w_i and e_i be the words corresponding to $x^i w(x)$ and $x^i e(x) \pmod{1+x^n}$, respectively:

$$w_i \equiv x^i w(x), \quad e_i \equiv x^i e(x) \pmod{1+x^n} \quad 0 \leq i \leq n-1$$

Define $s_i = w_i \mathbf{H}$ as the syndrome of w_i for $0 \leq i \leq n-1$, called the *shifted syndrome*. So s_0 is the usual syndrome of w .

7.6.4 Theorem Let C be a t -cyclic burst error correcting linear cyclic (n, k) -code. Suppose $w + e$ is a codeword, where $e \in \mathfrak{B}_n(t)$. Then there exists i with $0 \leq i \leq n-1$ such that $e(x) \equiv x^{k-i} s_i(x) \equiv x^{n+k-i} s_i(x) \pmod{1+x^n}$.

(The last congruence holds as $x^n \equiv 1$. Use whichever of $x^{k-i} s_i(x) \equiv x^{n+k-i} s_i(x)$ avoids negative exponents.)

So assuming $e \in \overline{\mathfrak{B}}_n(t)$, the error pattern is one of the shifted syndromes $s_i(x)$.

Theorem 7.6.4 shows how to decode a received word w : calculate the syndromes of the cyclic shifts of w until a syndrome S_i is found which is a burst error pattern with burst of length at most t ; then shift to give the most likely cyclic burst error pattern e .

Proof Since C is cyclic and $w + e \in C$, all the shifts $w_i + e_i$ are in C for each i . Thus w_i and e_i are always in the same coset of C , and hence

$$s_i = w_i \mathbf{H} = e_i \mathbf{H}.$$

We are assuming that e has cyclic burst length at most t , and $t \leq n-k$ by Lemma 7.1.8. So e must contain at least k cyclically adjacent 0s, so we can find i with $0 \leq i \leq n-1$ for which the shifted word e_i has 0s in its first k places. Say $e_i = (\mathbf{0}_k \mid f_i)$.

Then $s_i = e_i \mathbf{H} = (\mathbf{0}_k \mid f_i) \begin{pmatrix} \vdots \\ \hline \mathbf{I}_{n-k} \end{pmatrix} = f_i$. Thus $e_i = (\mathbf{0}_k \mid s_i)$.

So $x^i e(x) = e_i(x) = x^k s_i(x) = x^{n+k} s_i(x)$ (as $x^n \equiv 1$). Thus $e(x) \equiv x^{k+n-i} s_i(x) \equiv x^{k-i} s_i(x)$. $\square$

Note that since e_i is a cyclic burst error pattern with burst of length at most t , so is s_i .

The drawback to this method is the number of syndromes that need to be calculated. For each received word w , we have to calculate all the shifted w_i , and then find their syndromes s_i . The next lemma gives us a way to simplify this process, by showing how to calculate s_{i+1} from s_i .

7.6.5 Lemma If s is the syndrome of w with corresponding polynomial $s(x)$, then $xs(x) \pmod{g(x)}$ is the polynomial corresponding to the syndrome of w_1 , so $s_1(x) = xs(x)$. The syndrome of the shift is the shift of the syndrome.

Proof It is convenient to label the rows of $\mathbf{H}$ from 0 to $n-1$ in this proof. For $0 \leq i \leq n-1$, let h_i be the i th row of $\mathbf{H}$ and let $h_i(x)$ be its polynomial representation. Recall from Equation 7.6.1 that the rows of $\mathbf{H}$ come from $x^{n-k}, x^{n-k-1}, \dots, x^{n-1}$ and then $1, x, x^2, \dots, x^{n-k-1}$. Thus

$$h_{i+1}(x) \equiv xh_i(x) \pmod{g(x)}.$$

(Also $xh_{n-1} \equiv h_0$.)

Let $w = a_0a_1 \dots a_{n-1}$ be a received word. The syndrome of w is the sum of corresponding rows of $\mathbf{H}$: $s = a_0h_0 + \dots + a_{n-1}h_{n-1}$. Now $w_1(x) = xw(x)$ is the cyclic shift of w , so $w_1 = a_{n-1}a_0a_1 \dots a_{n-2}$, with syndrome $s_1 = a_{n-1}h_0 + a_0h_1 + a_1h_2 + \dots + a_{n-2}h_{n-1}$.

In polynomial terms, using $h_1(x) = xh_0(x)$, $h_2(x) = xh_1(x)$ etc:

$$\begin{aligned} s_1(x) &\equiv a_{n-1}h_0(x) + a_0h_1(x) + a_1h_2(x) + \dots + a_{n-2}h_{n-1}(x) \\ &\equiv a_{n-1}xh_{n-1}(x) + a_0xh_0(x) + a_1xh_1(x) + \dots + a_{n-2}xh_{n-2}(x) \\ &\equiv x \sum_{i=0}^{n-1} a_i h_i(x) = xs(x) \pmod{g(x)}. \quad \square \end{aligned}$$

This means that s_1 (which we defined as the syndrome of w_1) can easily be computed from s (the syndrome of w) by $s_1 \equiv xs(x) \pmod{g(x)}$. Repeating this process, each s_i is $x^i s(x) \pmod{g(x)}$.

Summary: Let C be a t burst error correcting cyclic linear code, with generator $g(x)$. Suppose we receive w .

7.6.6 Algorithm [Cyclic Burst Error Correction]

- Calculate $\mathbf{H}$ using Equation 7.6.1
- Calculate the syndrome $s = w\mathbf{H}$, and write it as a polynomial.
- For $i = 0, 1, \dots, n-1$ calculate the *shifted syndrome* $s_i = x^i \cdot s \pmod{g(x)}$.
- Stop as soon as s_i has burst length $\leq t$. If none of the s_i satisfy this, too many errors have occurred and we cannot correct.
- Otherwise, with the s_i found in (d), let

$$e(x) \equiv x^{n+k-i} s_i(x) \equiv x^{k-i} s_i(x) \pmod{1+x^n}.$$

This is the desired burst error pattern.

- Correct $w(x)$ to $w(x) + e(x)$.

7.6.7 Example The Hamming bound can be used to show that there is no 3-error correcting $(15, 7)$ -code. In contrast, it can be shown that the cyclic linear code C generated by $g(x) = 1 + x^4 + x^6 + x^7 + x^8$ is a $(15, 7)$ -code that is 4-burst error correcting. For the code C , use Algorithm 7.6.6 to decode the received word

$$w = 110\ 000\ 100\ 000\ 101$$

Solution: First we find $\mathbf{H}$. We calculate $r_8, r_9, \dots, r_{14}$ in Equation 7.6.1. Working mod $g(x)$:

$$\begin{array}{llll} x^8 & \equiv & 1 + x^4 + x^6 + x^7 & \longleftrightarrow & 10001011 \\ x^9 & \equiv & x + x^5 + x^7 + (1 + x^4 + x^6 + x^7) & & \\ & \equiv & 1 + x + x^4 + x^5 + x^6 & \longleftrightarrow & 11001110 \\ x^{10} & \equiv & x + x^2 + x^5 + x^6 + x^7 & \longleftrightarrow & 01100111 \\ x^{11} & \equiv & 1 + x^2 + x^3 + x^4 & \longleftrightarrow & 10111000 \\ x^{12} & \equiv & x + x^3 + x^4 + x^5 & \longleftrightarrow & 01011100 \\ x^{13} & \equiv & x^2 + x^4 + x^5 + x^6 & \longleftrightarrow & 00101110 \\ x^{14} & \equiv & x^3 + x^5 + x^6 + x^7 & \longleftrightarrow & 00010111 \end{array}$$

$$\therefore \mathbf{H} = \left(\begin{array}{cccccccc} 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline & & & & \mathbf{I}_8 & & & \end{array} \right).$$

To decode the received word $w = 110\ 000\ 100\ 000\ 101$, calculate the syndrome

$$s = w\mathbf{H} = 01010111, \quad \text{so} \quad s(x) = x + x^3 + x^5 + x^6 + x^7.$$

This has burst length 7, too large. Calculate the shifts of s :

$$s_1(x) = xs(x) = x^2 + x^4 + x^6 + x^7 + (1 + x^4 + x^6 + x^7) = 1 + x^2$$

with burst length 3, success! So form $e = x^{k-1}s_1(x) = x^6(1 + x^2) = 000\ 000\ 101\ 000\ 000$.

$$\therefore w + e = 110\ 000\ 001\ 000\ 101.$$

⊛

This concludes the course.